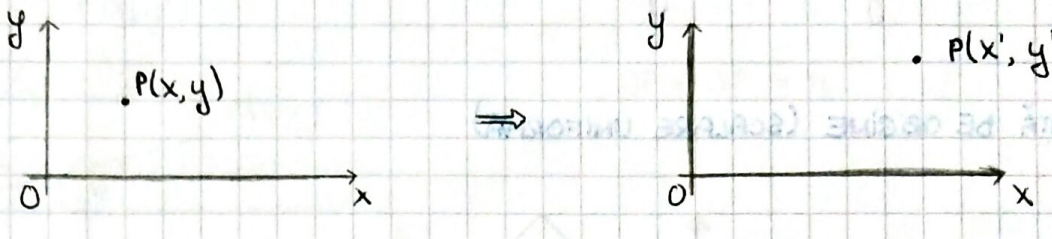
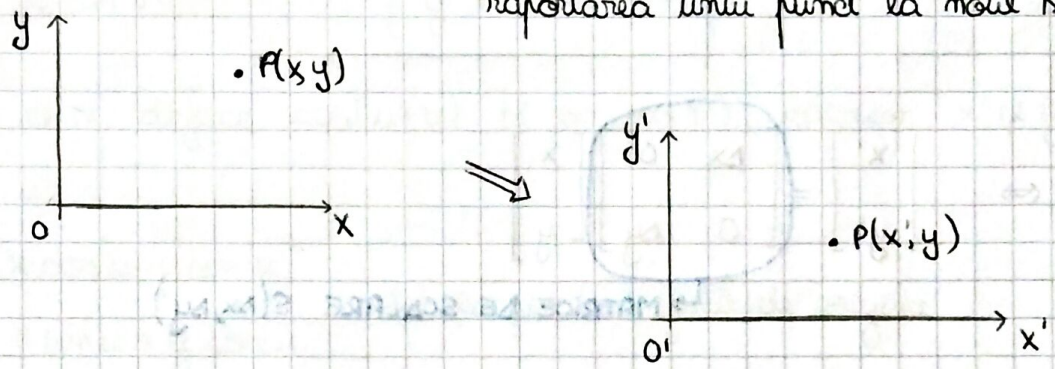
27.09.2017

CURS 1

TRANSFORMARE GEOMETRICĂ = transformarea unui punct raportat la un sistem de coordonate fix



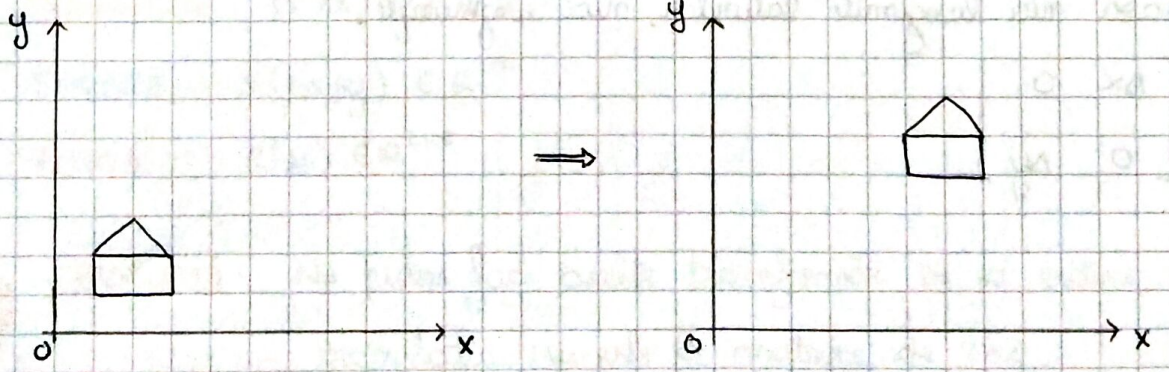
TRANSFORMĂRI ALE SISTEMELOR DE COORDONATE = transformarea sistemului de coordonate și raportarea unui punct la noul sistem de coordonate



TRANSFORMĂRI GEOMETRICE ELEMENTARE

- TRANSLAȚIA
- SCALARE FAȚĂ DE ORIGINE
- ROTAȚIA FAȚĂ DE ORIGINE
- OGLINDIREA
- FORFECAREA

- **TRANSLAȚIA**



t_x → vector de translație pe axa Ox

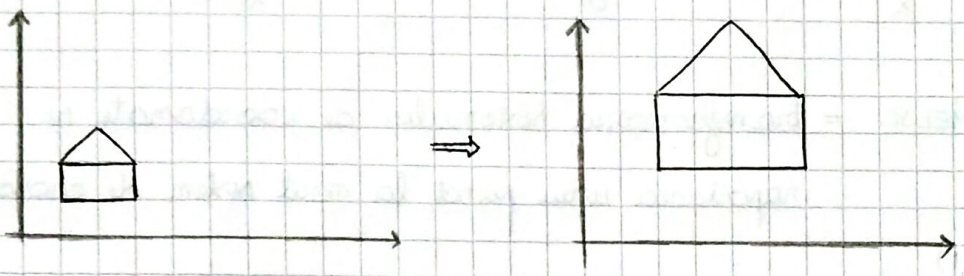
t_y → vector de translație pe axa Oy

$$\begin{cases} x' = x + t_x \\ y' = y + t_y \end{cases} \iff \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

→ MATRICEA DE TRANSLAȚIE $T(t_x, t_y)$

! Se păstrează lungimile și unghiurile!

- **SCALARE FAȚĂ DE ORIGINE (SCALARE UNIFORMĂ)**

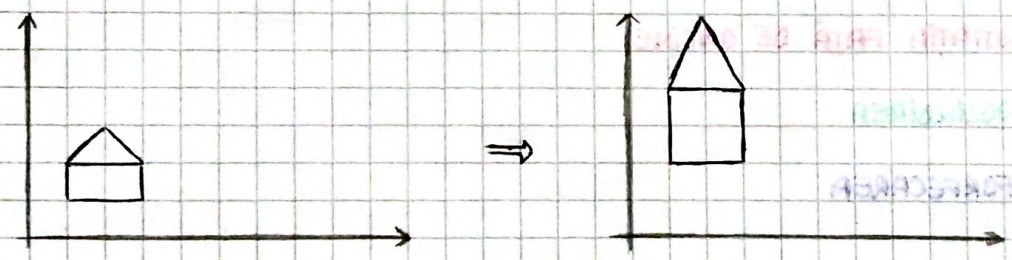


$$\begin{cases} x' = x \cdot s_x \\ y' = y \cdot s_y \end{cases} \iff \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

↳ MATRICE DE SCALARE $S(s_x, s_y)$

! Nu păstrează lungimile laturilor, dar păstrează unghiurile.

- **SCALARE FAȚĂ DE ORIGINE (SCALARE NEUNIFORMĂ)**



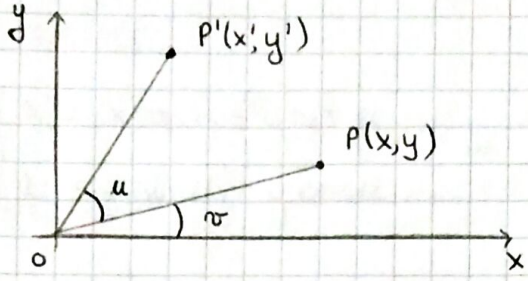
! Nu se păstrează nici lungimile laturilor, nici unghiurile.

$$S(s_x, s_y) = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

- ROTAȚIA FAȚĂ DE ORIGINE

ROTAȚIE POZITIVĂ
$R(u)$

ROTAȚIE NEGATIVĂ
$R(-u)$



$OP = OP' = r$

$$\begin{cases} x = r\cos v \\ y = r\sin v \end{cases} \qquad \begin{cases} x' = r\cos(u+v) \\ y' = r\sin(u+v) \end{cases}$$

După ce se desface $\cos(u+v)$ și $\sin(u+v)$, obținem x' și y' în funcție de x, y și u:

$$\begin{cases} x' = x\cos u - y\sin u \\ y' = x\sin u + y\cos u \end{cases}$$

Rotație pozitivă față de origine

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos u & -\sin u \\ \sin u & \cos u \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

↳ MATRICEA DE ROTAȚIE $R(u)$

! Sunt păstrate lungimile și unghiurile

TRANSLAȚIE : $T(t_x, t_y) \in \mathbb{R}^{2\times 1}$

SCALARE : $S(s_x, s_y) \in \mathbb{R}^{2\times 2}$

ROTAȚIE : $R(u) \in \mathbb{R}^{2\times 2}$

! PROBLEMĂ : Nu putem face aceste transformări în ce ordine dorim, deoarece translația nu este o matrice de 2×2

SOLUȚIE : coordonate omogene (x, y, w)

COORDONATE OMOGENE

$(x, y) \longrightarrow (x_w, y_w, w)$

(x, y) ↑ sistem de coordonate carteziam

(x_w, y_w, w) ↑ sistem de coordonate omogen

$$\begin{cases} x_w = x \cdot w \\ y_w = y \cdot w \\ w \in \mathbb{R}^* \end{cases}$$

$(1, 2)$ carteziam

- $(1, 2, 1)$
- $(2, 4, 2)$ ← omogen
- $(0{,}5, 1, 0{,}5)$
- etc.

TRANSFORMĂRI GRAFICE
- GEOMETRICE
- ALE SISTEMULUI DE COORDONATE

$(x, y, 0) \rightarrow$ PUNCTE LA ∞ (DIRECȚIA UNUI VECTOR)

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \rightarrow$$ REPREZENTARE PRIN VECTORI COLOANĂ

⇒ Se modifică matricele de la translație, scalare, rotație.

- **TRANSLAȚIE**

$$\begin{cases} x' = x + tx \\ y' = y + ty \end{cases} \Rightarrow \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

MATRICE DE TRANSLAȚIE

$T(tx, ty) = T_{3,3}$

! Nu mai contează ordinea!

- **SCALARE**

$$\begin{cases} x' = x \cdot sx \\ y' = y \cdot sy \end{cases} \Rightarrow \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

↳ MATRICE DE SCALARE
$S(sx, sy) = S_{3,3}$

- **ROTAȚIE**

$$\begin{cases} x' = x\cos u - y \sin u \\ y' = x \sin u + y \cos u \end{cases} \Rightarrow \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos u & -\sin u & 0 \\ \sin u & \cos u & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

↳ MATRICE DE ROTAȚIE
$R(u) = R_{3,3}$

COMPUNEREA TRANSFORMĂRILOR

Contează ordinea în care aplicăm transformările, deoarece se obțin rezultate diferite

Ex: translație și după rotație ≠ rotație și după translație

SOLUȚIE: Matricea cea mai din dreapta trebuie să fie prima transformare care se aplică.

- Translație și după rotație:

$$R(u) \cdot \left(T(tx, ty) \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \right)$$

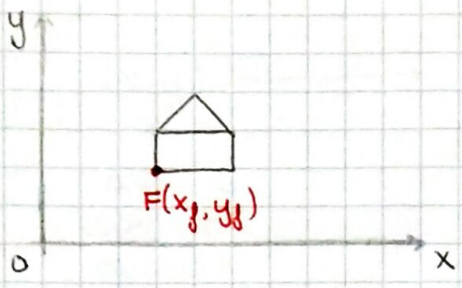
- Rotație și după translație:

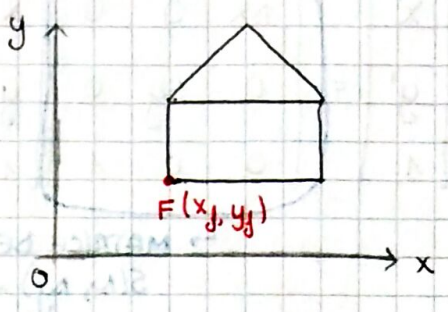
$$T(tx, ty) \cdot \left(R(u) \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \right)$$

TRANSFORMĂRI GEOMETRICE INVERSE

- **TRANSLAȚIE INVERSĂ**: $T^{-1}(tx, ty) = T(-tx, -ty)$
- **SCALARE INVERSĂ FAȚĂ DE ORIGINE**: $S^{-1}(sx, sy) = S\left(\frac{1}{sx}, \frac{1}{sy}\right)$
- **ROTAȚIE INVERSĂ FAȚĂ DE ORIGINE**: $R^{-1}(u) = R(-u)$

SCALAREA FAȚĂ DE UN PUNCT FIX





METODA 1:

- PAS 1: Scalare față de origine $\Rightarrow$ $F'(x_f \cdot sx \ , \ y_f \cdot sy)$
- PAS 2: Translație astfel încât F' obținut după scalare să ajungă înapoi în F

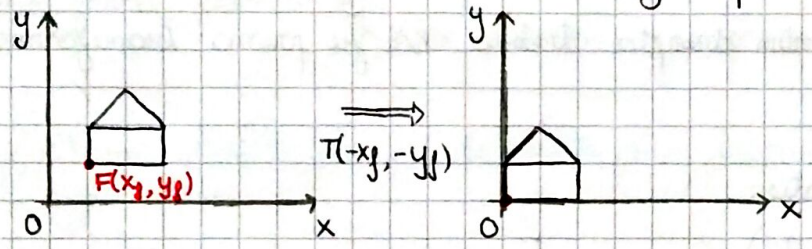
$$t_x = -(x_f \cdot sx - x)$$

$$t_y = -(y_f \cdot sy - y)$$

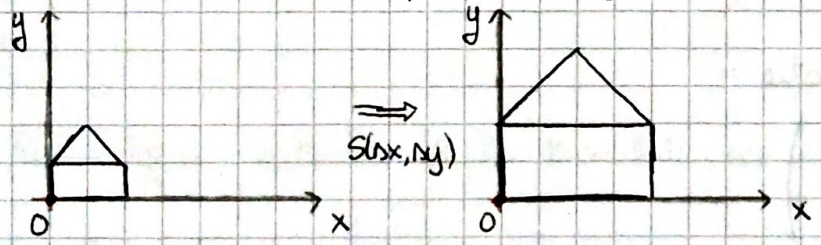
METODA 1 ESTE INEFICIENTĂ (greu de calculat cu cât trebuie să translatăm)!

METODA 2:

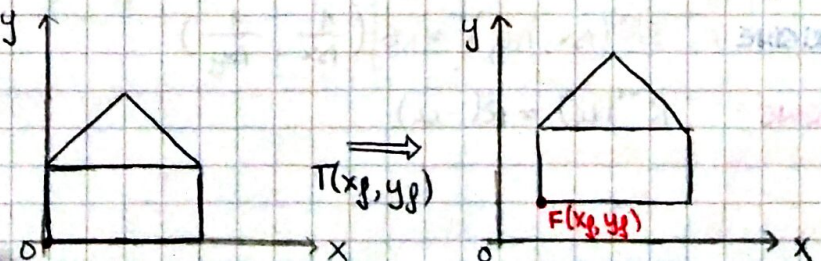
- PAS 1: mutarea punctului F în origine prin translație $T(-x_f, -y_f)$



- PAS 2: scalare în raport cu originea $S(sx, sy)$



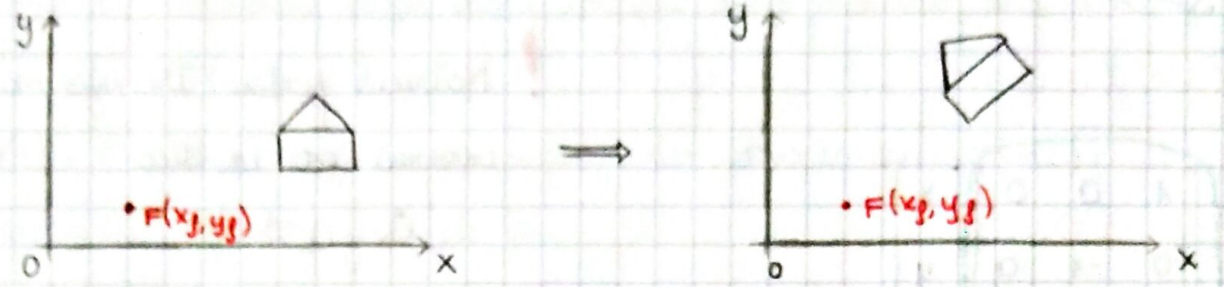
- PAS 3: mutăm înapoi în punctul F (nu trebuie să mai calculăm factorul de translație, deoarece îl cunoaștem $T(x_f, y_f)$)



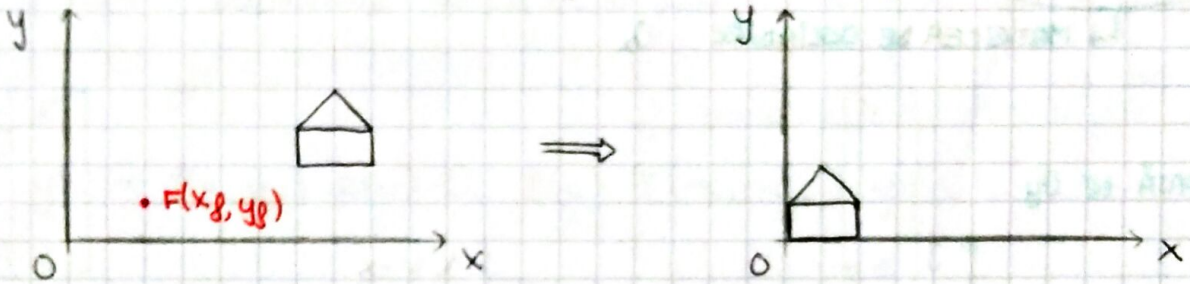
$$\Rightarrow \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = T(x_f, y_f) \cdot S(s_x, s_y) \cdot T(-x_f, -y_f) \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

↑ prima transformare care se aplică

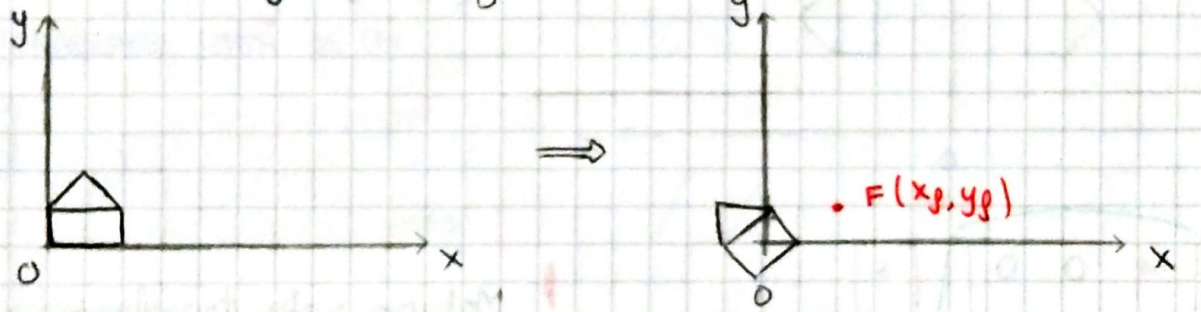
- **ROTAȚIE FAȚĂ DE UN PUNCT FIX**



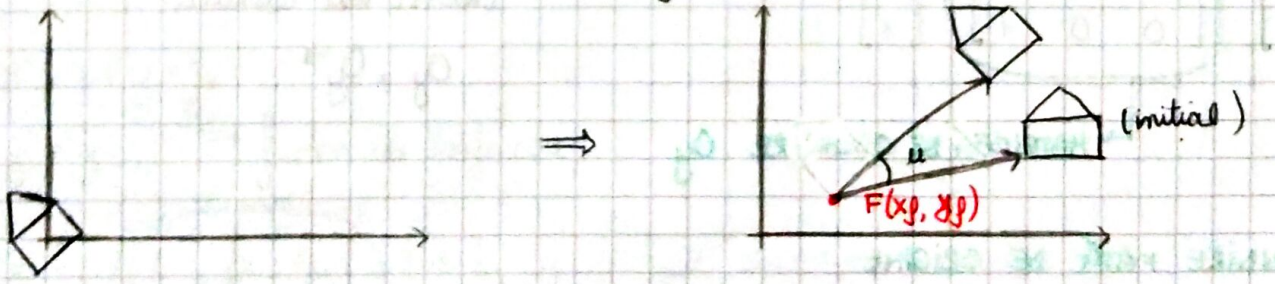
- PAS 1: translație astfel încât punctul F să ajungă în origine $T(-x_f, -y_f)$



- PAS 2: Rotație față de origine $R(u)$

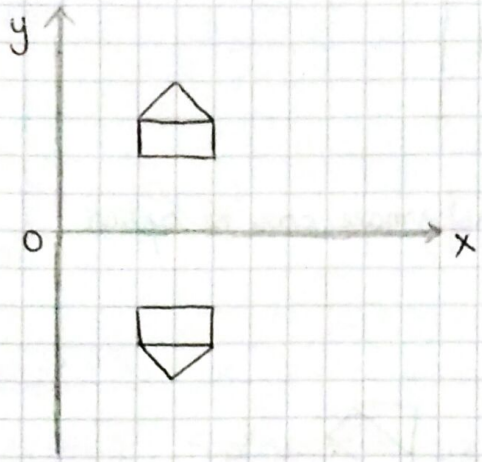


- PAS 3: translație astfel încât să ajungem înapoi în F $T(x_f, y_f)$



$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = T(x_f, y_f) \cdot R(u) \cdot T(-x_f, y_f) \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- OGLINDIRE FAȚĂ DE O_x



$$\begin{cases} x' = x \\ y' = -y \end{cases}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

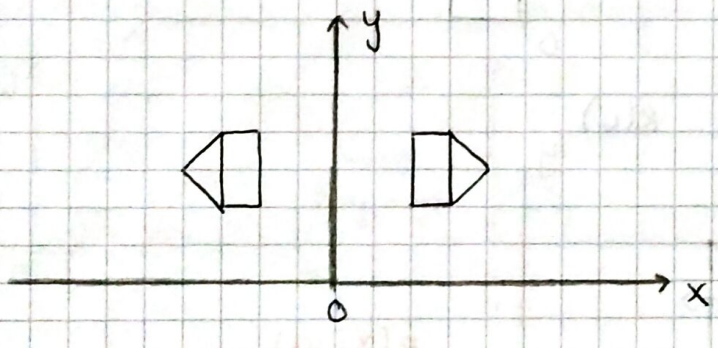
↳ MATRICEA DE OGLINDIRE O_x

! Matricea pentru transformarea inversă este identică.

$$O_x = O_x^{-1}$$

! $O_x = S(1, -1)$

- OGLINDIRE FAȚĂ DE O_y



$$\begin{cases} x' = -x \\ y' = y \end{cases}$$

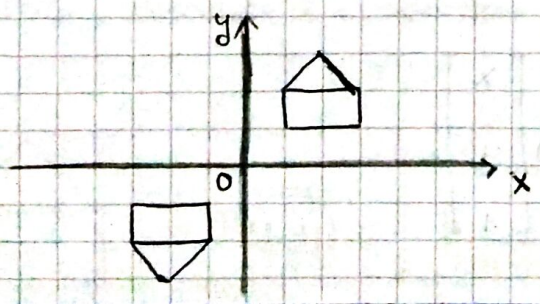
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

↳ MATRICEA DE OGLINDIRE O_y

! Matricea pentru transformarea inversă este identică.

$$O_y = O_y^{-1}$$

- OGLINDIRE FAȚĂ DE ORIGINE



$$\begin{cases} x' = -x \\ y' = -y \end{cases}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

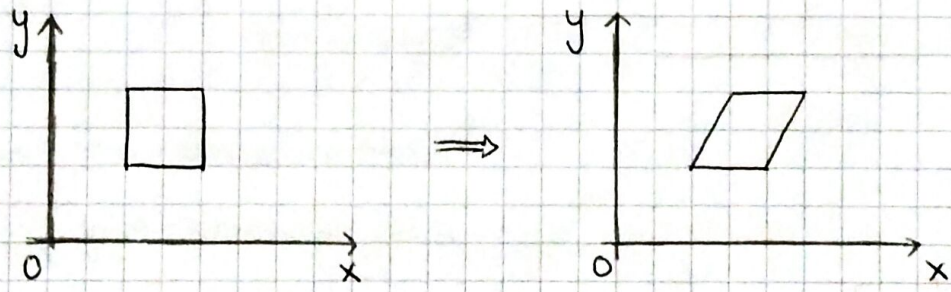
↳ MATRICEA DE OGLINDIRE O_0

! $O_0 = O_0^{-1}$

- **OGLINDIREA FAȚĂ DE O DREAPTĂ OARECARE**
  - PASUL 1: Translație astfel încât dreapta trece prin origine ($T(-x_1, -y_1)$ sau $T(-x_2, -y_2)$)
  - PASUL 2: $R(-u)$ astfel încât suprapunem dreapta cu axa Ox
  - PASUL 3: Oglindire față de Ox
  - PASUL 4: Rotația inversă de la pasul 2: $R^{-1}(-u) = R(u)$
  - PASUL 5: Translația inversă de la pasul 1: $T^{-1}(-x_1, -y_1) = T(x_1, y_1)$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = T(x_1, y_1) \cdot R(u) \cdot O_x \cdot R(-u) \cdot T(-x_1, -y_1) \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- **FORFECARE FAȚĂ DE Ox**



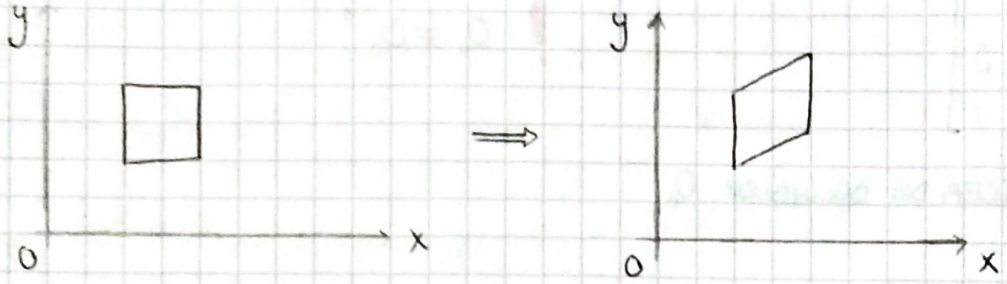
$$\begin{cases} x' = x + y \cdot f_x \\ y' = y \end{cases}$$

↳ factor de forfecare

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & f_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

↓ MATRICE DE FORFECARE F_x

- FORFECARE FAȚĂ DE Oy



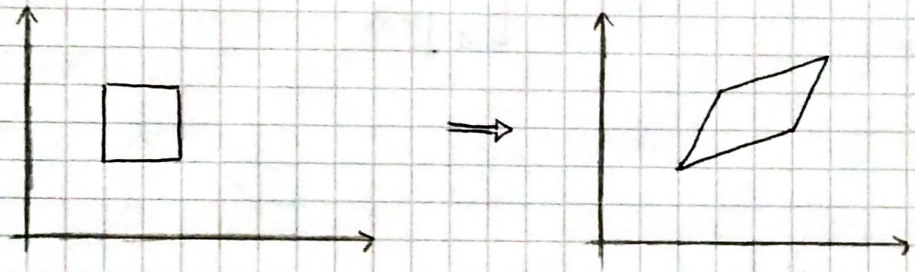
$$\begin{cases} x' = x \\ y' = y + x \cdot \underline{f_y} \end{cases}$$

factor de forfecare pe Oy

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ f_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

↳ MATRICE DE FORFECARE F_y

- FORFECARE FAȚĂ DE AMBELE AXE



$$\begin{cases} x' = x + y \cdot f \\ y' = y + x \cdot f_y \end{cases}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & f_x & 0 \\ f_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

↳ MATRICE DE FORFECARE F_{xy}

- Păstrează lungimile laturilor: translație, rotație, oglindire, forfecare
- Păstrează unghiurile: translație, oglindire,
- Păstrează paralelismul: translație, scalare, rotație, oglindire, forfecare

TRANSFORMĂRI AFINE → păstrează paralelismul dreptelor

$xOy : P(x,y)$

$xOy \xrightarrow[T(tx,ty)]{} x'Oy' \Rightarrow x'Oy' : P(x',y')$

Calculăm x' și y' în funcție de x, y, tx, ty:

$$\begin{cases} x = tx + x' \\ y = ty + y' \end{cases} \Rightarrow \begin{cases} x' = x - tx \\ y' = y - ty \end{cases}$$

⇒ Transformările sistemelor de coordonate sunt inversele Transformărilor geometrice

$T(tx, ty)$ pt. sistemul de coordonate = $\underbrace{T^{-1}(tx, ty)}$ transformare geometrică aplicată asupra punctelor raportată la un sistem fix de coordonate

- Rotație:

$x'Oy' \begin{cases} x' = OP\cos v \\ y' = OP\sin v \end{cases}$ $xOy \begin{cases} x = OP\cos(u+v) \\ y = OP\sin(u+v) \end{cases}$

$$\Rightarrow \begin{cases} x' = x\cos u + y\sin u \\ y' = -x\sin u + y\cos u \end{cases}$$

04.10.2017

CURS 2

TRANSFORMĂRI 2D GRAFICE
- transformări geometrice (sistem de coordonate fix)
- transformări ale sistemelor de coordonate (sistemul de coordonate se modifică)

TRANSFORMĂRI GEOMETRICE

$T(t_x, t_y)$, $S(s_x, s_y)$, $R(u)$, O_{ox}, O_{oy}, O_o, F

$S(s_x, s_y)$ față de origine

$M_{3\times3}$

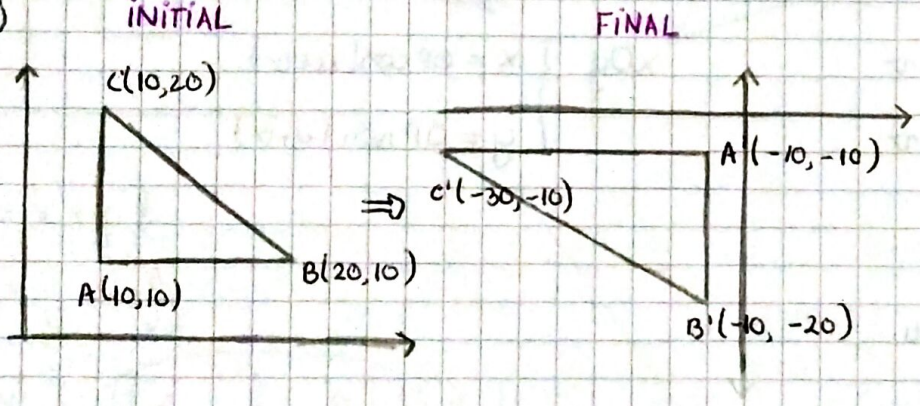
TRANSFORMĂRI SISTEME COORDONATE

Sunt inversele transformărilor geometrice.

! **PENTRU EXAMEN**

Să se scrie transformările necesare pentru a obține figura:

(EX. 1)



1) O_o

2) Scalare față de A'

METODA 1

2a) T astfel încât $A' \to O$ $(T(10,10))$

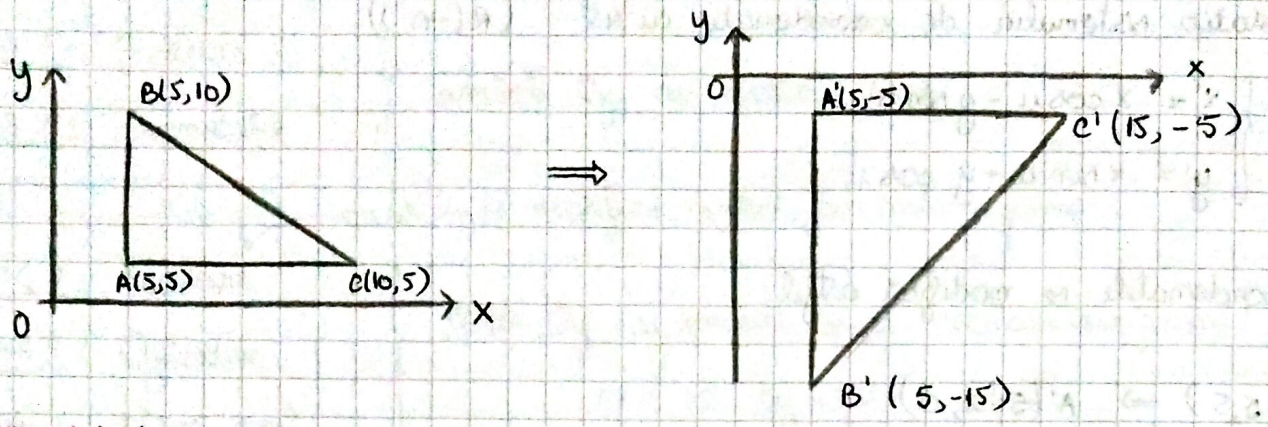
2b) $S(2,1)$

2c) $T^{-1}(10,10) = T(-10,-10)$

METODA 2

1) T în origine
2) Scalare
3) Translație finală

(EX. 2) Transformările necesare



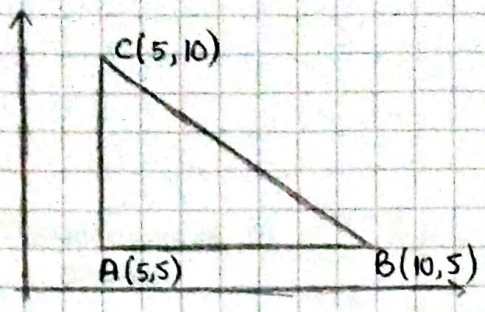
METODA 1

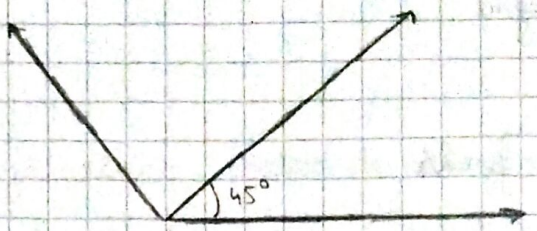
- PAS 1: Oglindire O_{Ox}
- PAS 2: Translație în origine
- PAS 3: Scalare
- PAS 4: Translație inversă pas 2

METODA 2

- PAS 1: Translație în origine
- PAS 2: Scalare
- PAS 3: Translație inversă
- PAS 4: Oglindire

Rotim sistemul de coordonate cu 45°. Vrem să aflăm noile coordonate ale vârfurilor triunghiului





1. Rotația sistemului de coordonate cu 45° $(R(-45°))$

$$\begin{cases} x' = x\cos u - y\sin u \\ y' = x\sin u + y\cos u \end{cases}$$

Coordonatele se modifică astfel:

$A(5,5) \Rightarrow A'(5\sqrt{2}, 0)$

$B(10,5) \Rightarrow B'\left(\frac{15\sqrt{2}}{2}, -\frac{5\sqrt{2}}{2}\right)$

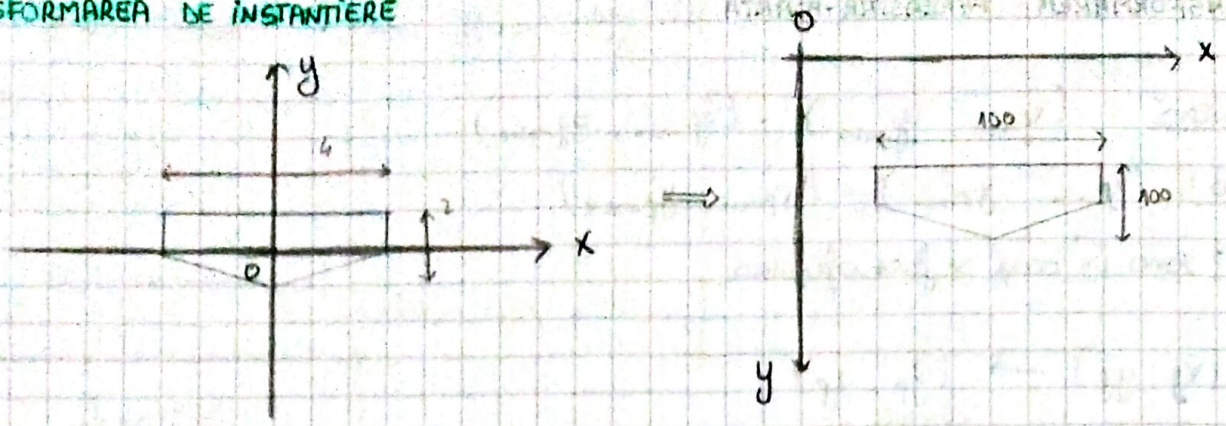
$C(5,10) \Rightarrow C'($

Dacă vrem să ducem triunghiul cu vârful A în origine în noul sistem

2. Translatare în origine $T(-5\sqrt{2}, 0)$

$\Rightarrow$ Matricea de transformare: $T(-5\sqrt{2}, 0) \cdot R(45°)$

TRANSFORMAREA DE INSTANȚIERE



PAS 1: Scalare
PAS 2: Translație
} ambele Oy au același sens

↳ ca să nu se modifice centrul, fac scalarea prima

PAS 1: Scalare
PAS 2: Oglindire
PAS 3: T(300, 200)
} axele Oy au sensuri opuse (scalare neuniformă)

$\Delta x = 25 \quad \Delta y = 50$

Ox: $4 \to 100$ Oy: $2 \to 100$

scalare uniformă: $\Delta x = \Delta y = 50$

$(x, y) \longrightarrow (x', y')$

sistem de coordonate obiect — sistem de coordonate global

$$\begin{cases} x' = x \cdot \Delta x + t_x \\ y' = -y \cdot \Delta y + t_y \end{cases}$$

Matricea de transformare: $M = \underbrace{T(300, 200)}_{T(t_x, t_y)} \cdot \underbrace{O_{ox} \cdot S(25, 25)}_{S(\Delta x, \Delta y)}$

$\Updownarrow$

$S(\Delta x, -\Delta y) = S(25, -25)$

$$T = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}$$

$$\Rightarrow TS = \begin{pmatrix} \Delta x & 0 & t_x \\ 0 & -\Delta y & t_y \\ 0 & 0 & 1 \end{pmatrix}$$

$$S = \begin{pmatrix} \Delta x & 0 & 0 \\ 0 & \Delta y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

TRANSFORMAREA FEREASTRĂ-POARTĂ

Fereastră: $(x_{f\min}, y_{f\min}) - (x_{f\max}, y_{f\max})$

Poartă: $(x_{p\min}, y_{p\min}) - (x_{p\max}, y_{p\max})$

↳ zona în care se face afișarea

$(x_f, y_f) \longrightarrow (x_p, y_p)$

fereastră — poartă

$$\frac{x_f - x_{f\min}}{x_{f\max} - x_{f\min}} = \frac{x_p - x_{p\min}}{x_{p\max} - x_{p\min}}$$

Analog pentru y.

$$\frac{x_p - x_{p\min}}{x_f - x_{f\min}} = \frac{x_{p\max} - x_{p\min}}{x_{f\max} - x_{f\min}}$$

factor de scalare Δx

$$x - x_{\min} = x_f \Delta x - x_{f\min} \Delta x$$

$$x_p = x_f \Delta x \underbrace{- x_{f\min} \Delta x + x_{f\min}}_{t_x}$$

$$\begin{cases} x_p = x_f \Delta x + t_x \\ y_p = y_f \Delta y + t_y \end{cases}$$

! Avem nevoie de o scalare și o translație pentru a aduce spațiul real (fereastra) în spațiul unde se face afișarea (poarta).

⇒ Transformarea fereastră-poartă:

$$\begin{pmatrix} \Delta x & 0 & t_x \\ 0 & \Delta y & t_y \\ 0 & 0 & 1 \end{pmatrix}$$

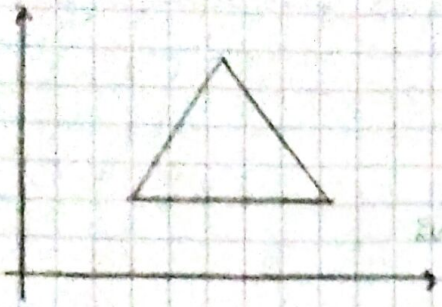
Fereastra: $(0,0) - (1,1)$

Poarta: $(0,0) - (400, 200)$

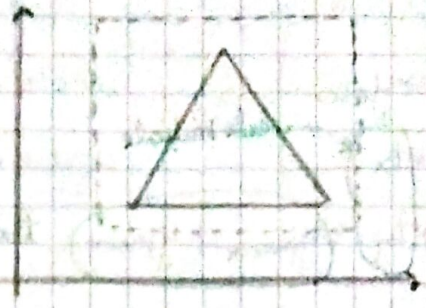
t'_x = Translație suplimentară pentru centrarea în poartă

$$t'_x = \left[(x_{p\max} - x_{p\min}) - \underbrace{(x_{f\max} - x_{f\min}) \cdot \Delta x}_{\text{cât ocupă desenul pe } Ox} \right] / 2$$

$$t_y' = [(y_{pmax} - y_{pmin}) - (y_{fmax} - y_{fmin})/s_y]/2$$

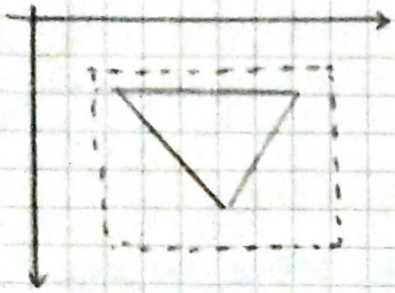


fereastră



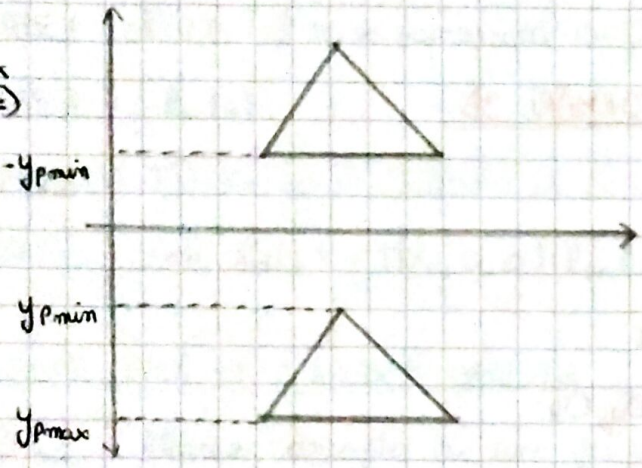
poartă

S+T ⟹

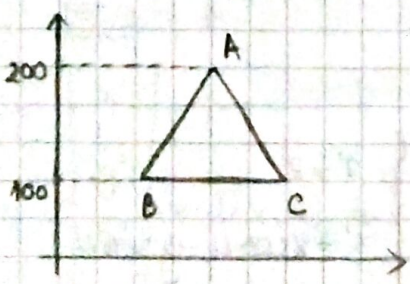


$$\begin{cases} x_p = x_f s_x + t_x \\ y_p = y_f s_y + t_y \end{cases}$$

Oax ⟹



$$t_y' = y_{pmax} + y_{pmin}$$



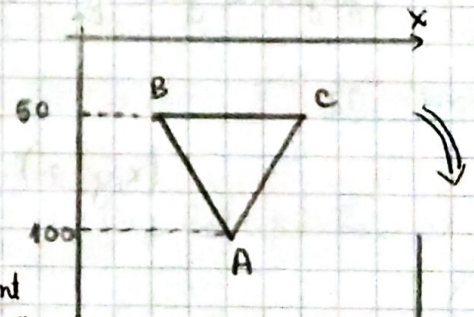
SISTEM REAL (FEREASTRĂ)

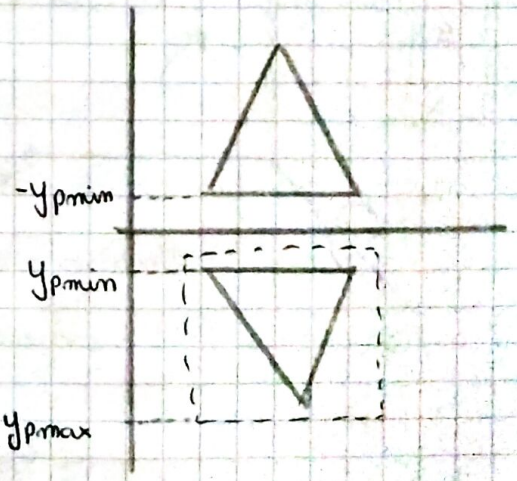
⟹

$x_p = x_f s_x + t_x$

$y_p = y_f s_y + t_y$

(fără să țin cont de semnul pentru Oy din poartă)





Trebuie să translatăm astfel încât baza triunghiului să ajungă pe dreapta de y_{max} a portii

$t_y = y_{pmin} + y_{pmax}$

$x_p = x_f \cdot s_x + t_x$ ← cele inițiale

$y_p = y_f \cdot s_y + t_y + (y_{pmax} + y_{pmin})$ translație suplimentară

$$\begin{cases} x_p = x_f \cdot s_x + t_x \\ y_p = -y_f \cdot s_y + y_{pmin} + y_{pmax} - t_y \end{cases}$$

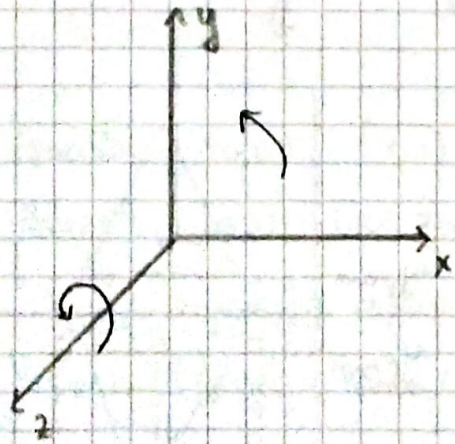
Matricea de transformare este:

$$M = \begin{pmatrix} s_x & 0 & t_x \\ 0 & -s_y & y_{pmax} + y_{pmin} - t_y \\ 0 & 0 & 1 \end{pmatrix}$$

TRANSFORMĂRI 3D

TRANSFORMĂRI ELEMENTARE
- translație
- scalare față de origine
- rotație față de O_x, O_y, O_z
- oglindire față de xOy, yOz, xOz
- forfecare față de axe

- **ROTAȚIA ÎN JURUL AXEI Oz**



$$(x, y, z) \xrightarrow{R_{oz}(u)} (x', y', z') \begin{cases} z' = z \\ x' = x\cos u - y \sin u \\ y' = x \sin u + y \cos u \end{cases}$$

$$R_{oz}(u) = \begin{pmatrix} \cos u & -\sin u & 0 & 0 \\ \sin u & \cos u & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- **ROTAȚIA FAȚĂ DE AXA Ox**

$(x,y,z) \xrightarrow{R_{ox}(u)} (x',y',z') \Rightarrow \begin{cases} x' = x \\ y' = y\cos u - z\sin u \\ z' = y\sin u + z\cos u \end{cases}$

$$R_{ox} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos u & -\sin u & 0 \\ 0 & \sin u & \cos u & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- **ROTAȚIE FAȚĂ DE AXA Oy**

$(x,y,z) \xrightarrow{R_{oy}(u)} (x',y',z') \Rightarrow \begin{cases} y' = y \\ z' = z\cos - x\sin \\ x' = z\sin + x\cos \end{cases}$

$$R_{oy}(u) = \begin{pmatrix} \cos u & 0 & \sin u & 0 \\ 0 & 1 & 0 & 0 \\ -\sin u & 0 & \cos u & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- **ROTAȚIE ÎN JURUL UNEI DREPTE PARALELE CU Ox**

PAS 1 : T a.î. d să se suprapună cu $Ox \Rightarrow T(-x_c, -y_c, -z_c)$ sau $T(-x_c, 0, 0)$

PAS 2 : $R_{ox}(u)$

PAS 3 T^{-1}

$\Rightarrow R_d(u) = T(x_c, 0, 0) \cdot R_{ox}(u) \cdot T(x_c, 0, 0)$

- **ROTAȚIE FAȚĂ DE O DREAPTĂ OARECARE**

PAS 1 : Aliniem dreapta cu una din axe $(Ox/Oy/Oz) \Rightarrow A_d$

PAS 2 : R față de axa peste care s-a suprapus d

PAS 3 : A^{-1}

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = T(x_c, y_c, z_c) \cdot R_{ox}(-\alpha) \cdot R_{oy}(\beta) \cdot R_{oz}(u) \cdot R_{oy}(-\beta) \cdot R_{ox}(\alpha) \cdot T(-x_c, -y_c, -z_c) \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

- **OGLINDIRE FAȚĂ DE UN PLAN OARECARE**

PAS 1 : aliniere plan cu unul din planele sistemului de coordonate

PAS 2 : oglindire față de planul cu care s-a făcut alinierea

PAS 3 : A^{-1}

11.10.2017

CURS 3

PROIECȚII

Se caracterizează prin → 1. plan de proiecție
2. centru de proiecție (poziția observatorului / direcția)
3. mulțimea proiectilor

PROIECȚIE PARALELĂ / PERSPECTIVĂ

observatorul se află la ∞ ⇓ direcție de proiecție

observatorul se află la o distanță finită

PROIECȚIE PLANĂ ⇒ drepte care pleacă din C până în fiecare vârf al obiectului = proiectori

! Dacă C se află între planul de proiecție și obiectul de proiectat, atunci imaginea se va inversa.

PROIECȚIA PERSPECTIVĂ

- imagine realistă
- cu cât ne depărtăm mai mult, cu atât dimensiunea este mai mică
- laturile fețelor care nu sunt paralele cu planul de proiecție se vor intersecta într-un punct la infinit, care se numește **PUNCT DE CONVERGENȚĂ**.

Dacă punctul de convergență aparține unei axe, atunci acesta se numește **PUNCT DE CONVERGENȚĂ PRINCIPAL**

PENTRU EXAMEN : CARACTERISTICILE UNEI PROIECȚII PERSPECTIVE !

- observator la distanță finită
- dreptele paralele nu se proiectează neapărat ca ~~puncte~~ paralele ⇒ puncte de convergență
- imagine realistă
- cu cât ne depărtăm, obiectele apar mai mici (nu se păstrează dimensiunea obiectelor)

PROIECȚIA PARALELĂ

- imagine mai puțin realistă
- proiector paralel cu planul de proiecție

Slide 18

- dreapta jos ⇒ perspectivă
- restul ⇒ paralelă

- sus S+D ⇒ proiectori paraleli + perpendiculari pe planul de proiecție
- jos S ⇒ proiectori paraleli, dar nu sunt ⊥ pe planul de proiecție

PROIECȚII PARALELE → proiectori ⊥ plan proiecție ⇒ PROIECȚIE ORTOGRAFICĂ
→ proiectori ⊥ plan proiecție ⇒ PROIECȚIE OBLICĂ

PROIECȚIE ORTOGRAFICĂ DE TIP VEDERE

- caz particular de proiecție oblică cu unghi = 90°
- exemplu: Slide 18 a
- vedere ⇒ se vede o singură față a obiectului
- de obicei planul de proiecție este paralel cu un sistem de coordonate

$(x, y, z) \xrightarrow[xOy]{} (x, y, 0)$

↑ perpendiculară din punct pe plan

$(x, y, z, 1) \xrightarrow[\text{coord. omogene}]{} (x, y, 0, 1)$

PROIECȚIE ORTOGRAFICĂ

- se văd mai multe fețe ale obiectului ⇒ PROIECȚIE AXONOMETRICĂ

⇒ SOLUȚIE 1: rotesc obiectul

SOLUȚIE 2: modific planul de proiecție ca să nu mai fie paralel

PENTRU EXAMEN: CARACTERISTICILE PROIECȚIEI PARALELE

- centru de proiecție la ∞
- direcție de proiecție
- imaginea nu e realistă
- dreptele paralele rămân paralele indiferent de poziția față de planul de proiecție

!

PENTRU EXAMEN: CLASIFICARE PROIECȚII + CRITERII DE CLASIFICARE

!

- în funcție de poziția centrului de proiecție (poziția observatorului)

PROIECȚII → PERSPECTIVĂ ⇒ centru de proiecție la d finit
PROIECȚII → PARALELĂ ⇒ centru de proiecție la d infinit

- în funcție de numărul de puncte de convergență principale, proiecțiile perspective se împart:

PROIECȚII PERSPECTIVE CU → 1 PUNCT DE CONVERGENȚĂ PRINCIPAL
PROIECȚII PERSPECTIVE CU → 2 PUNCTE ⫽
PROIECȚII PERSPECTIVE CU → 3 PUNCTE

- în funcție de unghiurile pe care le fac proiectorii cu planul de proiecție, proiecțiile paralele se împart în:

PROIECȚII PARALELE → OBLICE ⇒ proiectori $\not\perp$ plan proiecție
PROIECȚII PARALELE → ORTOGRAFICE ⇒ proiectori $\perp$ plan proiecție

- în funcție de poziția planului de proiecție față de sistemele de coordonate, proiecțiile ortografice se împart în:

PROIECȚII ORTOGRAFICE → DE TIP VEDERE ⇒ plan proiecție $\parallel$ un plan al sist. coord.
PROIECȚII ORTOGRAFICE → DE TIP AXONOMETRIC ⇒ plan proiecție $\not\parallel$ niciun plan al sist. de coordonate

CONCLUZIE

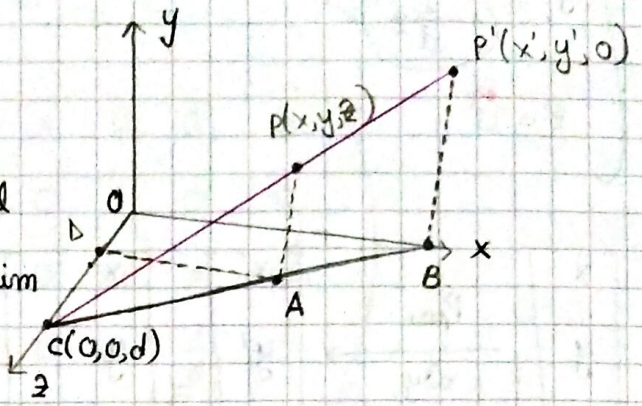
PROIECȚII → PERSPECTIVE → 1 PUNCT / 2 PUNCTE / 3 PUNCTE DE CONVERGENȚĂ PRINCIPALE
PROIECȚII → PARALELE → ORTOGRAFICE → DE TIP VEDERE / AXONOMETRICE
PROIECȚII → PARALELE → OBLICE

OBSERVAȚIE!

Numărul de puncte de convergență principale este egal cu numărul de intersecții dintre planul de proiecție și axele sistemului de coordonate

PROIECȚIE PERSPECTIVĂ CU 1 PUNCT DE CONVERGENȚĂ (CAZ PARTICULAR)

$$\begin{cases} \text{Plan de proiecție: } xOy \\ C: (0,0,d) \end{cases}$$



Pentru a determina proiecția lui P în planul xOy, plecăm din C, intersectăm P și ne oprim când intersectăm și planul xOy.

Vrem să calculăm x', y' în funcție de x, y, z, d

Fie $PA \perp xOz$, $PB \perp xOz$, $AD \perp yOz$

Triunghiuri asemenea: $\Delta CPA \sim \Delta CP'B$ și $\Delta CDA \sim \Delta COB$ (au raportul comun $\frac{CA}{CB}$)

$$\Rightarrow \frac{CA}{CB} = \underbrace{\frac{PA}{PB}}_{\frac{y}{y'}} = \frac{CP}{CP'} = \underbrace{\frac{DA}{OB}}_{\frac{x}{x'}} = \underbrace{\frac{CD}{CO}}_{\frac{d-z}{d}}$$

$$\Rightarrow \frac{y}{y'} = \frac{x}{x'} = \frac{d-z}{d} \Rightarrow \begin{cases} x' = \frac{xd}{d-z} \\ y' = \frac{yd}{d-z} \\ z' = 0 \end{cases}$$

Coordonate carteziene

$$\begin{bmatrix} x' \\ y' \\ z' \\ w' \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -\frac{1}{d} & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

↑ coordonate omogene

Proiecție perspectivă cu $p.p = xOy$ și $C = (0,0,d)$

Numitorul depinde de z-ul punctului proiectat

w' rezultă din proiecția perspectivă și nu va mai fi 1.

$$\begin{cases} x' = \dfrac{x}{1-\frac{1}{d}z} \\ y' = \dfrac{y}{1-\frac{1}{d}z} \\ z' = 0 \end{cases} \qquad w' = 1-\frac{1}{d}z$$

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \xrightarrow[\substack{xOy \\ c(0,0,d)}]{P_{pers}} \begin{bmatrix} x' \\ y' \\ z' \\ w' \end{bmatrix} = \begin{bmatrix} x \\ y \\ 0 \\ 1-\frac{1}{d}z \end{bmatrix} \Rightarrow \begin{cases} x' = x \\ y' = y \\ z' = 0 \\ w' = 1-\frac{1}{d}z \end{cases}$$

Coordonate omogene

Ca să trecem înapoi la coordonate omogene:

$$x' = \frac{x}{w'} \qquad y' = \frac{y}{w'} \qquad z' = 0$$

(EX) PENTRU EXAMEN !

Plan proiecție : yOz

Centru de proiecție : $c(d,0,0)$

$P(x,y,z) \Rightarrow P'(0,y',z')$ $\quad P' = ?$

Nu se va scrie direct matricea (se va face ca modelul anterior)

$$\begin{cases} x' = 0 \\ y' = y \\ z' = z \\ w' = 1-\frac{1}{d}x \end{cases} \Rightarrow \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -\frac{1}{d} & 0 & 0 & 1 \end{pmatrix}$$

Coordonate omogene

PROIECȚIE PERSPECTIVĂ CU CENTRUL ÎN ORIGINE

Slide 26 :

$\Delta OAC \sim \Delta OBD$

$\Delta OAP' \sim \Delta OBP$

$$\Rightarrow \frac{OA}{OB} = \frac{x'}{x} = \frac{d}{z} = \frac{y'}{y} \Rightarrow \begin{cases} x' = \frac{xd}{z} \\ y' = \frac{yd}{z} \\ z' = d = \frac{dz}{z} \end{cases}$$

$$\begin{cases} x' = \frac{x}{\frac{1}{d}z} \\ y' = \frac{y}{\frac{1}{d}z} \\ z' = \frac{z}{\frac{1}{d}z} \end{cases} \Rightarrow w' = \frac{1}{d}z \Rightarrow \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & \frac{1}{d} & 0 \end{pmatrix}$$

$P_{pers.}$ cu $C(0,0,0)$ și $p.p \parallel xOy$

PROIECȚIE PARALELĂ ÎN PLANUL xOy

Slide 30

- centrul de proiecție aflat la $\infty \Rightarrow$ direcție de proiecție

$\Rightarrow (dx, dy, dz, 0)$

$w \neq 0 \Rightarrow$ puncte finite în plan (tratate ca poziții)

$w = 0 \Rightarrow$ puncte la infinit (tratate ca direcții)

$P(x,y,z) \xrightarrow[pr.\ xOy]{} P'(x',y',0)$

direcție de proiecție $= \vec{v}$

direcție de proiecție $\parallel$ proiector : $\vec{v} \parallel \overrightarrow{PP'}$

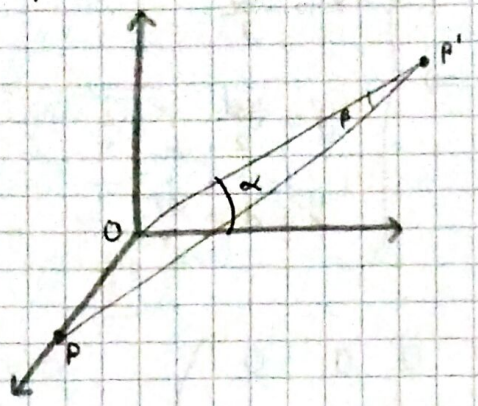
$\vec{v}$: a, b, c ; $\overrightarrow{PP'}$: $x'-x,\ y'-y,\ z'-z$

! La proiecția paralelă putem păstra $w=1$, dar la cea perspectivă nu, deoarece w depinde de coordonate pe care o modific.

$$\begin{bmatrix} x' \\ y' \\ z'=0 \\ 1 \end{bmatrix} = \begin{pmatrix} 1 & 0 & -a/c & 0 \\ 0 & 1 & -b/c & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

PROIECȚIE OBLICĂ

Știm β făcut de dreaptă cu plan



Punctul P' nu e unic. Ca să îl fixăm, avem nevoie de α.

$\begin{cases} \beta \rightarrow \text{proiecția} \\ \alpha \rightarrow \text{pentru a avea un singur punct proiectat} \end{cases}$

$\overrightarrow{PP'} \Rightarrow \Delta(x', y', -1)$

direcția de proiecție în funcție de punctele care ne interesează. Vom încerca să le exprimăm în funcție de α și β

Considerăm $OP' = L \Rightarrow \begin{cases} x' = L\cos\alpha \\ y' = L\sin\alpha \end{cases}$

$f = \frac{OP'}{OP} = L$ ← Particularizare pentru $(0,0,1)$

↑ factor de scalare ← proiecție ← segment care se proiectează

$\Rightarrow \begin{cases} x' = f\cos\alpha \\ y' = f\sin\alpha \end{cases}$

$\Delta OPP'$ (Δ dr.) $\Rightarrow tg\beta = \frac{OP}{OP'} = \frac{1}{f} = \frac{1}{L}$

General: $\begin{cases} x' = x + z \cdot f \cdot \cos\alpha \\ y' = y + z \cdot f \cdot \sin\alpha \end{cases}$

$f = 0 \Rightarrow$ PROIECȚIE ORTOGRAFICĂ CU PROIECTORII $\perp$ PLAN PROIECȚIE

$f = 1 \Rightarrow$ CAVALIER

$f = 1/2 \Rightarrow$ CABINET $\rightarrow$ laturile $\parallel$ PP sunt la dim normală, iar cele $\perp$ PP sunt $\frac{1}{2}$ dim

VEDERI

- principale - Stânga, Dreapta, Sus, Jos, Spate, Față
  ↳ obiecte cu fețe ∥ cu planul de proiecție
- auxiliare ⇒ nicio față a obiectului nu e paralelă cu planul de proiecție
  ⇒ se văd mai multe fețe

! Ca să modific o vedere auxiliară în vedere principală, rotesc obiectul până când o față devine paralelă cu sistemul de coordonate sau aliniez.

- secțiune ⇒ obiectul intersectează sistemul de coordonate

PROIECȚIE AXONOMETRICĂ

- există factori de scalare de-a lungul fiecărei axe

$$f = \frac{\text{segment proiectat}}{\text{segment de proiectat}}$$

- izometrică : $f_x = f_y = f_z$
- dimetrică : $f_x = f_y$ sau $f_y = f_z$ sau $f_x = f_z$
- trimetrică : $f_x \neq f_y \neq f_z \neq f_x$

CURS 4

PLASAREA OBIECTELOR = raportarea la același sistem de coordonate (SISTEM REAL)

PLASAREA OBSERVATORULUI = trecerea la un sistem de coordonate observator

⇒ limitare a scenei prin volumul de vizualizare

SISTEM DE COORDONATE OBIECT ⟹ SISTEM DE COORDONATE REAL ⟹ SISTEM DE COORDONATE DE VIZUALIZARE

Raportăm obiectele la sistemul de coord. unn (Tranformări 3D)

VIEWPORT 3D

Se alege a.î. intersectiile să fie mai ușoare (în OpenGL: cub cu l=2 centrat în origine)

ECRAN

LANTUL DE TRANSFORMĂRI DIN OPENGL

① SPAȚIUL DE COORDONATE OBIECT → SPAȚIUL DE COORDONATE REAL / GLOBAL / AL LUMII

Se folosește matricea de modelare care constă în transformări elementare: translație, rotație, scalare, oglindire, forfecare

Spațiul de coordonate obiect ⇒ fiecare obiect are propriul sistem de coordonate

Spațiul global: obiectele sunt definite în aceeași scenă

② SPAȚIUL DE COORDONATE REAL → SPAȚIUL DE COORDONATE DE VIZUALIZARE

Presupune plasarea observatorului în sistemul de coordonate real.

Presupune transformarea de vizualizare, care constă în transformări, rotații ale sistemului de coordonate

! În sistemul de coordonate de vizualizare, observatorul este în origine.

③ DEFINIRE VOLUM DE VIZUALIZARE

Transformare Viewport = Transformare de proiecție
(înglobează proiectiile, dar rămân în 3D)

⇒ Se obține sistemul de coordonate dispozitiv

LANTUL COMPLET DE TRANSFORMĂRI

→ sistem de coordonate obiect

1. TRANSFORMAREA DE MODELARE

→ sistem de coordonate real

2. TRANSFORMAREA DE VIZUALIZARE

→ sistem de coordonate de vizualizare

3. TRANSFORMAREA DE PROIECȚIE

→ sistem de coordonate dispozitiv

4. ÎMPĂRȚIREA PERSPECTIVĂ (normalizarea lui w) : $(x, y, z, w) \Rightarrow (x', y', z', 1)$

→ sistem de coordonate normalizat

5. PROIECȚIE PARALELĂ ORTOGRAFICĂ DE TIP VEDERE (renunțare la coordonata z)
(planul față al viewport-ului 3D se mapează în zona de afișare)
→ sistem ecran

MATRICILE CORESPUNZĂTOARE LANȚULUI DE TRANSFORMĂRI ÎN OPENGL

1. Matricea de modelare = **M** (S.C obiect ⇒ S.C. real)
2. Matricea de vizualizare = **V** (S.C real ⇒ S.C. vizualizare) } combinații de transformări geometrice
3. Matricea de proiecție = **P** (~~în~~ S.C. vizualizare definește volumul de vizualizare specific proiecției care se transformă în viewport 3D)

cub cu latură = 2, centrat în origine cu fețe paralele cu planele sistemului de coordonate

OBSERVAȚIE!

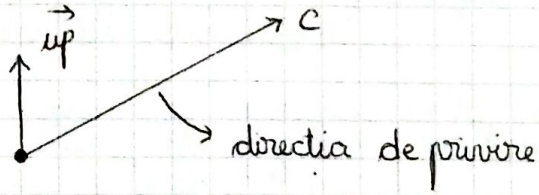
1 și 2 sunt văzute ca o singură transformare. Matricea asociată se numește

MATRICEA DE MODELARE-VIZUALIZARE (MODEL VIEW)

modelViewMatrix = VM

IMPLICIT: observator în originea sistemului de coordonate real

Observator → poziția
→ direcția de privire
→ orientare ($\vec{up}$)
} specificate față de S.C. real



LOOKAT

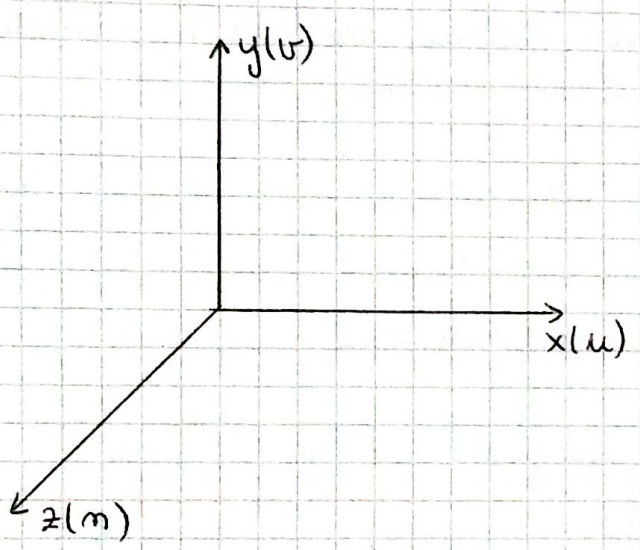
Construiește matricea de vizualizare (V) și trece de la sistemul de coordonate real la sistemul de coordonate de vizualizare.

- Parametrul 1 → poziție
- Parametrul 2 → punctul către care privește observatorul
- Parametrul 3 → orientarea

- Axa z este invers față de direcția de privire (axa n e inversă față de axa z a sistemului de coordonate de vizualizare)
- Axa x se obține din produsul vectorial dintre orientarea observatorului și direcția axei z pozitive a sistemului de coordonate de vizualizare
- Axa y se obține din produsul vectorial al axelor z și x

$$\begin{cases} n = (Obs - C) / |Obs - C| \\ u = (Up \times n) / |Up \times n| \\ v = n \cdot u \end{cases}$$

IMPLICIT $\begin{cases} \text{Observator} : (O_x, O_y, O_z) = (0,0,0) \\ \text{Direcția de privire} : (C_x, C_y, C_z) \in O_z \\ \text{Orientare (up)} : (Up_x, Up_y, Up_z) = (0,1,0) \end{cases}$



Sistem real = sistem de vizualizare
(în cazul implicit)
$\Rightarrow V = I_4$

Dacă nu suntem în cazul implicit:

$(x, y, z) \rightarrow u, v, n$

S.C. real — S.C. vizualizare (origine în poziția observatorului)

$$\begin{pmatrix} u_x & u_y & u_z \\ v_x & v_y & v_z \\ n_x & n_y & n_z \end{pmatrix} \Longrightarrow V = \begin{pmatrix} u_x & v_x & n_x & t_x \\ u_y & v_y & n_y & t_y \\ u_z & v_z & n_z & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

↑ direcțiile de normalizare ale axelor

asigură aliniera axelor

asigură repoziționarea centrelor

TRANSFORMAREA DE PROIECȚIE

- setare volum de vizualizare

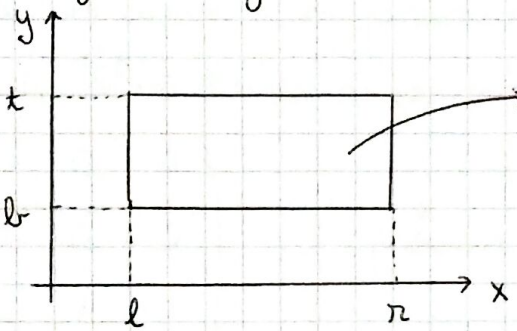
- **proiecție perspectivă** ⇒ trunchi de piramidă (vârful piramidei se află în poziția observatorului)

Funcțiile specifice în OpenGL:

- **frustum** : volumul de vizualizare poate fi simetric / asimetric
- **perspective** : volumul de vizualizare este simetric față de direcția de privire întotdeauna

frustum (l, r, b, t, n, f)

Se definește o fereastră de vizualizare în planul față



fereastra plan față (este specificată prin cei 4 parametri: l, r, b, t)
left ↓ right ↓ top ↓ bottom

n și f definesc cele 2 plane (distanțele până la planul față, respectiv planul spate)

perspective

Volumul de vizualizare este simetric față de direcția de privire

- n, f - definesc cele 2 plane (distanțele până la planul față, respectiv planul spate)
- fovy - unghiul de deschidere de la vârful piramidei specificat în planul yOz
- aspect - raportul dintre l și L la planul față

- **proiecție paralelă** ⇒ paralelipiped
  - direcția de proiecție = direcția de privire
  - planul față ⇒ l, r, b, t
  - n, f ⇒ distanțele până la planul față, respectiv planul spate

Paralelipipedul / Trunchiul de piramidă trebuie convertit în cubul de latură 2, centrat în origine.

MAPARE PARALELIPIPED ÎN CUBUL DE LATURĂ 2

1. Translație astfel încât C să se afle în origine

$$T\left(-(l+r)/2,\ -(b+t)/2,\ -(n+f)/2\right)$$

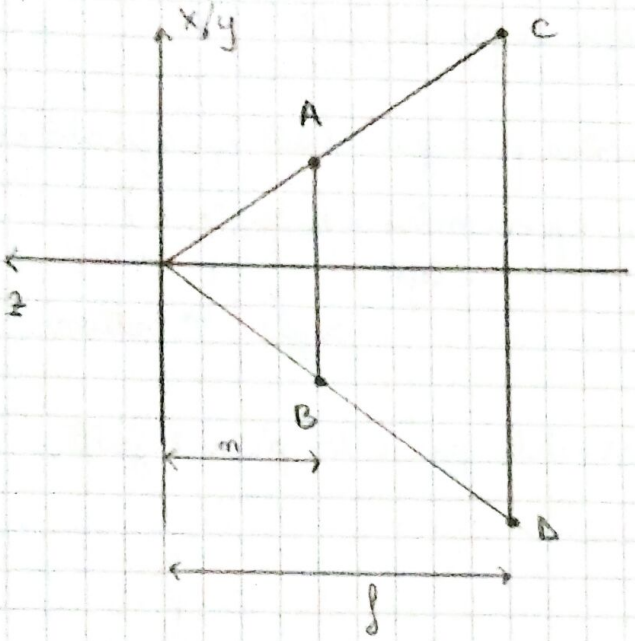
2. Scalare în cubul de latură 2, centrat în origine

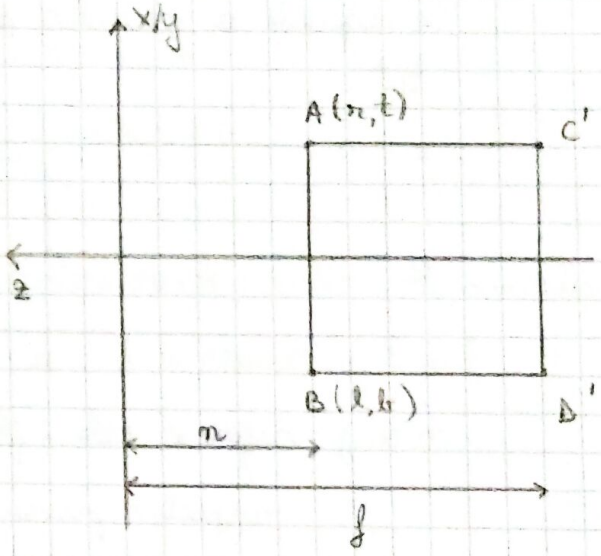
$$S\left(2/(r-l),\ 2/(t-b),\ 2/(f-n)\right)$$

am considerat z-ul ca fiind pozitiv

MAPARE TRUNCHI DE PIRAMIDĂ ÎN CUBUL DE LATURĂ 2

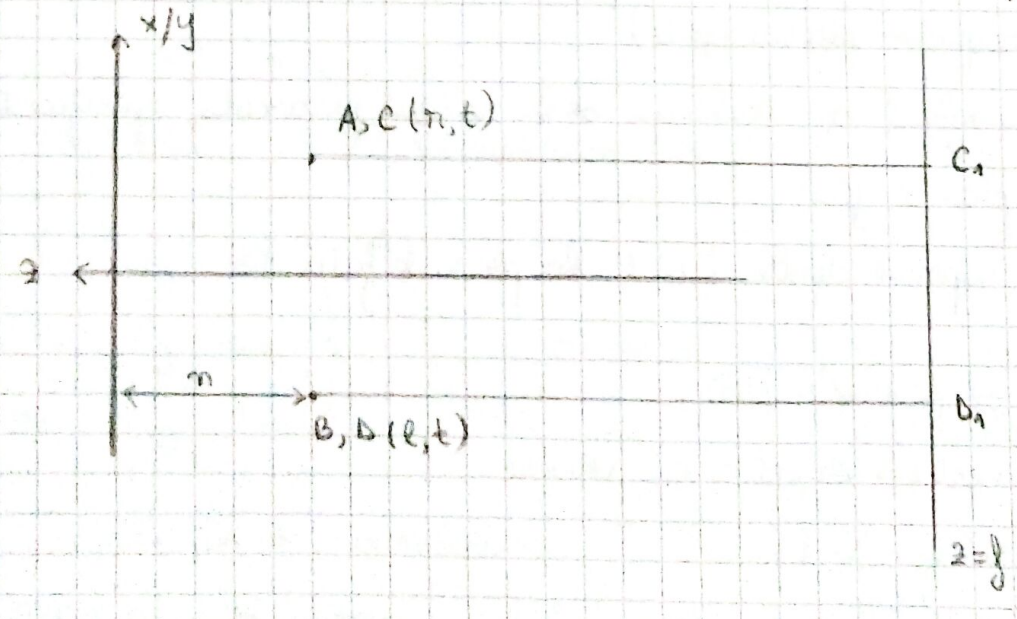
Planul față e la fel și la proiecția paralelă și la proiecția paralelă.





Considerăm:

proiecție perspectivă în planul față ($z = n$ în direcția de privire)



Orice punct de la A la C și de la B la D se proiectează în punctele A, respectiv B, deci trebuie mapate și ele (z nu e fix ⇒ variază între n și f)

$$P_{pers} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/n & 0 \end{pmatrix}$$

→ z rămâne constant, dar la noi trebuie să varieze liniar între n și f
⇒ Modificăm în a și b.

Proiecția perspectivă pe planul paralel cu xOy aflat la distanță n față de origine și având observatorul plasat în (0,0,0)

Proiecția perspectivă cu centrul de proiecție în (0,0,0) pe planul de proiecție ∥ xOy aflat la distanță n (în direcția de privire)

$$P_{pers} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/n & 0 \end{pmatrix}$$

proiecție plană (trec din 3D în 2D)

⇓ înglobare variație liniară z

$$T_{persp} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & a & b \\ 0 & 0 & 1/n & 0 \end{pmatrix}$$

proiecție perspectivă, dar mă lasă tot în 3D

a, b = ? astfel încât z să rămână între n și f

$z = az + b$ (coord. omogene)

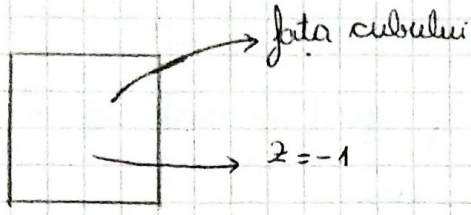
$w' = \frac{1}{n} \cdot z$

⇓ în coord. carteziene : $\frac{az+b}{\frac{1}{n}z} \in [n, f]$ pentru $z \in [n, f]$

$$\begin{cases} an+b=n \\ af+b=\frac{f^2}{n} \end{cases} \Rightarrow \boxed{a=\frac{n+f}{n}} \quad \boxed{b=-f}$$

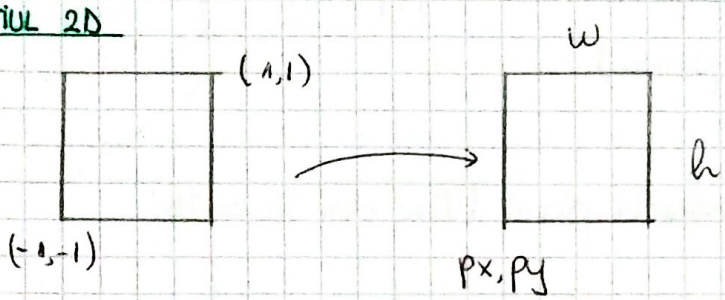
EXAMEN

! Care sunt transformările necesare pentru calculatea matricelor de transformare? (fără înmulțit matrici, calcule etc.) Doar ideea și ce presupun (T, S, F, O, R).



plan de proiecție = proiecție paralelă ortografică de tip vedere (x, y, ~~z~~)

SPAȚIUL 2D

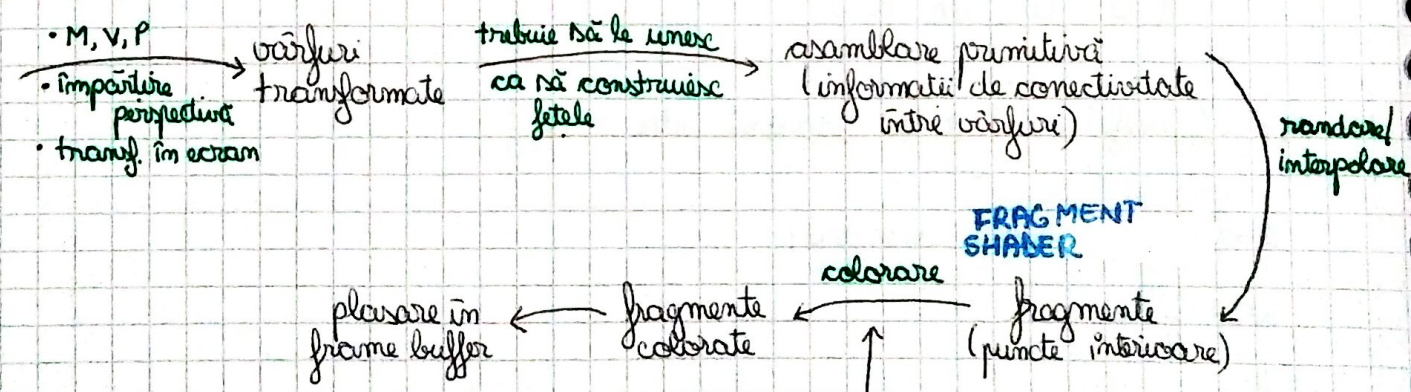


Ecran

TRANSFORMAREA FEREASTRĂ-POARTĂ (folosită pentru mapparea fetei cubului în poarta de afișare)

1. T(1,1)
2. S(w/2, h/2) sau S(min(w/2, h/2), min(w/2, h/2)) pt. scalare uniformă
3. T(px, py)

BANDA GRAFICĂ



CURS 5

VÂRFURI ⟶ VÂRFURI TRANSFORMATE

1. transformarea de modelare
2. sistem real → sistem de vizualizare
3. definire volum de vizualizare
4. transformare în cubul de latură 2, centrat în origine
5. trecere de la 3D la 2D

(x, y, z) sistem obiect ⟶

1. M (trecere de la sistem obiect la sistem real)
2. V (trecere de la sistem real la sistem de vizualizare)
3. P (transformare proiecție: transformarea volumului de vizualizare în volumul de vizualizare canonic
   ⇒ cub de latură 2, centrat în origine
4. Împărțire perspectică (ca să ajung la $w=1$)

Proiecție ortografică de tip vedere cu $z=-1$ și mapare în zona de afișare

(x_c, y_c, z) ← coord. 3D real

x_c, y_c — coord. ecran

z — adâncimea (distanța față de observator) (se folosește pentru eliminarea colorării zonelor care nu se văd)

VÂRFURI TRANSFORMATE ⟶ ASAMBLARE

Interpolare liniară / Rasterizare

Fragmente

Pentru culoare → interpolare în funcție de culoarea vârfurilor
→ modele de iluminare

VERTEX SHADER → se aplică paralel pe vârfuri (independent)

FRAGMENT SHADER → se aplică paralel pe fragmente (independent)

VERTEX SHADER → primește coord. vârfuri, matrici de modelare (transformă vârfuri)

FRAGMENT SHADER → obține culoarea specifică obiectului

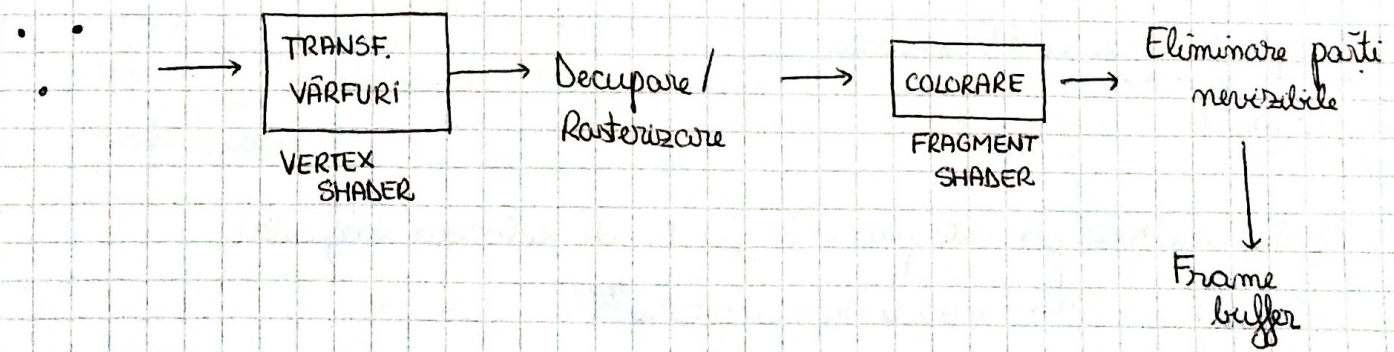
OPERAȚII SHADER → eliminare fragment nevizibile, combinare culori etc.

VERTEX SHADER

- realizează transformările pentru matricea de modelare-vizualizare și matricea de proiecție
- poate să primească și normalele vârfurilor pe care le putem folosi pentru iluminarea vertexului
- poate să primească coordonatele de texturare

FRAGMENT SHADER

- se execută în paralel pentru fiecare fragment
- nr. fragmente » nr. vârfuri
- sarcină principală: atribuire culoare fragment
  - fie preia de la Vertex Shader
  - fie o calculează singur printr-un model de iluminare / ia dintr-o imagine / aplică o textură



PARAMETRU IEȘIRE VERTEX SHADER = PARAMETRU INTRARE FRAGMENT SHADER

Vertex Shader
Fragment Shader } programe C cu anumite limitări (nu permit folosirea pointerilor, recursivitate, alocare dinamică)

→ vec2, vec3, vec4
→ ivec*, uvec*, bvec*

Elementele pot fi văzute ca:

- coordonate: (x, y, z, w)
- culori: (r, g, b, a)
  → transparență
- coordonate de texturare: (s, t, p, q)
  coordonate textură 2D

vec4 v; ← poate fi privit drept o coordonată, o culoare sau o coordonată de texturare

Accesare: v.x, v.r, v.s

v.(xy) → trebuie să fie de același tip ambele NU: v.xg

→ un fișier cu transformări pentru Vertex Shader
→ —//— pentru Fragment Shader

Parametrii constanți pentru toată scena = parametrii uniformi

- parametru de intrare (in)
- parametru de ieșire (out)

PARAMETRII CARE TREC PRIN INTERPOLARE:

Param. de ieșire de la Vertex Shader care același nume ca param. de intrare de la Fragment Shader.

modelare → modelMatrix proiecție → perspective
vizualizare → lookAt

vertex → vârf

object_color → culoare vârf

VERTEX SHADER
- Param. intrare : in vec4 position, in vec4 color
- Param. ieșire : out vertex_to_fragment_color
- "calculează" culoarea de ieșire

trece la interpolare și după la intrarea FS

FRAGMENT SHADER
- Param. intrare : in vertex_to_fragment_color
- Param. ieșire : out vec4 fragment_color

↳ culoare primită la intrare, trecută prin fază de interpolare

↓

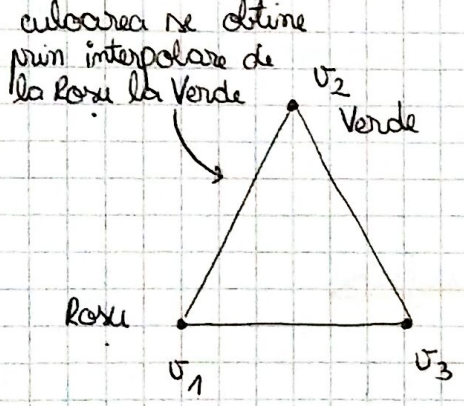
Se transmite către Frame Buffer

VERTEX SHADER

- Calculează culoarea și o transmite mai departe
- Calculează position în urma M, V, P și o trimite către rasterizare

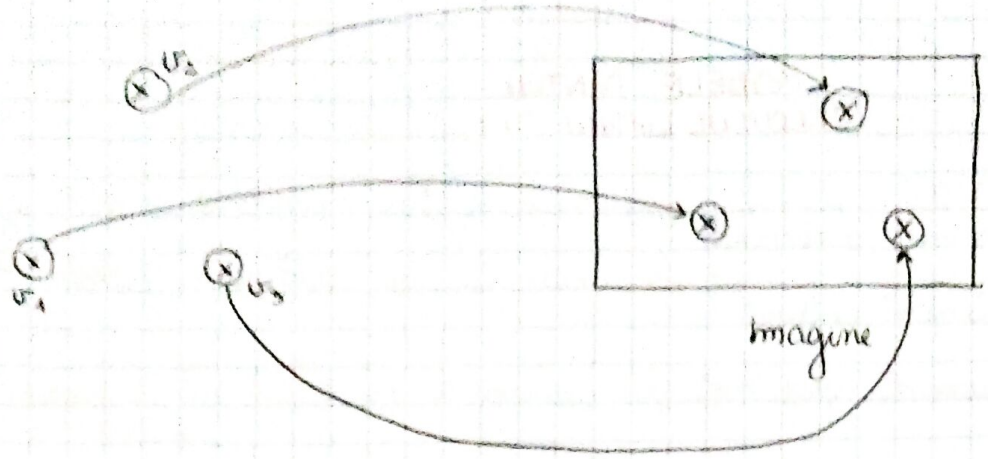
  gl_Position = P·V·M·position (VS o poate modifica sau nu, dar trebuie să existe!)

  vertex_to_fragment_color ← doar se retează valoarea culorii și se transmite mai departe



- Dacă vrem să aplicăm textură pe suprafața obiectului:

  Avem nevoie de fața obiectului pe care aplicăm și imaginea. Pentru fiecare punct asociez câte un punct din imagine de unde voi lua culoarea. Pe baza pixelilor se determină pixelii corespunzători laturilor și interiorului



(s, t) coord de texturare

Pentru fiecare coord, trebuie să am asociată o coord. de texturare
din Vertex Shader

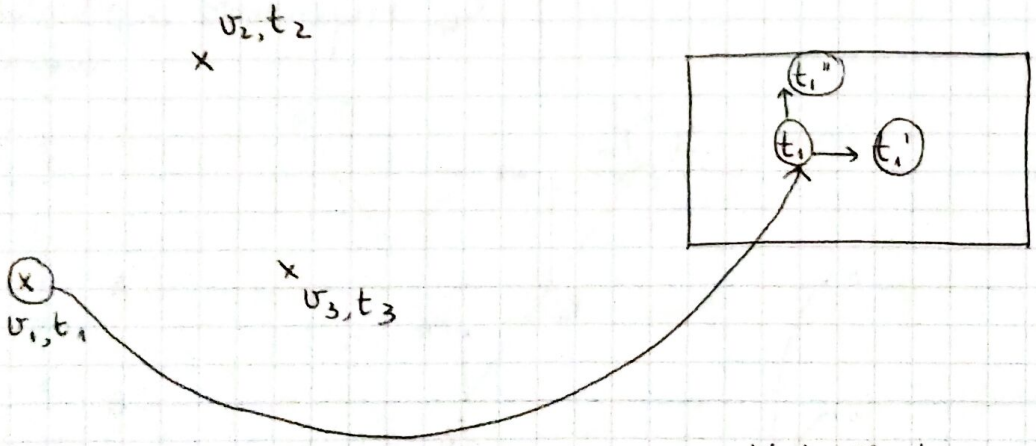
La ieșirea Vertex Shader-ului trebuie să avem textCoord cu valoarea primită la input (nu se modifică)

Fragment Shader: primește imaginea ca sampler 2D img
parametru uniform

Pentru accesarea imaginii, se folosește funcția texture(imagine, coord).xyz
pentru a lua doar RGB

Se va obține o culoare:

color = vec4(texture(imagine, coord).xyz, 1)
↑ culoare finală fragment
└ opacitate



v_1 = coord. 3D

t_1 = coord. texturare

Vertex Shader

```
in vec2 in_texCoord;
out vec2 tex1, tex2;
{
    tex1 = ...
    tex2 = ...
```

Fragment Shader

```
in vec2 tex1, tex2
```

MODELE PENTRU CALCULUL LUMINII

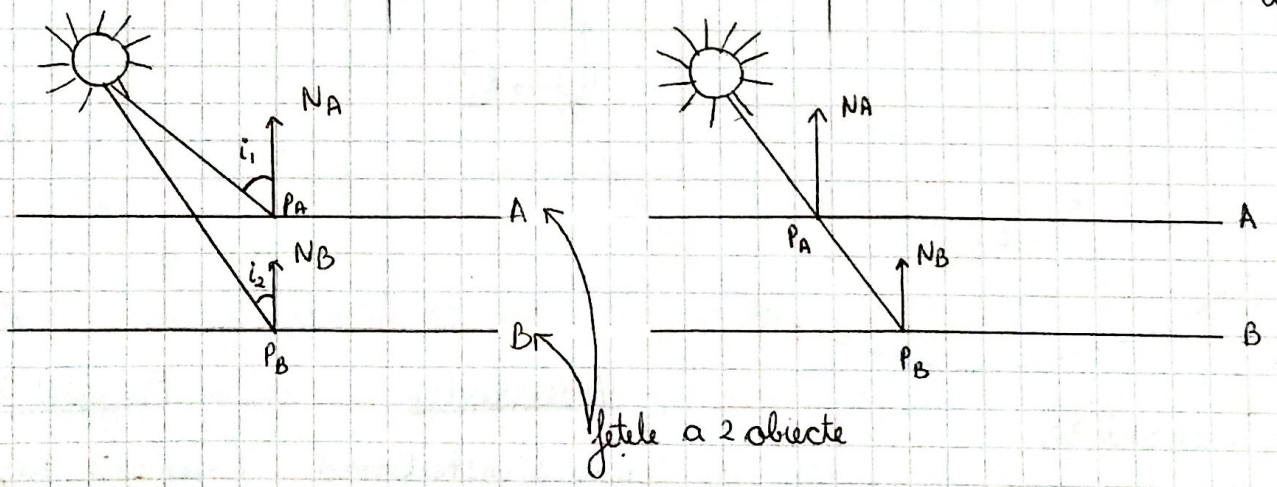
MODELE EMPIRICE → lumină ambiantă
→ lumină difuză
→ lumină speculară

↑ calculează lumina într-un singur punct

MODELE ANALITICE → modelul Lambert
→ modelul Gouraud
→ modelul Phong (cel mai apropiat de realitate, dar timp mare)

↑ calculează lumina pe o față întreagă

INFO VERTEX	PROPRIETĂȚI MATERIAL	PROPRIETĂȚI SURSĂ LUMINĂ
$P(x, y, z)$	K_a, K_d, K_s } culori	i_d, i_p } culori
$\vec{N}(nx, ny, nz)$	n = coef. strălucire → val. minimă ct.	poziție sursă / direcție
		K_c
		K_L → dependență de distanță
		K_Q → dependență de pătratul distanței



Indiferent de distanța d de la sursă la obiect, lumina difuză depinde doar de unghiul de incidență ⟹ în realitate: nu

SOLUȚIE: $i_d = f_{at} \cdot K_p \cdot i_p (\vec{N} \cdot \vec{L})$, unde $f_{at} = 1/(K_c + K_L \cdot d + K_Q \cdot d^2)$ factor de atenuare

K_p, i_p specifice sursei de lumină (constante)

CURS 6

- MODELE EMPIRICE : calcul iluminare într-un punct P
- MODELE ANALITICE : calcul iluminare pentru o față

MODELE DE ILUMINARE LOCALE ⇒ nu consider reflexia / refracția sursei de lumină
⇒ nu apar surse noi de lumină

MODELE EMPIRICE →
- componenta ambientală
- componenta difuză (lumină uniform reflectată în toate direcțiile indiferent de poziția observatorului)
- componenta speculară (obținem strălucirea obiectului vizibilă doar de partea observatorului)

$$i_{am} = k_a \cdot i_a$$

COMPONENTA AMBIENTALĂ

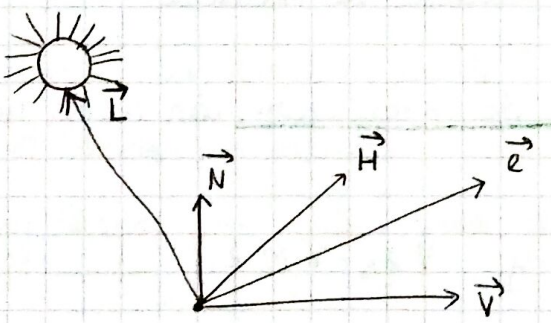
$$i_d = i_{sursă} \cdot k_d \, (\vec{N} \cdot \vec{L})$$

$$\Downarrow$$

$$i_d = f_{at} \cdot i_{sursă} \cdot k_d \, (\vec{N} \cdot \vec{L})$$

se consideră normalizați

COMPONENTA DIFUZĂ



$$i_s = f_{at} \cdot i_{sursă} \cdot k_s \, (\vec{R} \cdot \vec{V})^m$$

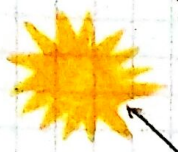
- f_{at} ↑ depinde de distanța de la punct la sursa de lumină
- $(\vec{R} \cdot \vec{V})$ ↑ raza reflectată
- $\vec{V}$ ↑ vector observator

$$\vec{R} \cdot \vec{V} \simeq (\vec{H} \cdot \vec{H})$$

$$\vec{H} = \vec{L} + \vec{V}$$

COMPONENTA SPECULARĂ

PROPRIETĂȚILE SURSEI DE LUMINĂ → I_a , $I_{sursă}$ → componenta difuză / componenta speculară



$S(x_s, y_s, z_s)$

SPAȚIUL REAL SAU DE VIZUALIZARE

$\vec{L}$

← Sursa se află la poziție finită

$P(x,y,z)$

SPAȚIU OBIECT

$\vec{L}(x_s - x, y_s - y, z_s - z)$

$S(x_s, y_s, z_s)$ poate fi definită → în spațiul real / în spațiul de vizualizare

SPAȚIUL OBIECT $\xrightarrow{M}$ SPAȚIUL REAL $\xrightarrow{V}$ SPAȚIUL DE VIZUALIZARE

- Dacă se face calculul în <u>spațiul real</u>:

  $\vec{L} = ?$

  $(x_s - \underbrace{M \cdot x}_{\text{x transformat prin matricea de modelare}} ; y_s - M \cdot y, z_s - M \cdot z)$

- Dacă se face calculul în <u>spațiul de vizualizare</u>, iar sursa rămâne în <u>sistemul real</u>:

  $\vec{L}(V \cdot x_s - V \cdot M \cdot x, V \cdot y_s - V \cdot M \cdot y, V \cdot z_s - V \cdot M \cdot z)$

- Dacă se face calculul în <u>spațiul obiect</u>, iar sursa rămâne în <u>sistemul real</u>:

OBSERVAȚIE!

În spațiul de vizualizare, poziția observatorului este în $(0, 0, 0)$.

→ Nu are sens să înmulțim cu V când vrem să calculăm $\vec{V}$

n surse de lumină

Intensitatea luminii în punctul P = ?

Răspuns:

Pentru fiecare sursă de lumină, o să obțin I_{p_i} (pe baza sursei i)

$I_{p_i} = (I_{am} + I_d + I_s)_i$

Intensitatea totală se obține însumând luminile de la fiecare sursă:

$$I_p = \sum_{i=1}^{n} I_{p_i}$$

MODELE ANALITICE

- Lambert, Gouraud, Phong
- se bazează pe calculul intensității culorii

LAMBERT

- asociază o singură culoare unei fețe
- intensitatea este dată de unghiul dintre normală și raza incidentă

  k = constantă de difuzie
- se alege centrul fetei pentru a calcula intensitatea (de obicei)
- sunt vizibile trecerile de la o față la alta

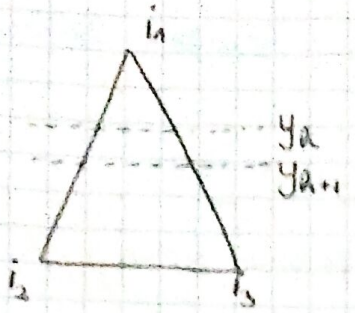
✗ Vertex Shader → am doar puncte (nu știu să determin mijlocul)

✗ Fragment Shader → calculez normale pentru fiecare segment

✓ Geometry Shader → între Vertex Shader și Fragment Shader

GOURAUD

1. Se calculează o normală în fiecare vârf
2. Se calculează o intensitate în fiecare vârf (comp. ambientală, difuză, speculară)
3. Se calculează intensitatea pentru fiecare punct interior prin interpolarea intensităților din vârfuri



i_1, i_2, i_3 = intensitățile calculate per vertex (folosind modelul empiric)

- latura $1 \to 2$:

$$i = i_1 + t(i_2 - i_1) \begin{cases} \to t = 0 \Rightarrow i_1 \\ \to t = 1 \Rightarrow i_2 \end{cases}$$

- pentru i_P (y_k)

$$i_P = i_1 + t_P(i_2 - i_1)$$

$$y = y_1 + t(y_2 - y_1)$$

$$y_P = y_1 + t_P(y_2 - y_1) \Rightarrow t_P = \frac{y_P - y_1}{y_2 - y_1}$$

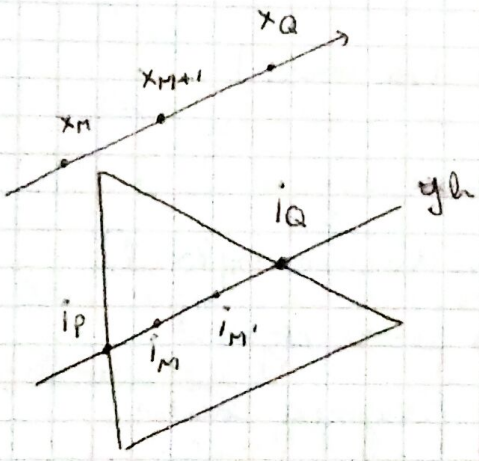
$$i_P = i_1 + (i_2 - i_1) \cdot \frac{y_P - y_1}{y_2 - y_1} = i_1 + (i_2 - i_1) \cdot \frac{y_k - y_1}{y_2 - y_1}$$

$\Downarrow$

$$i_P' \text{ pt. } y_{k+1} \Rightarrow i_P' = i_1 + (i_2 - i_1) \cdot \frac{y_k + 1 - y_1}{y_2 - y_1} = i_P + \frac{i_2 - i_1}{y_2 - y_1}$$

nu depinde de valoarea curentă a dreptei de intersecție (este o constantă pt. fiecare latură)

ne putem baza pe intensitatea calculată în punctul anterior



i_M se calculează prin interpolare între i_P și i_Q

$$i_M = i_P + t(i_Q - i_P)$$

$$x = x_P + t(x_Q - x_P)$$

$$x_M \Rightarrow t_M$$

$$x_M = x_P + t_M(x_Q - x_P) \Rightarrow t_M = \frac{x_M - x_P}{x_Q - x_P}$$

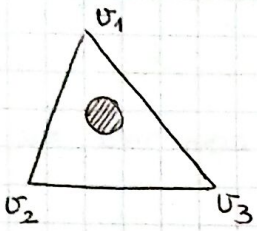
$$i_M = i_P + (i_Q - i_P) \cdot \frac{x_M - x_P}{x_Q - x_P}$$

$i_{M'}$ pentru x_{M+1}: $i_{M'} = i_P + (i_Q - i_P) \cdot \frac{x_M + 1 - x_P}{x_Q - x_P} = i_M + \frac{i_Q - i_P}{x_Q - x_P}$

↳ constanta pentru latura curentă y_A

Gouraud dă rezultate mai bune decât Lambert.

Întrebare test



Sursa de lumină ajunge doar pe suprafața din cerc.
Cum apare în realitate folosind modelul Gouraud ?

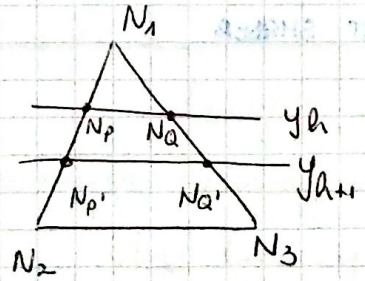
Răspuns: Triunghiul apare neiluminat complet (doar componenta ambientală), deoarece în modelul Gouraud se pornește de la intensitățile vârfurilor și apoi se interpolează, astfel că, nefiind iluminate vârfurile, iluminarea nu este vizibilă

!!! componenta specular depinde de cum sunt luminate vârfurile

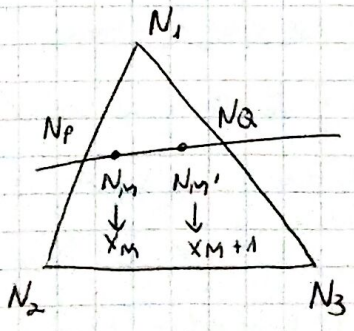
PHONG

Intensitățile sunt calculate pentru fiecare fragment ⇒ foarte multe calcule, deoarece nr. fragmente >> nr. vârfuri

1. Se interpolează normala din vârfuri
2. Se calculează intensitatea în fiecare fragment folosind modelul empiric



$N_{P'} = N_P + \frac{N_2 - N_1}{y_2 - y_1}$ → constantă pentru latura 1→2

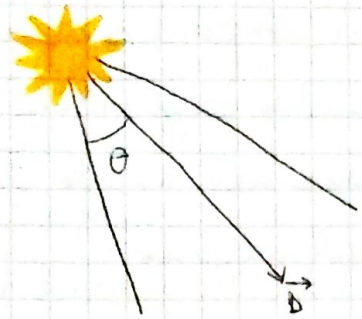


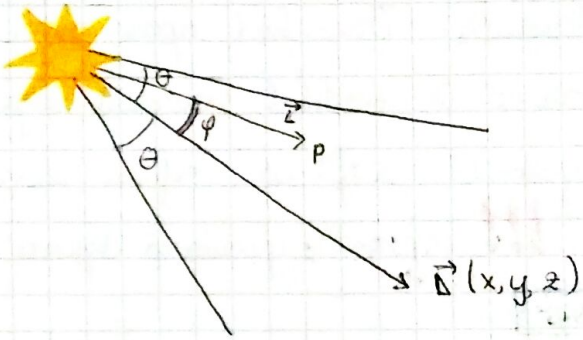
$N_{M'} = N_M + \frac{N_Q - N_P}{x_Q - x_P}$ → constantă pentru latura P→Q

Folosindu-se modelul Phong, în cazul întrebării de la test, de această dată se obține iluminarea din mijlocul triunghiului (există componentă specular)

LUMINA DE TIP SPOT

- lumina cade în interiorul unui con specificat prin axa de simetrie și unghiul de deschidere (ce e în afara conului nu primește lumină)
- putem alege să avem intensitate constantă sau intensitate constantă în zona de mijloc și să scadă spre margini
  ↳ poate fi privită ca un con mai mic într-un con mai mare





Se verifică dacă $\varphi < \theta$ (punctul e în con)

Ca să obțin φ: $\vec{D} \cdot \vec{L} \Rightarrow$ obțin $\cos\varphi$

$\Rightarrow \cos\varphi > \cos\theta$ pt. a fi în con P (primește lumină)

OBSERVAȚIE!

Phong se implementează în Fragment Shader.

VERTEX SHADER	FRAGMENT SHADER
coordonata punctului P (in)	culoare vârf (in)
normala punctului P (in)	culoare finală (= cea pe care o primește) (out)
glPosition (out)	
culoare asociată vârfului (out) →	
UNIFORM: M, V, P (in); proprietăți sursă / surse lumină (in); constante de material (in); poziție observator (in)	

EXAMEN

- o prelucrare → ce se întâmplă în Vertex Shader / Fragment Shader și tipul (in, out, uniform) și cum se face legătura între Vertex Shader și Fragment Shader
- stabilire sistem de coordonate în care se calculează iluminarea și calculul acesteia (ex: în spațiul obiect / real / de vizualizare)

CURS 7

MODEL EMPIRIC → calculează iluminarea într-un punct pe suprafața unui obiect
→ are 3 componente:

- **AMBIENTALĂ** (lumina care luminează uniform)

$$I_a = I_{a_\Delta} \cdot K_a$$

- **DIFUZĂ** (lumina reflectată uniform în toate direcțiile ⇒ depinde de unghiul de reflexie)

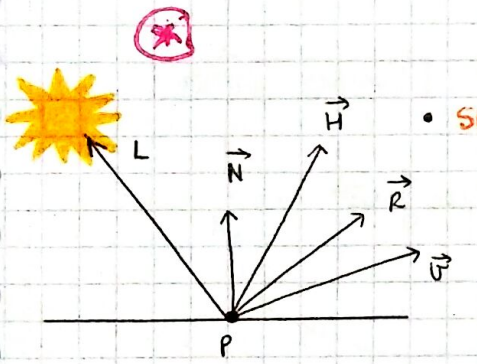
$$I_d = I_{\Delta_d} \cdot K_d \cdot (\vec{N} \cdot \vec{L}) \cdot \text{fatenuare}$$

- **SPECULARĂ** (oferă strălucire obiectelor)

$$I_\Delta = I_{\Delta_\Delta} \cdot K_\Delta \cdot (R \cdot V)^m \cdot \text{fatenuare}$$

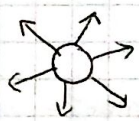
$$= I_{\Delta_\Delta} \cdot K_\Delta \cdot (N \cdot H)^m \cdot \text{fatenuare}$$

$$\vec{H} = \vec{L} + \vec{V}$$

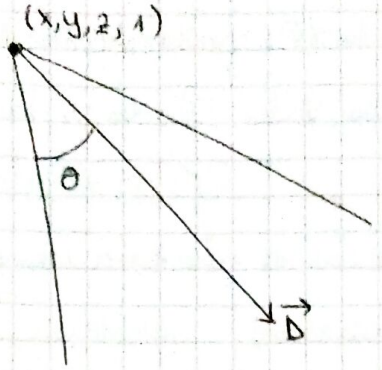


pentru a diferenția obiectele cu aceleași prop. dar la distanțe diferite

SURSE DE LUMINĂ → POZIȚIONALE
→ DIRECȚIONALE

SURSE POZIȚIONALE	SURSE DIRECȚIONALE
- plasare la distanță finită	- plasare la distanță infinită
 $w=1$ $w \neq 0$	- lumina apare după o direcție
⇒ lumină de tip spot (lumina cade în interiorul unui con)	$w=0$

LUMINA DE TIP SPOT



Lumina din interior poate fi constantă sau poate fi variabilă (specificată prin spot exponent)

1. INFO PUNCT (vertex/ fragment) → P(x, y, z, w), N

2. INFO SURSĂ → poziție/ direcție sursă
→ componente: I_a, I_{d_s}, I_{s_s}
→ coeficienți pentru factorul de atenuare (K_c, K_l, K_q) — dependenți de sursă

$$f_{at} = \frac{1}{K_c + K_l \cdot d + K_q \cdot d^2}$$

LUMINĂ DE TIP SPOT: $\vec{D}$, θ, spot exponent

PROPRIETĂȚI DE MATERIAL PER OBIECT: K_a, K_d, K_s, K_e, n

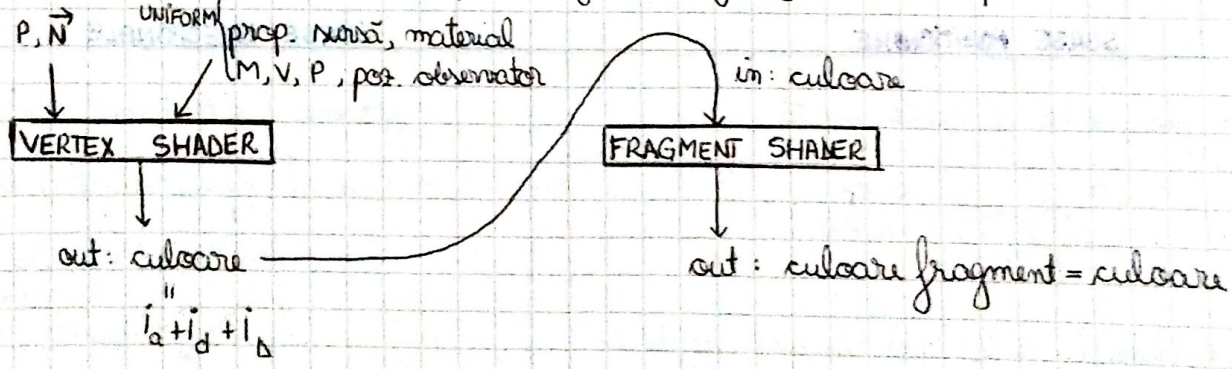
K_e → COMPONENTĂ INCANDESCENTĂ

ILUMINARE PENTRU UN OBIECT (TOATE FEȚELE)

Modele analitice → Lambert ⇒ o singură intensitate per față
→ Gouraud, Phong } intensități diferite

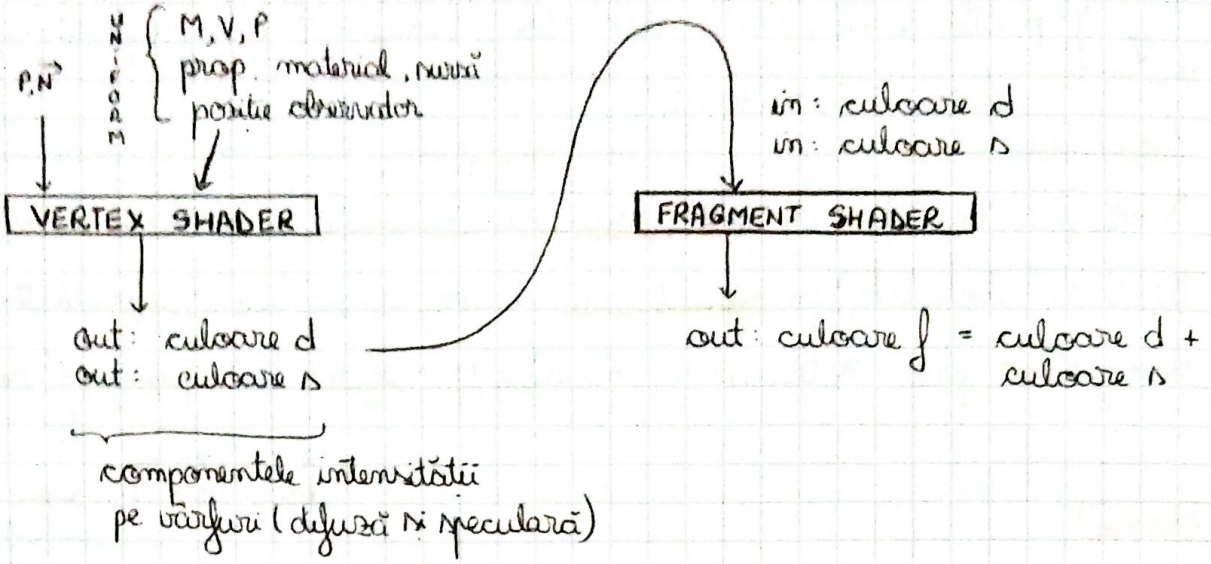
LAMBERT ⇒ calculează intensitățile în vârfuri și apoi interpolează / rasterizează

GOURAUD ⇒ calculează intensitățile pentru fiecare fragment și apoi rasterizează



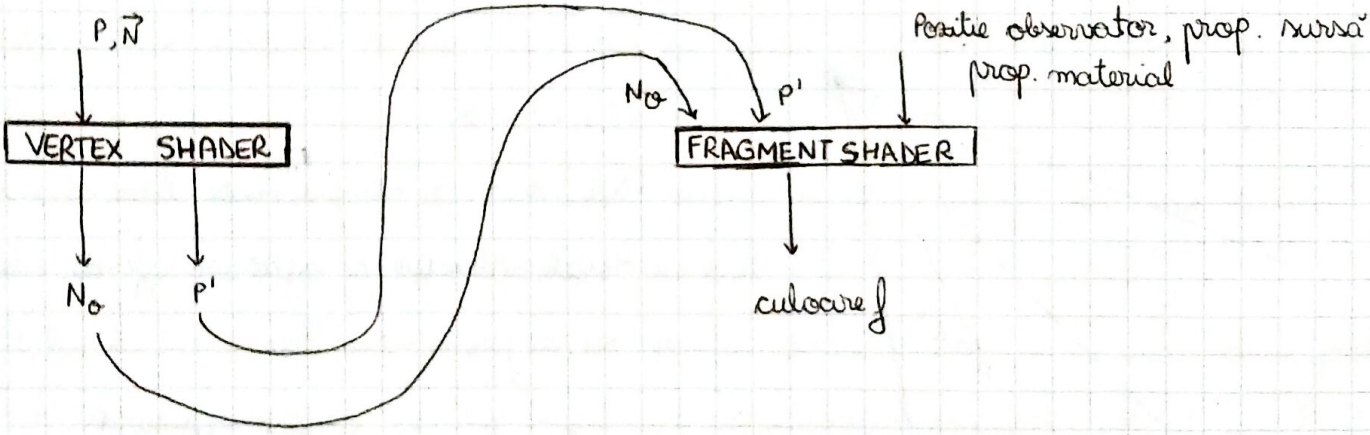
GOURAUD MODIFICAT SPRE PHONG

- se apropie de realitate, dar fără a adăuga calcule suplimentare
- obțin I_d și I_s, dar nu calculez culoarea finală ca la Gouraud; calculul se face în Fragment Shader, ca la modelul Phong



PHONG

- calculează culoarea în Fragment Shader în funcție de cele 3 componente



EXAMEN

- Care sunt pașii care trebuie să fie făcuți de Fragment/Shader?

  Ex. rezolvare pentru Fragment Shader:

  - desenul de la începutul cursului (marcat cu *)
  - formulele pentru I_d, I_s
  - proprietăți material
  - proprietăți sursă

$\vec{L}$ = (poz. sursă - coordonate vârf/fragment)

OBSERVAȚIE! Trebuie să ne asigurăm că sunt în același spațiu

$\vec{V}$ = poziție observator - coordonate vârf/ fragment

$\vec{L}$
$\vec{V}$ } poziție sursă, poziție observator, coordonate vârf/fragment definite în același spațiu de coordonate

Dacă fac calculele în spațiul obiect, avem nevoie de matricea de modelare

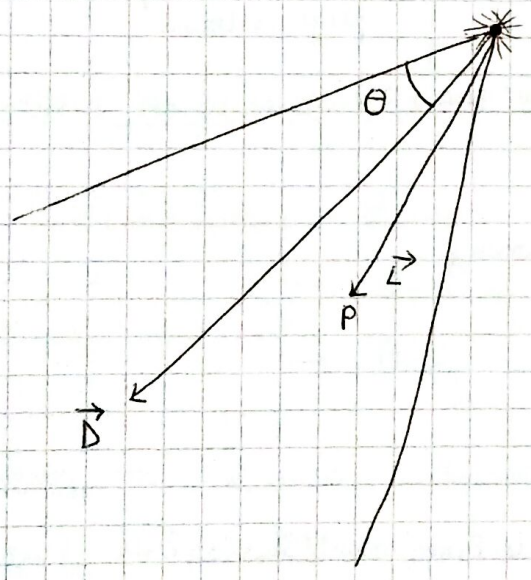
Dacă fac calculele în spațiul real, avem nevoie ca P' din Vertex Shader să fie transformat prin „M·P", iar N0 să fie transformată prin M'·N.

↙ ne interesează doar primele 3 linii

EXAMEN

Cum se implementează modelul Phong / Gouraud? Explicați cum funcționează.

LUMINA DE TIP SPOT



$\measuredangle(D, L) < \theta$

$(D, L) > \cos\theta \Rightarrow$ primește lumină

Altfel, punctul se află în afara conului și nu va primi lumina.

coef spot = $(D \cdot L)^{\text{spot exponent}}$

i_D * = coef. spot
i_d * = coef. spot
} doar dacă $D \cdot L > \cos\theta$, altfel nu are sens

DETERMINAREA VIZIBILITĂȚII

VIEW-FRUSTUM CULLING → păstrează doar obiectele din volumul de vizualizare

După asta, ne interesează cele mai apropiate obiecte de observator

Slide 2 (13. Vizibilitate.pdf)

linii punctate ⇒ nu vor fi vizibile de observator din cauză că alte fețe ale obiectului sunt mai apropiate de observator (Back-face culling) ⇒ eliminăm fețele spate

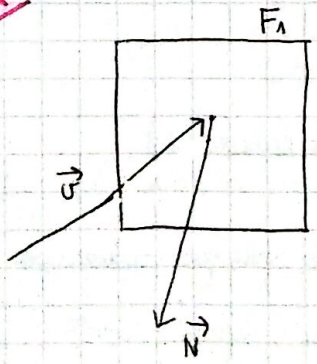
↳ raportat la obiectul însuși

Totuși, pot exista fețe care se suprapun ⇒ avem nevoie de detecția zonelor nevizibile ale unui obiect raportate la celelalte obiecte

BACK-FACE CULLING

- valabilă pentru obiecte convexe
- nu ne garantează rezultatele pentru obiectele concave
- avem nevoie de N pentru fiecare față și de vectorul $\vec{v}$
- fața trebuie parcursă în sens trigonometric pentru ca N să fie orientată spre exteriorul obiectului
- dacă știm coordonatele feței, calculez vectorii a două laturi succesive și după fac produsul vectorial pentru a determina normala
- pentru a calcula $\vec{v}$, avem nevoie de poziția observatorului și un punct de pe față (se consideră de obicei punctul în care se calculează normala). Sensul lui $\vec{v}$ este către punctul de pe față.

VARIANTA 1

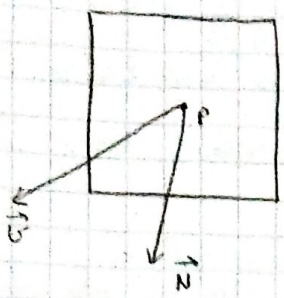


$\sphericalangle(N, v) > 90° \Rightarrow$ poligon față (orientată spre observator)

$\sphericalangle(N, v) < 90° \Rightarrow$ poligon spate (nu este vizibilă ⇓ trebuie eliminat)

VARIANTA 2

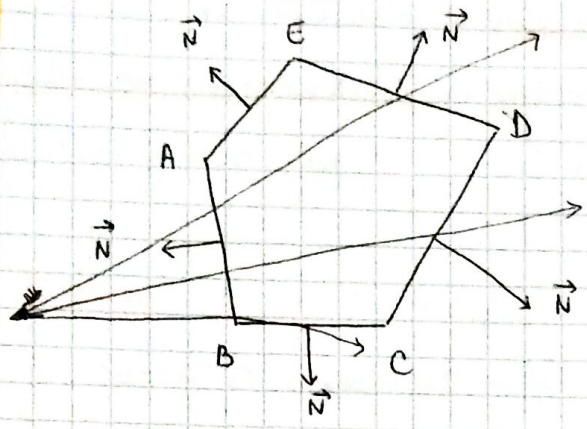
Se consideră v ca la iluminare (orientare către observator)



$\sphericalangle(N,v) < 90^\circ \Rightarrow$ poligon față

$\sphericalangle(N,v) > 90^\circ \Rightarrow$ poligon spate

ÎNTREBARE TEST



Care fețe sunt fețe față și care fețe spate?

AB → față

EA → față

BC, CD, ED → spate

glEnable → activează folosirea back-culling-ului

glCullFace → ce fețe elimin

glFrontFace → specifică care sunt fețele față în funcție de modul de parcurgere

ALGORITMI DE ELIMINARE A PĂRȚILOR NEVIZIBILE

- în spațiul imagine: calculele le fac când am coordonate ecrane (2D)

De fapt: Avem nevoie de x, y, z și de adâncime ⇒ viewport (cub de latură 2 centrat în origine)

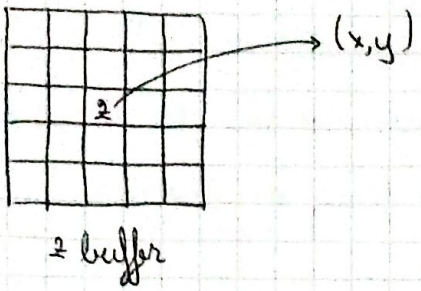
(x, y, z)

coord. ecran → coord. reală (definită în cubul de latură 2)

→ Pot să merg în Frame Buffer și să specific culoarea specifică pixelului respectiv

Când sunt în Fragment Shader, orice coordonată are valori între [-1, 1]. (din cauza cubului de latură 2)

ALGORITMUL Z-BUFFER



găsesc fragmentul curent: (x, y, z_c)

Când găsesc un obiect care se mapează în pixelul (x,y), compar adâncimea. Prima dată va fi adevărat (nu mai există altceva mapat în pixelul (x,y)).

$\Rightarrow$ general: ∞

SAU : z = plan spate volum de vizualizare

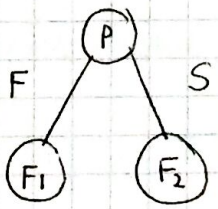
- buffer culoare
- buffer adâncime

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \xrightarrow[\text{împărțire perspectivă}]{M,V,P} \begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} \xrightarrow[\text{fereastră - poartă}]{\text{transformare}} \begin{pmatrix} x \\ y \\ z - z' \end{pmatrix}$$

SPAȚIU OBIECT — VIEWPORT 3D

glDepthMask → dacă actualizăm bufferul de adâncime sau nu

ALGORITMUL DE PARTITIONARE BINARĂ A SPAȚIULUI

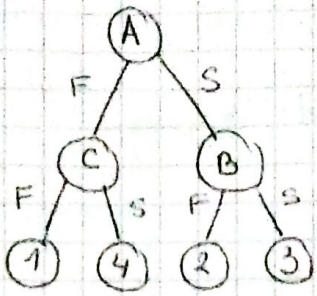


Parcurg arborele în funcție de poziția observatorului

CAZUL 1 Observatorul nu e de aceeași parte cu normala (e în spate) $\Rightarrow$ parcurg de la ce e mai depărtat de observator $\Rightarrow$ F_1, F_2 (parcurgere SRD)

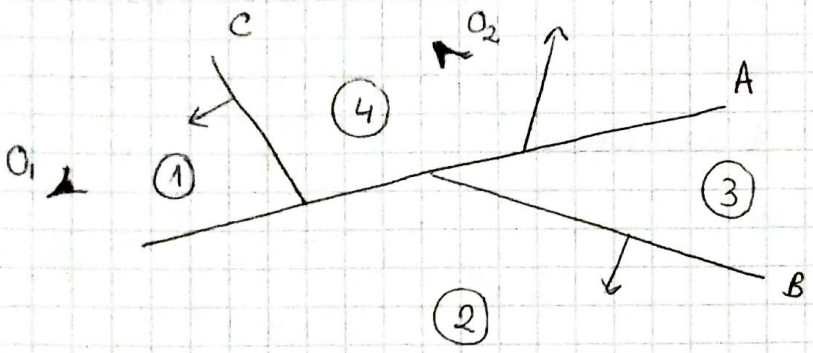
CAZUL 2 Observatorul e de aceeași parte cu normala (e în față) $\Rightarrow$ F_2, F_1 (parcurgere DRS)

Slide 26 (13. Vizibilitate.pdf)



F = față
S = spate

Afișăm doar fețele vizibile de la fiecare frunză (prin parcurgere SDR/DRS)



Observator 1

Este de aceeași parte cu A și cu B ⇒ 3, 2, 4, 1

Observator 2

Este de aceeași parte cu A ⇒ parcurgere DRS ⇒ încep cu subarborele drept (B)

Este de altă parte față de B ⇒ parcurgere SRD ⇒ 2, 3

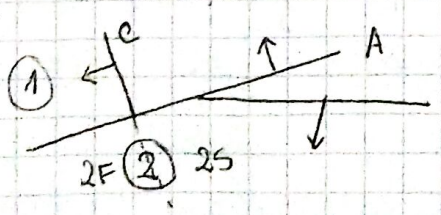
Trecem la subarborele stâng ⇒ 1, 4

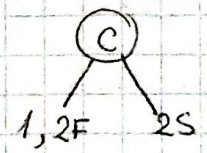
⇒ 2, 3, 1, 4

Concluzie → arborele a fost parcurs DRS
→ subarborii au fost parcurși SRD

OBSERVAȚII !

- Arborele trebuie reconstruit de fiecare dată când cera se modifică
- Planul de partiționare se alege fie aleator, fie astfel încât să avem cât mai puține intersecții.





EXAMEN

- Explicare algoritm partiționare / z-Buffer : cum se construiește, pași, parcurgere etc.
  NU pseudocodul!
- Se dă arborele asociat unei scene și poziția observatorului. Se cere parcurgerea arborelui de partiționare

CURS 8

ALGORITMUL WARNOCK

- combinație între cele 2 tipuri de algoritmi
- divizarea e recursivă până nu mai are sens → se ajunge la un pixel
  → până fiecare față e singură (nu mai sunt intersecții)
- intersectat ⇒ se intersectează și se determină zona comună ⇒ zona comună va avea culoarea feței, iar restul culoarea de fond
  ZONĂ INCLUSĂ / ZONĂ DISJUNCTĂ
- disjunct ⇒ se elimină

- La final vor rămâne doar fețe poligonale acoperitoare și incluse
- Sortare în funcție de adâncime: dacă cea mai apropiată de observator e acoperitoare, nu se mai face divizare și se afișează fața poligonală acoperitoare
- Avem nevoie de → coordonatele de adâncime (reale)
  → mapare în zona de afișare (coordonate ecran)

⇒ Folosim Viewport-ul, deoarece acolo avem aceste coordonate

ALGORITMI DE DECUPARE

Zona de decupare va fi de obicei ecranul sau volumul de vizualizare.

DETERMINARE DACĂ UN SEGMENT E INCLUS ÎN ZONA DE DECUPARE

- acceptat ⇒ în totalitate în zona de decupare
- rejectat ⇒ nu e deloc în zona de decupare
- intersectat ⇒ doar o parte se află în zona de decupare

Ne interesează valorile t pentru care punctele intersectează zona de decupare (în mod normal s-ar folosi ecuația parametrică, dar putem folosi Cohen-Sutherland, deoarece este mai rapid. Acesta realizează decuparea față de o zonă dreptunghiulară)

ALGORITMUL DE DECUPARE LINII COHEN-SUTHERLAND

- Se codifică zona de decupare și zonele exterioare ⇒ 9 zone ⇒ sunt necesari 4 biți
- Se clasifică capetele în funcție de zona unde se află

Slide 7 (14. Algoritmi de decupare. pdf)

(b_2) → este în afară

un capăt: 0010
celălalt capăt: 0110
} Sunt diferite de 0 ⇒ sunt în exterior (nu înseamnă că nu intersectează)

Au în comun un bit 1 pe aceeași poziție. Dacă facem & între codificări, obținem 0010 ⇒ în exterior

Analog pentru b_3 și b_5.

Codificări 0 ⇒ e inclus (CAZ TRIVIAL)

Codificări ≠ 0 ⇒ valoarea adevărată ⇒ în exterior (REJECTARE TRIVIALĂ)

Slide 8 (14. Algoritmi de decupare. pdf)

AB → inclus (acceptare trivială)

GH → rejectare trivială

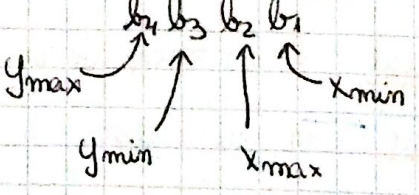
CD → nici acceptare, nici rejectare trivială ⇒ calcul punct intersecție

codif(C) = 0

codif(D) = ?

Analizând biții în ordine putem să ne dăm seama cu ce intersectează.

- ∩ x_{min} dacă $b_1 = 1$
- ∩ x_{max} dacă $b_2 = 1$
- ∩ y_{min} dacă $b_3 = 1$
- ∩ y_{max} dacă $b_4 = 1$



$\Rightarrow CD \cap y = y_{min} = \{i_1\}$

Punctul găsit se înlocuiește $\Rightarrow$ noul segment : Ci_1

Analizăm noul segment : Ci_1

$codif(C) = 0$
$codif(i_1) = 0$
$\Big\} \Rightarrow$ ACCEPTARE TRIVIALĂ

Segmentul EF :

Determin intersecția pe baza lui E :

$EF \cap \underbrace{y = y_{max}}_{bit\,4 = 1} = \{i_1\}$

i_1 înlocuiește $E \Rightarrow$ noul segment: i_1F

$codif(i_1) = 0$

$i_1F \cap x = x_{max} = \{i_2\} \Rightarrow$ noul segment $= i_1i_2$
$codif(i_2) = 0$
$\Big\} \Rightarrow$ Acceptare trivială

Slide 18

P_1P_2

- verific pe rând $x_{min}, x_{max}, y_{min}, y_{max} \Rightarrow$ prima dată fac intersecția cu x_{max} $\Rightarrow$ se obține $i_1 \Rightarrow$ noul segment : P_1i_1

P_1i_1

$codif(i_1) = 1000$

$P_1i_1 \cap y_{max} = \{i_2\} \Rightarrow$ noul segment : P_1i_2

$codif(i_2) = 0 \Rightarrow$ Acceptare trivială

Slide 25

Nu se încadrează în rejectarea trivială (un bit 1 pe aceeași poziție).

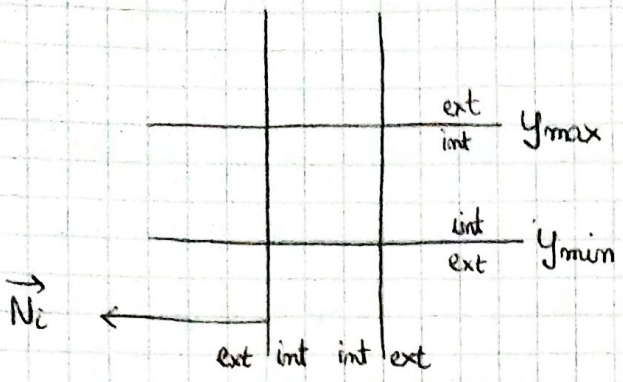
P_1P_2
↑

$P_1P_2 \cap y = y_{min} \Rightarrow i_1P_2 \Rightarrow$ Rejectare trivială

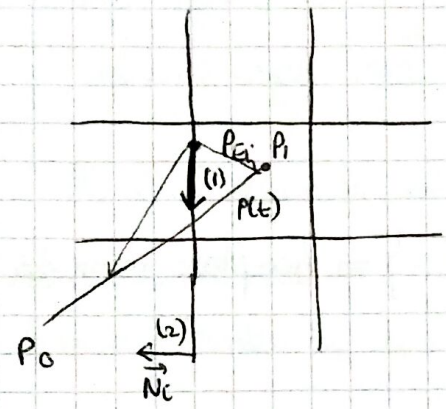
0010 0010

ALGORITMUL DE DECUPARE LINII CYRUS-BECK

Se bazează pe ecuația parametrică a dreptei.



Slide 28



Se face produsul scalar între (1) și (2) și ne interesează doar când are valoarea 0. Dacă are valoarea <0 sau >0, atunci e exterior

$\vec{N_i}$ vector normalizat $\Rightarrow \vec{N_i}[-1, 0]$
paralel cu Ox

↳ vector perpendicular pe latură (în acest caz x_{min})

$P_{Ei}(x_{min}, y)$
↑ oarecare

$$t = \frac{N_i(P_0 - P_{Ei})}{-N_i(P_i - P_0)}$$ → nu depinde de y

După găsirea punctului de intersecție, se calculează t-urile corespunzătoare punctului de intersecție.

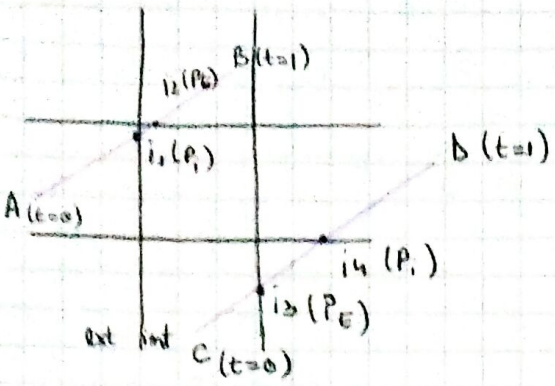
Slide 29

AB ∩ fiecare dreaptă ce delimitează zona de decupare

dreapta AB ∩ $x = x_{min}$
AB ∩ $x = x_{max}$
AB ∩ $y = y_{min}$
AB ∩ $y = y_{max}$

} ⇒ t-uri pentru punctele de intersecție

! Ne interesează doar $t \in [0, 1]$ (aparțin segmentului). Un capăt al segmentului are t=0, iar celălalt t=1



i_1, i_2, i_3 și i_4 se determină folosind t.

Sens de parcurgere segmente: A→B, C→D

A→B ↳ am început din ext și am ajuns în int pt. i_1
↳ am început din int și am ajuns în ext. pt. i_2

C→D ↳ am început din int și am ajuns în ext. pt. i_3
↳ am început din ext. și am ajuns în int. pt. i_4

PUNCTE DE INTERSECȚIE

- potențial de intrare (Pi) : parcurgere din ext către int
- potențial de ieșire (PE) : parcurgere din int către ext

⇒ $i_1 = Pi$ $i_3 = PE$

$i_2 = PE$ $i_4 = Pi$

⇒ $t_{i_1} < t_{i_2}$ și $t_{i_3} < t_{i_4}$

↳ potențial intrare > potențial ieșire

↳ potențial intrare < potențial ieșire

Ne interesează doar cazul când potențialul de intrare < potențialul de ieșire

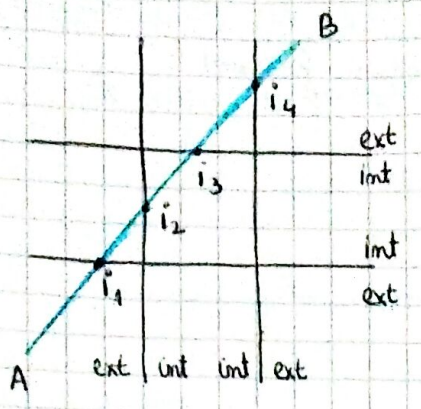
$t_{i_1} < t_{i_2}$ ✓

$t_{i_3} < t_{i_4}$ ✗

Punctele de intersecție sunt date de t-urile calculate

↳ Dacă $t_{P_i} < t_{P_E}$ ⇒ segment inclus $(P(t_{P_i}), P(t_{P_E}))$

↳ Dacă $t_{P_i} > t_{PE}$ ⇒ segment exclus zonei de decupare



$i_1 \rightarrow$ Pi

$i_2 \rightarrow$ Pi

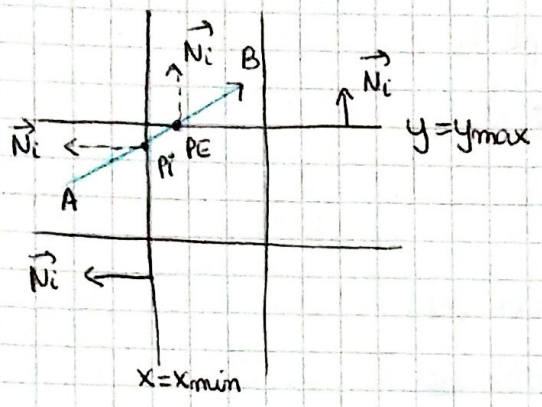
$i_3 \rightarrow$ PE

$i_4 \rightarrow$ PE

Pi $\Rightarrow \max(t_{Pi}) \Rightarrow$ Alegem punctul cu potențialul de intrare maxim $\Rightarrow i_2$

PE $\rightarrow \min(t_{PE}) \Rightarrow$ Alegem punctul cu potențialul de ieșire minim $\Rightarrow i_3$

ALGORITMUL LIANG-BARSKY



pentru Pi : $\sphericalangle(\vec{N_i}, \vec{AB}) > 90°$

pentru PE : $\sphericalangle(\vec{N_i}, \vec{AB}) < 90°$

$$t = \frac{(P_0 - P_{Ei}) \cdot N_i}{-N_i \cdot \Delta}$$

Ne putem da seama dacă e Pi/PE pe baza numitorului lui t.

Zona de decupare: $(x_{min}, y_{min}\ ;\ x_{max}, y_{max})$

- Pentru fiecare latură l_i a zonei de decupare se determină t-ul pentru punctul de intersecție.

  - $(x_1, y_1) - (x_2, y_2) \cap l_i$
  - Se păstrează doar $t \in [0,1]$ (aparțin segmentului)
  - Se clasifică punctul PE/Pi (de intersecție)

ALGORITM INITIAL

!!! Implicit t-ul pentru PE este 1 și pentru Pi este 0.

  - Actualizare valoare pentru t_{PE} / t_{Pi}

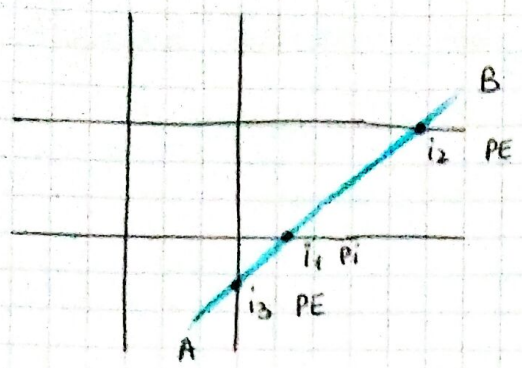
Dacă $t_{PE} < t_{Pi}$, atunci segment rejectat

Altfel $(P(t_{Pi}), P(t_{PE}))$

dacă $t_{PE} < t_{Pi} \Rightarrow$ segment rejectat

↑ corecție algoritm inițial

GREȘEALĂ :



Din algoritmul anterior se obține că $i_1 i_2$ este segment inclus ⇒ FALS

MODIFICARE ALGORITM în for la final se verifică dacă $t_{PE} < t_{Pi}$ și dacă e adevărat, ne oprim ⇒ segment rejectat

Se adaugă modificarea cu albastru din algoritmul inițial.

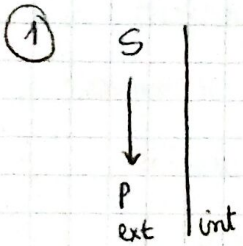
DECUPAREA POLIGOANELOR

ALGORITMUL SUTHERLAND-HODGMAN

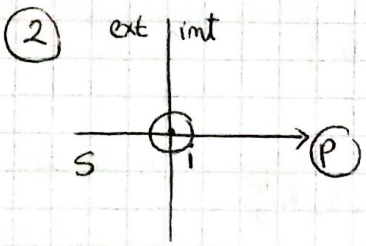
La fiecare pas se consideră o față poligonală.

S → vârful examinat la pasul anterior

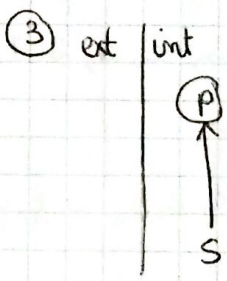
P → vârful examinat la pasul curent



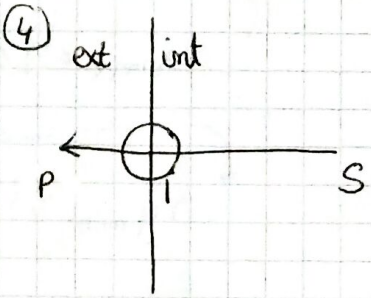
Nu se păstrează nimic



Păstrăm I și P



Păstrez P (S a fost analizat deja anterior)



Păstrăm punctul de intersecție I

Slide 44 Poligon P

$P \cap x_{min} \Rightarrow P' \cap x_{max} \Rightarrow P'' \cap y_{min} \Rightarrow P''' \cap y_{max} \Rightarrow P_f$

$P \cap x_{min}$

5 → 1 = latură curentă Se trece din int în ext ⇒ păstrez a (punct intersecție)

1 → 2 Păstrez b (punct intersecție), 2

2 → 3 Nu păstrez nimic

3 → 4 păstrez 4

4 → 5 păstrez 5

$\Rightarrow P \cap x_{min} = \{a, b, 2, 3, 4, 5\}$

se intersectează cu x_{max}

$\cap x_{max} = \{a, b, 2, c, d, 5\}$

$\cap y_{min} = \{e, a, b, 2, c, f\}$

$\cap y_{max} = \{e, a, b, g, h, f\}$ — Poligonul interior zonei de decupare

Slide 45

$P \cap x_{min} = \{a, 1, 2, 3, 4, 5, 6, 7, 8, b\}$

$\cap x_{max} = \{a, 1, 2, 3, c, d, 6, 7, 8, b\}$

$\cap y_{min} = \{e, a, 1, 2, 3, c, f, g, 7, 8, h\}$

$\cap y_{max} = \{e, i, j, 2, m, f, g, 7, 8, h\}$ — Poligonul interior zonei de decupare

Slide 52

$P \cap x_{min} = \{1, a, b, 4, 5, c, d, 8\}$

$\cap x_{max} = \{1, a, b, 4, 5, c, d, 8\}$

$\cap y_{min} = \{1, a, b, e, f, d, 8\}$

$\cap y_{max} = \{g, 1, h, i, b, e, f, c\}$

sunt laturi false care ar trebui eliminate

CONCLUZIE

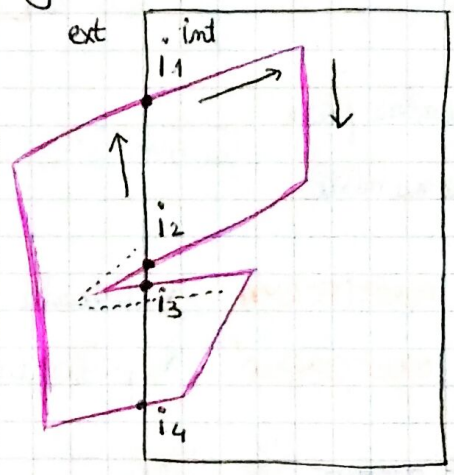
Algoritmul Sutherland-Hodgman → funcționează mereu corect pentru poligoane convexe
→ nu garantează corectitudinea în cazul poligoanelor concave (pot apărea laturi false)

SOLUȚIE: Algoritmul Weiler-Atherton

ALGORITMUL WEILER-ATHERTON

- parcurge conturul fețelor poligonale
- la primul punct de intersecție continui să merg pe fața poligonală până:
  - se termină
    SAU
  - am ajuns la următorul punct de intersecție

Când ajung înseamnă că ies din zona de decupare ⇒ Continui să merg pe conturul zonei de decupare până ajung la un punct de intersecție anterior
După continui să termin de parcurs fața poligonală.



$A \to i_1$

Parcurg pe fața poligonală până la i_2 / terminare față

Găsesc i_2 ⇒ Schimb parcurgerea
⇒ Parcurg conturul
⇒ Încep iar de la i_2 (punctul care mă scotea din contur) și parcurg în mod asemănător

Se alege un sens pentru contur și pentru fața poligonală. Dacă punctul de intersecție e de potențial exterior, pot continua să parcurg până la un nou punct de potențial interior.
Când se ajunge la punctul de potențial interior, schimb parcurgerea pe contur.

↳ așa a fost eliminată zona falsă $i_2 i_3$

DECUPAREA ÎN 3D

- **COHEN SUTHERLAND**
  - Codificarea se face pentru fiecare zonă interioară / exterioară
    ⇒ Sunt necesari 6 biți
  - Intersecțiile se fac cu dreapta și un plan
- **SUTHERLAND - HODGMAN**

  Latura din fiecare față poligonală față de fiecare plan care delimitează cubul ⇒ se obține fața poligonală interioară din volumul de vizualizare

EXAMEN

Cum se calculează punctele de intersecție (nu algoritm), cum se clasifică etc. pt. algoritmul Cohen-Sutherland/ Cyrus-Beck etc.

CURS 9

- umbre pure
- penumbre

SURSĂ PUNCTIFORMĂ → distanță finită (proiecție perspectivă)
SURSĂ DIRECȚIONALĂ → distanță infinită (proiecție paralelă)

UMBRE PURTATE → umbra pe care o generează un obiect asupra altui obiect în scenă
UMBRE PROPRII → generat de obiectul însuși asupra fețelor sale

Umbrele proprii sunt fețele nevizibile din poziția sursei de lumină. Dacă sunt mai multe surse de lumină, se va determina ce nu se vede pentru fiecare sursă în parte.

Umbrele purtate se obțin prin proiecția perspectivă cu centrul de proiecție în sursa de lumina (ne trebuie obiectul care generează umbra și unde se generează umbra)

Slide 7 (15. Umbre. pdf)

Știm ecuația planului și coordonatele sursei de lumină (L) ⇒ avem nevoie de V

V se calculează prin intersecția dintre intersector și planul umbrei

↳ nu se cere la examen

- lumină direcțională ⇒ w = 0 (parabolă)
- V se calculează utilizând cei 2 vectori paraleli
- proiectorul este paralel cu planul de proiecție

Slide 14

Avem nevoie de fețele pe care se generează umbre + ecuațiile acestor plane

⇒ Ineficient

Slide 15

Nu va fi afișată ca în imaginea de pe slide

Conul generează mai mult ⇒ rezultatul e cel din slide-ul 17 (umbră și în lateral și sub)

Soluție: Activăm z-buffer pentru a rămâne ce e mai aproape de observat

Rezultat: Slide 19

↳ este așa, deoarece se lucrează cu adâncimi reale

⇒ Trebuie să condiționăm afișarea de un alt aspect ⇒ Folosim Stencil Buffer oferit de OpenGL (păstrăm o valoare pentru a condiționa afișarea)

z-buffer activat ⇒ modificarea din bufferul culorii se face după ce se actualizează bufferul de adâncime (se modifică doar dacă e mai aproape de observator)

Când se folosește z-buffer și stencil-buffer, se pune problema dacă bufferul culorii e actualizat doar dacă se trece testul de stencil-buffer. Dacă e falsă, nu se face nimic la afișare, ci doar se actualizează stencil bufferul.

glStencilFunc → specific condiția

glStencilOp → modalitatea în care se actualizează

CONDIȚIE STENCIL - BUFFER:

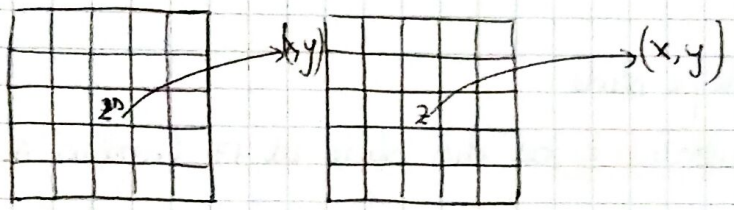
Stencil_buffer (x,y) & mască funcție ref & mască

parametrii glStencilFunc

SLIDE 27

Părțile lipsă din ceainic nu sunt afișate, deoarece nu îndeplinesc condiția, nu pentru că nu au loc în ecran.

ALGORITM Z-BUFFER CU UMBRE



$z^\wedge$ (sursă) z (observator)

$z^\wedge$ poate fi văzută ca o imagine (~textură) ⇒ se face maparea pe o textură

afișarea scenei ⇒ aplicarea z-bufferului față de observator. (comparațiile se fac în Fragment Shader

Se aplică z-buffer din sursa de lumină ⇒ $z_\wedge$. După se aplică z-buffer din punctul observatorului

Din $z^\wedge$ (privită ca o textură), ne interesează adâncimile, nu culorile

↳ se generează prin poziție

OBSERVAȚIE !

MVP este raportată la sursa de lumină.

- Fragment Shader-ul nu mai generează la ieșire o culoare, ci o adâncime (o valoare reală)
- În programul principal se asociază o textură frame buffer-ului.
- Accesarea texturii se face prin coordonate de texturare

(x, y, z)

coordonate [-1,1]

↳ vrem să le folosim drept coordonate de texturare (valori între [0,1]) ⇒ Trebuie să împărțim la 2 intervalul și să adunăm 0.5 ⇒ Scalare și translație suplimentară pentru a ajunge de la [-1,1] la [0,1]

M, P, V față de sursa de lumină ([-1,1]) + Scalare + translație ([0,1]) ⇒ coordonate de texturare

Vertex Shader + Fragment Shader care calculează adâncimile

Vertex Shader + Fragment Shader care calculează culorile

⇒ la ieșire Vertex ⇒ Shadow Coord

la ieșire Vertex UV ⇒ coordonate de texturare proprii (asociate vârfurilor)

Fragment Shader

ieșire : culoare

uniform { textura proprie
textura creată prin generarea adâncimilor

Se împarte la w ShadowCoord pentru a realiza împărțirea perspectivă (ne asigurăm că w = 1)

⇒ Rezultat: decodarea adâncimile cu valori reale, se întrepătrund (slide 46)

SOLUȚIE: scădem un offset pentru a scăpa de întrepătrundere (slide 47)

Dacă sunt mai multe surse de lumină:

- textură pentru fiecare sursă de lumină

  nr. texturi = nr. surse lumină

- se accesează toate texturile în fragment shader pentru a verifica dacă sunt în umbră sau nu (umbra poate fi variabilă în funcție de câte zone de umbră se află)

VOLUME DE UMBRĂ

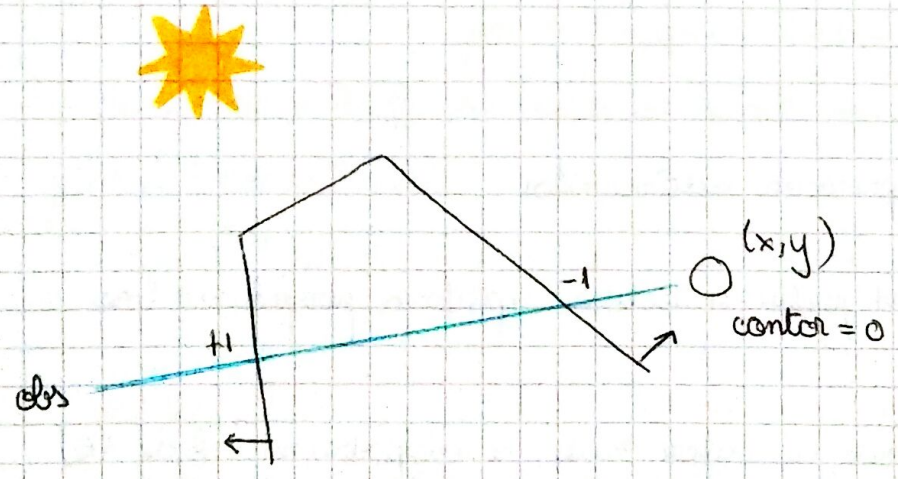
Fiecare obiect poate să genereze umbre în interiorul unui volum (=piramidă)

- vârful în sursa de lumină
- corespunzător fiecărui corp generator de umbră

Trebuie să știm dacă un obiect se află în interiorul volumului de umbră ⇒ asociem un contor. Fiecare fragment va avea asociat un contor+=1 dacă raza intersectează o dată volumul de umbră sau un contor+=-1 dacă raza intersectează de 2 ori volumul de umbră (o dată intră, o dată iese)

SLIDE 53

- trasăm raza de la observator la observator
- +1 la contor când raza intră într-un volum de umbră, -1 când raza iese din volumul de umbră



Punctele de intersecție s-ar mapa în același punct (x, y) ⇒ sunt mai în față

Afișăm volumul de umbră
Considerăm doar fețele față ⇒ găsim un punct de intersecție
Considerăm doar fețele spate ⇒ găsim un punct de intersecție

Totuși, nu ar trebui să afișăm volumul de umbră, ci doar să simulăm
→ Afișăm volumul de umbră fără să modificăm bufferul de culoare și cel de adâncime

fețe față + test z-buffer ⇒ +1
fețe spate + test z-buffer → -1

La final se vor afișa doar obiectele care au contor=0

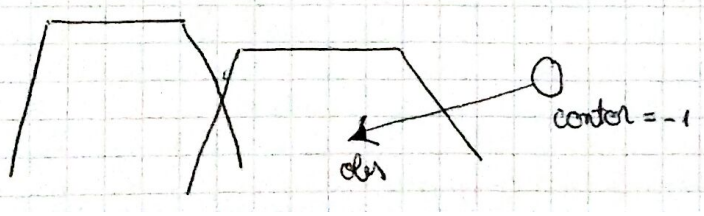
ALGORITMUL 2-PASS PENTRU VOLUME DE UMBRĂ

1. Se afișează scena fără iluminare (folosind numai lumina ambientală) cu z-bufferul activat.
   - Se calculează volumele de umbră
   - inițializare contor fragment pentru toate fragmentele posibile cu 0
   - Se dezactivează modificarea buffer-ului de adâncime
   - Se dezactivează modificarea buffer-ului de culoare

   CORECT: numără volume în care se află observatorul

2. Se simulează afișarea fețelor față pentru fiecare volum de umbră. Dacă testul z-buff e adevărat, atunci incrementează contor fragment
3. Se simulează afișarea fețelor spate pentru fiecare volum de umbră. Dacă testul z-buff e adevărat, atunci decrementează contor fragment
4. Se activează folosirea iluminării, buffer culoare, se modifică condiția de test din z buffer (dacă z-ul curent pentru fragment curent == z-ul aflat în z-buffer)
   Se afișează scena cu iluminare doar dacă contor fragment = 0

PROBLEMĂ ALGORITM 2-PASS PENTRU VOLUME DE UMBRĂ



Obiectul nu va fi afișat cu iluminare, deși e în exteriorul volumelor de umbră. Față de cazul anterior, observatorul se află în interiorul volumului de umbră $\Rightarrow$ contorul ar trebui inițializat cu 1, nu cu 0.

CORECTARE ALGORITM:

contor fragment = nr. volume umbră în care se află observatorul

DEZAVANTAJ 2-PASS PENTRU VOLUME DE UMBRĂ : este destul de costisitor să calculăm numărul de volume de umbră în care se află observatorul pentru a inițializa contorul

CURS 10

REDARE UMBRE

→ PROIECȚIE PLANĂ

Dezavantaj: trebuie să știm ecuațiile planelor, fețele pe care se generează umbre

→ SHADOW MAPPING (HĂRȚI DE UMBRE)

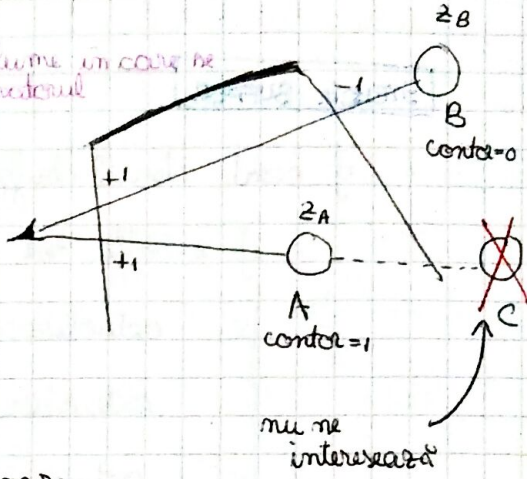
Se modifică z-buffer când se găsește un fragment mai aproape de observator. Dacă există un alt obiect între obiect și sursa de lumină, înseamnă că obiectul este umbrit (folosim adâncimile)

→ VOLUME DE UMBRĂ

- fiecare obiect este generator de umbră
- toate obiectele din volumele de umbră trebuie să apară umbrite
- bază mică volum umbră = plan care delimitează obiectul (se obține prin scheletizare 3D ⇒ se obține 2D pe baza corpului 3D)
- bază mare volum umbră = infinit

ALGORITMUL 2-PASS PENTRU VOLUME DE UMBRĂ

- trasăm razele de la observator la obiecte (corespunzător adâncimilor din z-buffer)
- verificăm dacă intersectează volumul de umbră
  - când raza intră în volumul de umbră ⇒ +1
  - când raza iese din volumul de umbră ⇒ -1
- fiecare obiect are un contor inițializat cu 0 (nr. de volume în care se află observatorul)



(1) Afișare obiecte cu lumină ambientală (se dezactivează iluminare)

(2) - activare z-buffer ⇒ se oprește doar A și B
- se simulează afișarea volumelor de umbră
  - se dezactivează modificarea buffer-ului de culoare
  - se dezactivează modificarea buffer-ului de adâncime
- se inițializează contorul pentru fiecare fragment cu 0

(3) Se simulează afișarea fețelor față pentru volumele de umbră
- se activează back-face culling-ul cu eliminare fețe spate

Dacă condiția pentru z-buffer este adevărată, contorul se incrementează.

(4) Se simulează afișarea fețelor spate pentru volumele de umbră
- se activează back-face culling-ul cu eliminare fețe față.

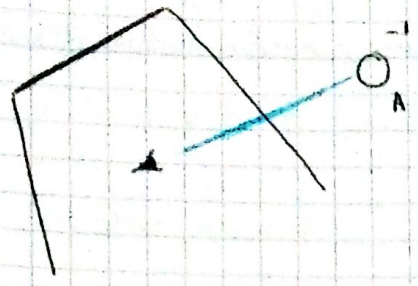
Dacă condiția pentru z-buffer este adevărată, contorul se decrementează.

! În cazul pașilor 3 și 4, condiția pentru z-buffer este următoarea:
z fragment curent mai aproape de observator față de z existent

(5) Se activează folosirea iluminării, buffer culoare, buffer adâncime

Condiție z-buffer: z fragment curent = z fragment existent

(6) Afișează cu iluminare fragmentele pentru care contorul are valoarea 0



Se obține un rezultat greșit folosind algoritmul anterior. Corectăm algoritmul:

inițializare contor cu nr. volume de umbră în care se află observatorul

STENCIL BUFFER

```
if cond Stencil-buffer
    if z-buffer test
        actualizare z-buffer
        actualizare buffer culoare ← culoare fragment
        actualizare Stencil-buffer
    else
        actualizare stencil-buffer
else
    actualizare stencil-buffer
```

glStencilFunc (cond, ref, mască)
↳ setăm condiția, referința pentru condiția din stencil-buffer

Condiție stencil-buffer:
if valoare stencil-buffer & mască funcție ref & mască

gl StencilOp (StencilFail, zfail, zpass)
↳ condiție stencil-buffer falsă
↳ modalitatea de actualizare a stencil-buffer-ului

FEȚE FAȚĂ

- zpass ⇒ se incrementează valoarea
- nu ne interesează stencil fail

FEȚE SPATE

zpass ⟹ se decrementează valoarea

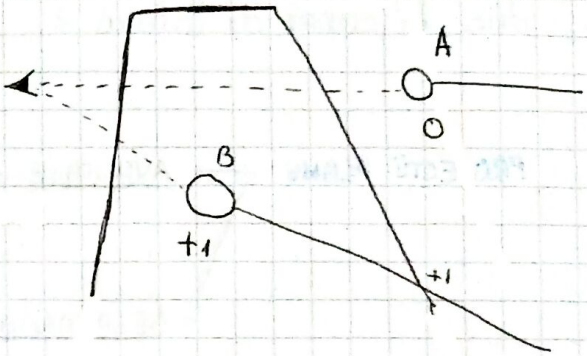
La afișare ne interesează valoarea din stencil buffer (egalitate) ⟹ ref & mască = 0

Condițiile de actualizare (zfail, zpass) nu ne interesează (rămân aceleași valori)

ALGORITMUL Z-FAIL PENTRU VOLUME DE UMBRĂ

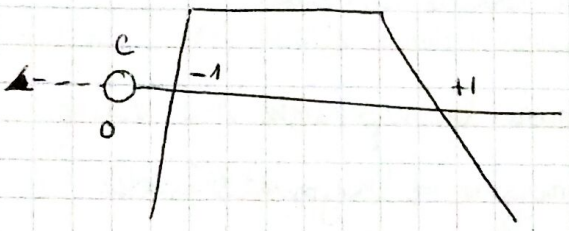
Ne interesează raza de la infinit la obiect

(raza punctată este pentru algoritmul z-pass, cealaltă pentru z-fail)



A ⟹ nu intersectează volumul de umbră

B ⟹ intersectează volumul de umbră



Modificări ale algoritmului față de z-pass:

① la fel

② inițializare contor cu 0

③ condiție z-buffer falsă ⟹ decrementare

④ condiție z-buffer falsă ⟹ incrementare

⑤, ⑥ la fel

SLIDE 64

R_1, R_2 → contor = 0 (nu se modifică)

R_3 → contor = -1 (intră în 2 volume de umbră și iese din unul)

⟹ Nu mai apare problema cu inițializarea contorului cu numărul de volume de umbră în care se află observatorul

SLIDE 70

Deoarece volumul de umbră este limitat, se consideră doar intersecțiile din cadrul volumului de umbră ⇒ ultimul fragment nu se consideră ⇒ contor = -1 ⇒ greșit

SOLUȚIE nu decupăm volumul de vizualizare corespunzător planului spate

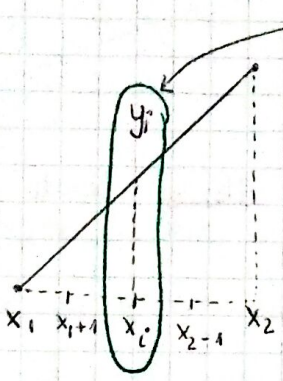
Cum facem asta? ⇒ Modificăm matricea astfel încât f va tinde la ∞. (va fi matricea de la slide-ul 73) ⇒ modificând astfel matricea, nu ne vom duce în cubul de latură 2.

PROIECȚII PLANE
- AVANTAJE
  - eficientă pentru scene statice
  - eficientă pentru puține obiecte în scenă
- DEZAVANTAJE
  - ineficientă pentru scene dinamice
  - ineficientă pentru obiecte complexe

SHADOW MAPPING
- AVANTAJE
  - se pot genera umbre transparente
  - reprezentate prin imagine ⇒ Fragment Shader ⇒ rapid
- DEZAVANTAJE
  - ineficient dacă sunt multe surse deoarece se folosește multă memorie
  - se recalculează în cazul în care se mută obiectele

VOLUME DE UMBRĂ
- AVANTAJE
  - rezultate mai bune
  - pot reda atât umbra, cât și penumbra
- DEZAVANTAJ
  - trebuie generată fiecare pereche (obiect, volum umbră)

ALGORITMI DE GENERARE SEGMENTE DE DREAPTĂ ÎN SPAȚIUL DISCRET



punct de pe dreapta reală
⇓
pixel corespunzător: $(x_i, \text{int}\, y_i)$

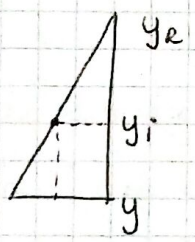
- Segmentul de dreaptă e afișat mereu corespunzător?

Segment vertical ↑| → } nu pun probleme
Segment orizontal ↑— →

Nu merge pentru cazurile când panta nu e între $[0, 1] \Rightarrow$ are un singur punct intermediar.

Ex: Slide 4

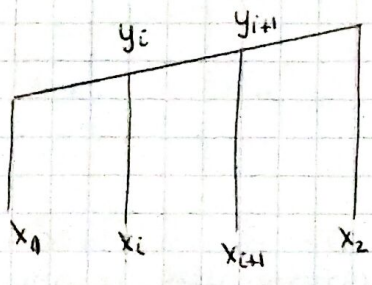
SOLUȚIE



$\Rightarrow (\text{int}(x_i), y_i)$ SLIDE 5 $\Rightarrow$ se obțin 6 puncte intermediare

$y_i = (\text{int})(mx_i + b)$

Nu trebuie făcute mereu aceste calcule, deoarece ne putem baza pe punctul anterior.



Luăm toți pixelii între x_1 și x_2.

$y_i = mx_i + b$

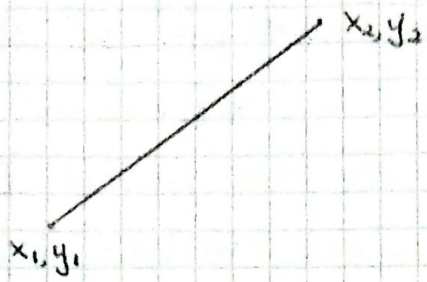
$y_{i+1} = mx_{i+1} + b = m(x_i + 1) + b = y_i + m$ (deja calculat; m: constantă pe întreg segmentul)

$y_i = mx_i + b \Rightarrow x_i = \frac{y_i - b}{m}$

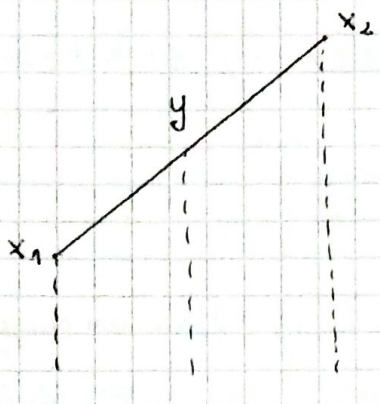
$y_{i+1} = mx_{i+1} + b \Rightarrow x_{i+1} = \frac{y_{i+1} - b}{m} \Rightarrow x_{i+1} = x_i + \frac{1}{m}$ ($\frac{1}{m}$: constantă)

CURS 11

GENERARE SEGMENT DREAPTĂ



Generăm capetele și ulterior toate punctele x din interval.



conversie la cel mai apropiat y_{ecran}

$\Rightarrow (x, y)$

operații cu nr. reale, deoarece folosim ecuația dreptei $\Rightarrow$ necesită conversie la nr. întreg

operații cu numere întregi

! Aproximarea se poate face în acest mod doar dacă panta $\in [0, 1]$.

Dacă panta > 1 (echivalent cu nr. pixeli y $\gg$ nr. pixeli x):

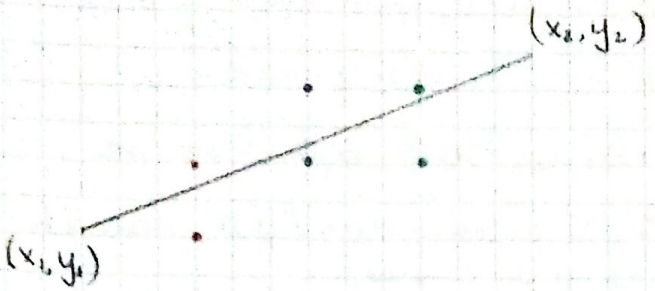
$\Rightarrow (x, y)$

conversie

ALGORITMUL BRESENHAM

- se folosește pentru a elimina calculele reale
- valabil doar pentru panta $\in [0, 1]$
- se încearcă trasarea segmentul lucrând direct pe coordonatele ecran și folosind operații întregi
- la fiecare pixel trebuie să găsim punctul întreg cel mai apropiat

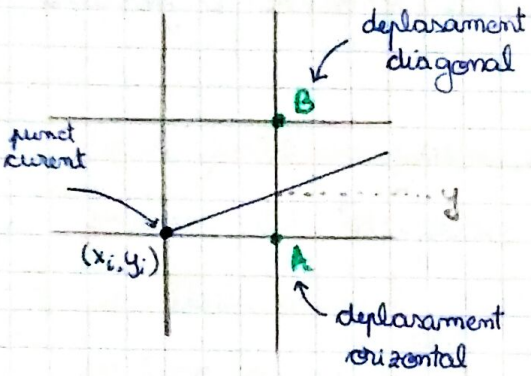
Exemplu: Slide 11 (16. Generare primitive pdf)

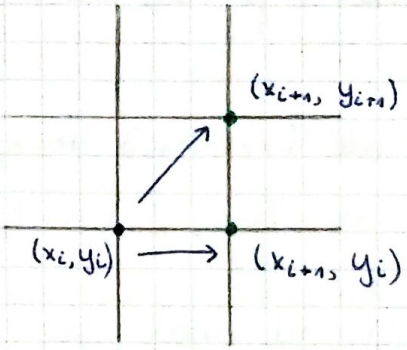


Alegerea se face în funcție de y-ul real (dacă este mai apropiat sau nu de punctul cu y mai mare). Punctul ales devine punct curent

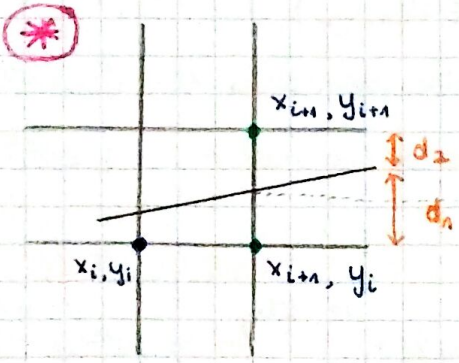
⇒ Se alege punctul roșu de sus (y_{real} mai apropiat de y_{max} față de y_{min}), punctul mov de jos, punctul verde de sus.

întregi





De fiecare dată facem referire față de punctul real de pe dreaptă, însă nu îl calculăm (așa am ajunge iar la operații reale și vrem să nu le mai folosim)



Ne interesează $d(y, y_i)$ și $d(y, y_{i+1})$

- Dacă $d_1 - d_2 < 0 \Rightarrow$ y_{real} e mai apropiat de pixelul (x_{i+1}, y_i)
- Dacă $d_1 - d_2 > 0 \Rightarrow$ y_{real} e mai apropiat de pixelul (x_{i+1}, y_{i+1})

$d_1 - d_2 = 2y - 2y_i - 1$

$y = \frac{dy}{dx} \cdot x_{i+1} + b$

Ne interesează doar semnul diferenței $d_1 - d_2$, nu și valoarea $\Rightarrow$ Ne interesează de fapt $(d_1 - d_2) \cdot dx$ pentru a scăpa de dy/dx care este un calcul real. (dx pantă $\in [0,1] \Rightarrow dx$ pozitiv)

$t_i = (d_1 - d_2) \cdot dx$ = eroare

↑ DOAR ASTA PENTRU EXAMEN (FĂRĂ CALCULELE DE PE SLIDE-UL 14)

Această eroare se calculează pe baza valorii anterioare (se obține o formulă de recurență)

- Dacă $t_i \leq 0 \Rightarrow$ am ales $(x_{i+1}, y_i) \Rightarrow t_{i+1} = t_i + 2dy$
- Dacă $t_i > 0 \Rightarrow$ am ales $(x_{i+1}, y_{i+1}) \Rightarrow t_{i+1} = t_i + 2dy - 2dx$

$\Rightarrow$ Actualizarea se face folosind doar $\underbrace{2dy \text{ și/sau } -2dx}_{\text{constante pentru tot segmentul de dreaptă}}$

CONCLUZIE:

Se alege pixelul următor în funcție de eroarea t_i și ulterior se actualizează eroarea în funcție de punctul ales.

Eroarea inițială se calculează pentru primul punct (x_1, y_1).

$t_i = 2dy - dx$

Avem nevoie de:
- $dx = x_2 - x_1$
- $dy : y_2 - y_1$
- valorile cu care actualizez erorile: $2dy$ / $2dy - 2dx$
- valoarea inițială a erorii: $2dy - dx$

PASUL 1 : Scriem pixelul (x_1, y_1)

PASUL 2 : Aproximări pentru toți pixelii.

Avem pixelul anterior selectat și eroarea curentă

- Dacă eroare $< 0 \Rightarrow$ mergem pe orizontală
- Dacă eroare $> 0 \Rightarrow$ mergem pe diagonală

PANTA $\in [0, 1]$

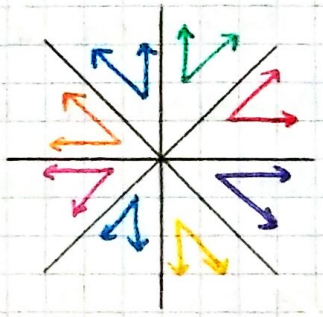
- Actualizare eroare în funcție de eroare < 0
- Actualizare eroare în funcție de eroare ≥ 0

PASUL 3 : Scriem pixelul (x_2, y_2)

Algoritmul de mai sus merge doar pentru segmente cu panta $\in (0, 1)$

$\Rightarrow$ dx mereu pozitiv

Dacă panta nu mai e doar $(0, 1)$, ci orice dreaptă:



- folosim $|dx|$ la eroare
- în toate octanele rămâne deplasarea pe diagonală

$\Rightarrow$ Avem nevoie să ne dăm seama în ce octan ne

aflăm pentru a ști dacă facem incrementare / decrementare

și în ce direcție e diagonala

- $|dx| > |dy|$, altfel le interschimb

PASUL 1 : Se determină octanul

PASUL 2 : În funcție de octan, aleg tipurile de deplasare

PANTA = orice, nu doar $[0, 1]$

TRASARE SEGMENT GENERALIZAT (SLIDE 23)

- pentru segment vertical, baleez y
- pentru segment orizontal, baleez x
- eroare inițială : $t_i = 2|dy| - |dx|$ (cu interschimbarea necesară făcută înainte

$t_{i+1} = t_i + 2|dy|$, dacă $t_i < 0$

$t_{i+1} = t_i + 2|dy| - 2|dx|$, dacă $t_i \geq 0$

ALGORITMUL BRESENHAM PENTRU $m \in [0,1]$ $\quad (x_1, y_1) - (x_2, y_2)$

t - se calculează eroare inițială

SetPixel (x_1, y_1, c)

for ($x = x_1, y = y_1$; $x < x_2$;)

dacă $t \leq 0$, atunci { actualizează $t = t + 2dy$; $x++$

altfel { actualizează $t = t + 2dy - dx$; $x++$; $y++$;

nu sunt necesare pentru examen

SetPixel (x_2, y_2, c)

PENTRU EXAMEN

Dacă avem de descris algoritmul : desenul marcat cu (*), condițiile de actualizare, care sunt pixelii alesi, cum se aleg, cum se generează (dar fără formulele de actualizare)

ALGORITMUL BRESENHAM PENTRU CAZUL GENERAL

PASUL 1 : Segment vertical / orizontal

PASUL 2 : determinare octan

PASUL 3 : Se interschimbă $|dx|$ cu $|dy|$ dacă $|dx| < |dy|$ — avem nevoie de interschimbare, deoarece vrem să baleem x/y (cel care are mai multi pixeli)

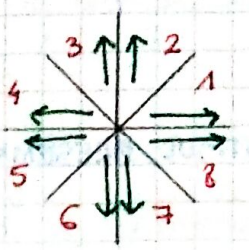
PASUL 4 : Se calculează t

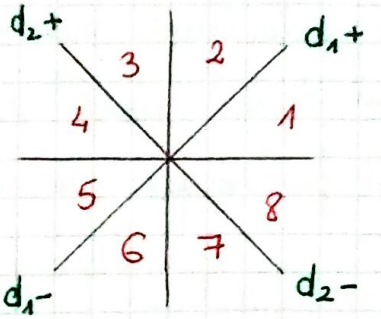
PASUL 5 Se scrie pixel 1.

PASUL 6 : Se generează $x_2 - x_1$ pixeli.

Doar că noi putem parcurge de fapt pe y pentru că am pus valoarea maximă la dx (parcurgem cu dx pe verticală / orizontală).

```
for ( i=1 , x = x1 , y = y1 ; i < |dx| ; i++)
    dacă t ≤ 0 actualizez t
        switch (octant)
            case 1,8 : x++
            case 4,5 : x--
            case 2,3 : y++
            case 6,7 : y--
    altfel actualizez t
        switch (octant):
            case 1,2 : deplasare pe d1+
            case 3,4 : deplasare pe d2+
            case 5,6 : deplasare pe d1-
            case 7,8 : deplasare pe d2-
```





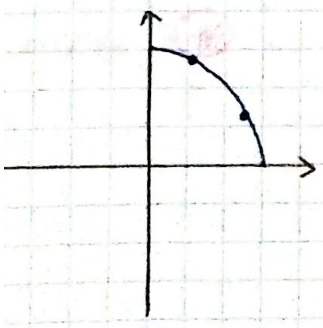
PASUL 7 : Se scrie pixelul 2

Folosind Bresenham obținem doar calcule întregi : înmulțire cu 2, adunare cu 1.

PENTRU EXAMEN

Algoritmul Bresenham generalizat : cum se obțin punctele, descriere algoritm fără valori etc.

ALGORITM DE GENERARE CERCURI SLIDE 30



Considerăm primul cadran

Unim punctele prin drepte.

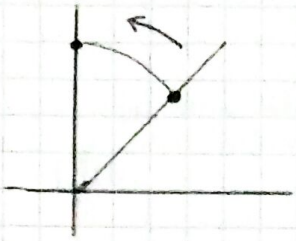
Cu cât mai multe puncte ⇒ mai multe segmente ⇒

⇒ cerc mai exact ⇒ timp mai mare

⇒ Nu avem nevoie de 2π, ci doar de un octan și ne vom folosi de simetria punctelor pe cerc.

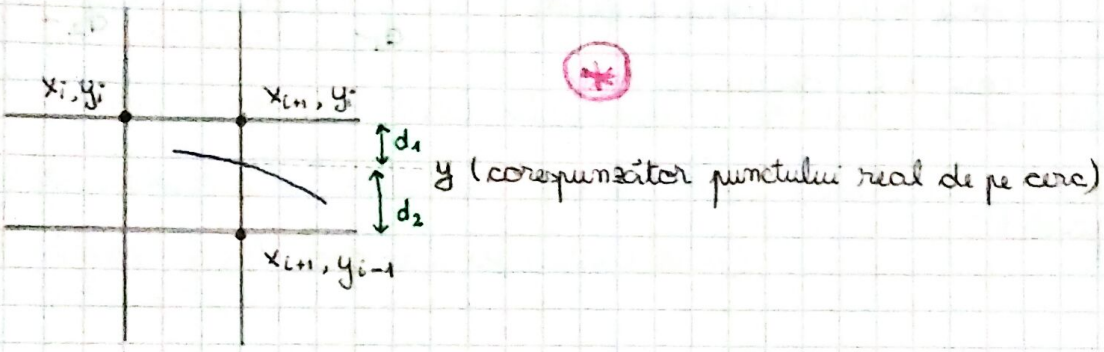
ALGORITMUL BRESENHAM PENTRU CERC

- se bazează pe simetria punctelor și folosește doar un octan
  ⇒ trasează doar acest octan ($45° - 90°$)



Nu generează punctele și apoi le unește, ci calculează punctele corespunzătoare din primul octan și pe restul le calculează prin simetrie.

⇒ Primul punct trasat este $(0, r)$ și apoi se balează arcul de cerc. La fiecare pixel se alege punctul cel mai apropiat de punctul real.



$$\begin{cases} \text{Dacă } t_i \leq 0 \Rightarrow (x_{i+1}, y) \\ \text{Dacă } t_i > 0 \Rightarrow (x_{i+1}, y_{i-1}) \end{cases}$$

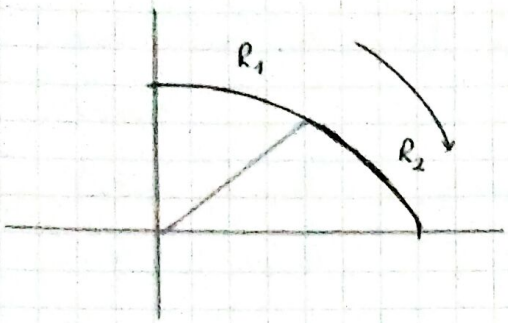
Ca să generez cercul: încep de la $(0, r)$, mă opresc la $x = y$, calculez t_i. În momentul când se calculează un punct, se calculează și celelalte 7 puncte simetrice.

PENTRU EXAMEN

Algoritmul Bresenham pentru generare cercuri: desenul marcat cu (*), eroare, cum se actualizează punctul, etc.

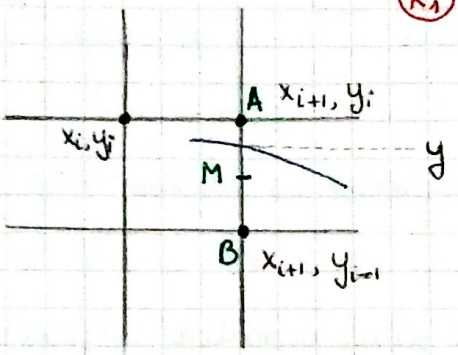
GENERARE ELIPSĂ

Trebuie să generăm punctele corespunzătoare unui cadran ⇒ metoda punctului de mijloc



R_1, R_2 → folosesc pixeli diferiți pentru a determina punctul cel mai apropiat de punctul real

(R_1)



$$F(P) = \begin{cases} = 0 \Rightarrow \text{punct pe elipsă} \\ > 0 \Rightarrow \text{punct exterior elipsă} \\ < 0 \Rightarrow \text{punct interior elipsă} \end{cases}$$

$F(x, y)$ = ecuație elipsă

Se calculează M = mijlocul lui [AB] și în funcție de poziția lui M față de elipsă

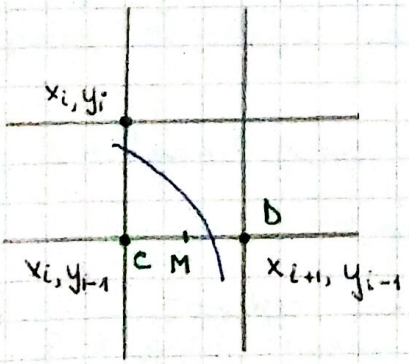
$F(M)$

$$F(M) \begin{cases} < 0 \Rightarrow A(x_{i+1}, y_i) \\ > 0 \Rightarrow B(x_{i+1}, y_{i-1}) \end{cases}$$

~ eroarea de la Bresenham

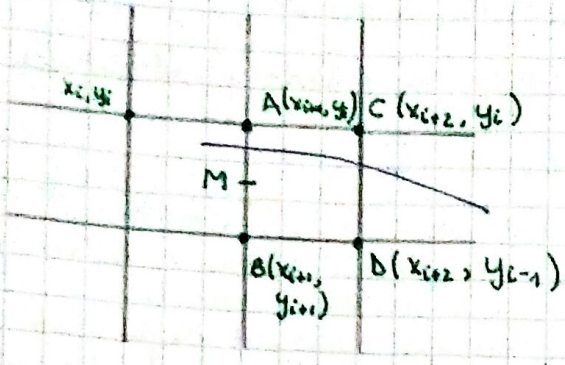
⇒ Se poate calcula $F(M_{urm})$ în funcție de $F(M_{curent})$

(R_2)



M = mijlocul lui [CD]

$$F(M) \begin{cases} < 0 \Rightarrow D(x_{i+1}, y_{i-1}) \\ > 0 \Rightarrow C(x_i, y_{i-1}) \end{cases}$$



$d_i = F(M)$

↳ mijlocul segmentului determinat de cei 2 pixeli posibili

$d_{i+1} = f(d_i)$

$d_{i+1} = F(M_{CD})$

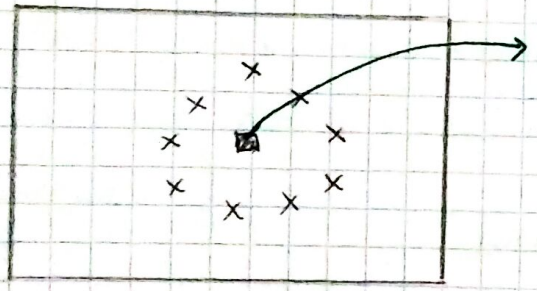
$d_i = F(M_{AB})$

PENTRU EXAMEN

Algoritmul pentru generarea elipselor:

- Se generează punctele din cadranul I, restul prin simetrie
- Se pornește pe elipsă de pe punctul de pe Oy.
- Alegerile se fac în funcție de R_1 sau R_2, desene, mijlocul în fiecare caz, alegeri
- eroare = F(M) + faptul că se poate actualiza la pasul următor în funcție de cea curentă
- Slide 46 (fără actualizare erori)
- Slide 47 (fără actualizare erori)

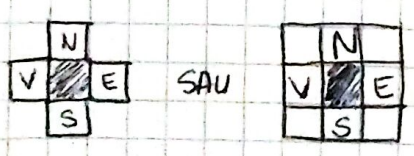
Ca să generăm și interiorul: unim pe verticală/orizontală cele 4 puncte



→ pixel care știm sigur că e în interior

FRAME BUFFER

Orice pixel care nu e colorat și este în interior, trebuie colorat.



⇒ folosesc recursivitatea

GENERAREA SUPRAFEȚELOR DEFINITE GEOMETRIC

SLIDE 58

Baleem poligonul cu linii orizontale.

(i4) Când găsim intersecțiile x_1 și x_2, trasez segmentul cu y = valoarea liniei de baleere.

(i2) Găsim punctele de intersecție x_1, x_2, x_3, x_4 și apoi trasăm segmentele $x_1 - x_2$ și $x_3 - x_4$.

SLIDE 59

Parcurgem poligonul în sensul $\overrightarrow{ABCDA}$.

Se obțin următoarele puncte de intersecție: x_1, x_2, x_3, x_4 (în această ordine). Se observă că dacă unim 2 câte 2 (x_1 cu x_2 și x_3 cu x_4) nu se obține rezultatul corect.

SOLUȚIE: Ordonăm crescător după x și după unim primele 2 și ultimele 2.

$x_1, x_2, x_3, x_4 \xrightarrow{\text{ordonare}} x_1, x_4, x_2, x_3 \longrightarrow$ unim x_1 cu x_4 și x_2 cu x_3

SLIDE 60

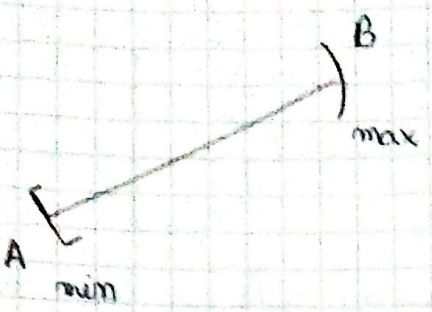
(i3) Vârful B nu este folosit

(i4) Nu trebuie folosit C $\Rightarrow$ $x_1 \to x_2$, $x_3 \to x_4$

Totuși, dacă $\measuredangle BCD$ este mare se va vedea că lipsește C, deci trebuie să găsim o soluție pentru a afișa C.

B → punct maxim (segmentele cu celălalt capăt mai mic)
↳ nu se iau în considerare la trasare

C → punct minim (segmentele au celălalt capăt mai mare)
↳ se iau în considerare
O să punem duplicate în lista de intersecție

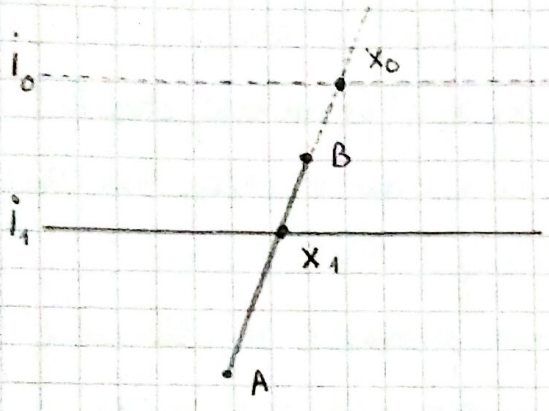


x_1, x_2, C, C, x_3, x_4

B este trasat prin segmentul $x_1 \rightarrow x_2$

Ne interesează doar laturile active atunci când calculăm punctele de intersecție

↳ care îl au $y_{max} < y_{baleere}$



$$x_1 = x_0 - \frac{d}{m}$$

x_0 → obținut prin intersecția prelungirii laturii AB cu dreapta de baleere anterioară

Poligon:

lista de laturi l_i

pentru fiecare l_i : m

$$x_0 = x_{y_max} + \frac{d - y_{max}}{m} + \frac{y_A}{m}$$

y_A → nu îl știm

x_{y_max} → inițial

$$(x_{y_min}, y_{min}) \leq l_i \leq (x_{y_max}, y_{max})$$

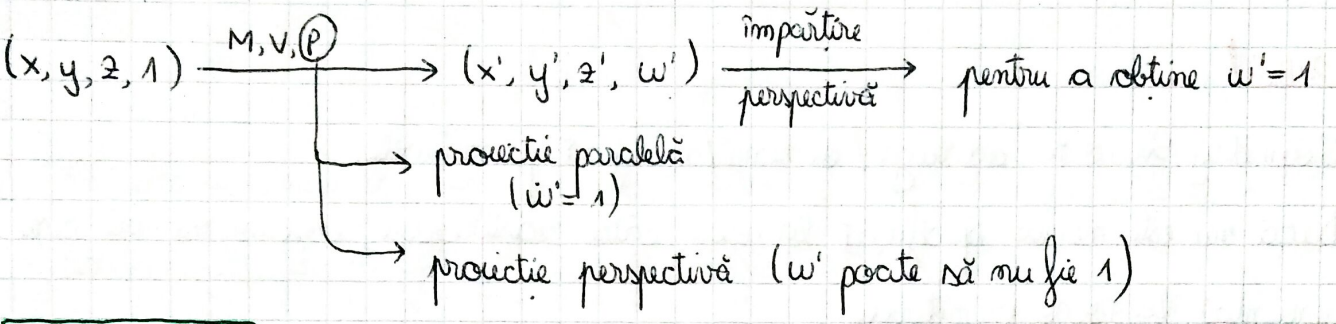
$$x_{i+1} = x_i - \frac{d}{m}$$

- pentru fiecare y_k (de la y_{max} poligon la y_{min} poligon) cu pas = d
  - pentru fiecare latură l_i verifică dacă intersectează y_k (la prima intersecție)
  - actualizez x_0 pentru l_i : $x_0 += \frac{y_k}{m}$ ← doar prima dată
  - se marchează toate laturile pentru care nu există intersecție cu y_k

  lista laturilor active care intersectează y_k
- pentru fiecare latură activă l_i se calculează x_{i+1}
- ordonare crescătoare după x ale punctelor de intersecție
- se unesc punctele de intersecție câte 2 (x_1 cu x_2, x_3 cu x_4 etc.)

CURS 12

După faza de asamblare a vârfurilor, există GEOMETRY SHADER.



VERTEX SHADER

- în cazul în care se face calcul de iluminare: normala se transformă pe baza matricei M (ne interesează doar primele 3 linii și primele 3 coloane). În cazul în care se fac calcule în spațiul de vizualizare, transformăm pe baza M și V.
- dacă vreau să am un obiect plin cu o textură: Vertex Shader-ul trimite mai departe coordonatele de texturare (așa cum sunt sau le modifică înainte de a fi trimise)

GEOMETRY SHADER

- gl_Position din Vertex Shader trebuie mapat în vectorul gl_in care reprezintă intrarea în GS.

- face transformările în spațiul obiect și apoi aplică M, V, P
- se specifică primitiva de intrare, cum vor fi unite vârfurile și câte vârfuri vor rămâne după unire.

INTRARE

- GL_POINTS
- GL_LINES: 1 cu 2, 2 cu 3 etc.
- GL_LINE_LOOP: specifică conturul unui poligon (segmente unite câte 2 puncte)
- GL_LINE_STRIP: la fel ca GL_LINE_LOOP, doar că nu mai unește primul punct cu ultimul punct
- GL_TRIANGLE_FAN: primul punct este folosit pentru fiecare triunghi
- GL_TRIANGLE_STRIP: 1, 2, 3 ; 2, 3, 4 etc.

EXEMPLUL 1 (SLIDE 7 – GEOMETRY SHADER)

Se obține un obiect colorat uniform. (GS nu modifică nimic, ci doar trimite mai departe).

! Culoarea putea fi setată și în Vertex Shader, dacă o setam pentru fiecare vertex.

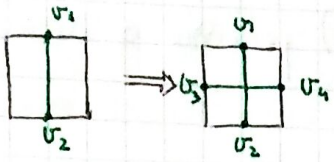
OBSERVAȚII!

- Geometry Shader adaugă / elimină / modifică primitivele.
- Dacă nu este nimic specificat la ieșire către rasterizare, înseamnă că am renunțat la toate primitivele.
- Se specifică atunci când se termină o primitivă pentru a ști când începe alta

  EmitVertex() → am terminat coordonata corespunzătoare lui gl_Position, dar modificată de Geometry Shader ⇒ trece în etapa de rasterizare

  EndPrimitive() → specifică faptul că am terminat de generat primitiva curentă și că o pot începe pe următoarea

EXEMPLUL 2 (SLIDE 8 - GEOMETRY SHADER)



v_1 și v_2 se găsesc în gl_in.

Se parcurge gl_in și se generează la ieșire linia care are capetele v_1 și v_2 (primul for din cod).

Pentru a genera v_3 și v_4: din v_1 vom obține v_3 și din v_2 vom obține v_4: parcurgem gl_in, prelucrăm v_1 și v_2 pe rând și pentru fiecare interschimbăm x cu y.

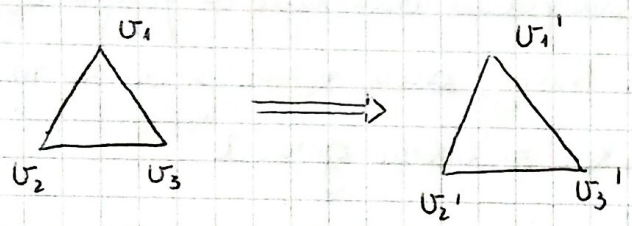
$v_3 = v_1.yx$

$v_4 = v_2.yx$

$\Rightarrow$ al doilea for din cod

EXEMPLUL 3 (SLIDE 9-11 - GEOMETRY SHADER)

Se primește un triunghi și vrem ca la ieșire să avem triunghiul și încă un triunghi (identic cu primul), dar translatat.



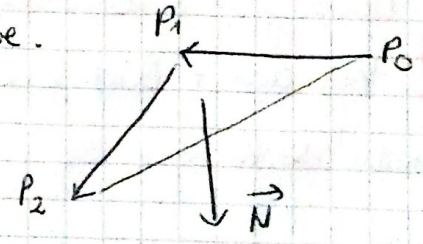
Slide 11 primul for $\Rightarrow$ triunghiul inițial

Slide 11 al doilea for $\Rightarrow$ triunghiul translatat

! Slide 10: deoarece la ieșire o să mai fie un triunghi în plus, max_vertices va deveni 6.

MESH EXPLODER

Translatăm fiecare față pe direcția normalei respective. Deoarece normala nu este primită la input, o putem calcula ca fiind produsul vectorial dintre vectorii corespunzători a 2 laturi.

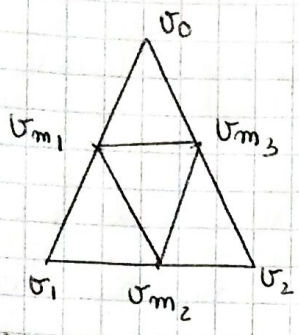


$\vec{N} = \vec{V_1} \times \vec{V_2}$

DIVIZAREA SUPRAFETELOR

Geometry Shader-ul primește o față (un triunghi) pe care o divizează în mai multe fețe.

EXEMPLU: Subdivizare Loop ⇒ fiecare triunghi este divizat în 4 triunghiuri



Se folosesc mijloacele laturilor: v_{m_1}, v_{m_2}, v_{m_3}

Geometry Shader:

- primește coordonatele pentru v_0, v_1, v_2
- calculează cele 3 mijloace: $v_{m_1}, v_{m_2}, v_{m_3}$
- se calculează noile culori pentru $v_{m_1}, v_{m_2}, v_{m_3}$. Acestea trebuie calculate doar dacă v_0, v_1, v_2 au culori diferite sau dacă vrem să aplicăm un model de iluminare. Dacă nu este niciunul din aceste cazuri, atunci se poate omite etapa, deoarece se va face interpolare între culorile pentru v_0, v_1, v_2.
- dacă se dorește aplicarea modelului Phong, avem nevoie și de normalele fiecărui fragment în Fragment Shader ⇒ trebuie generate.

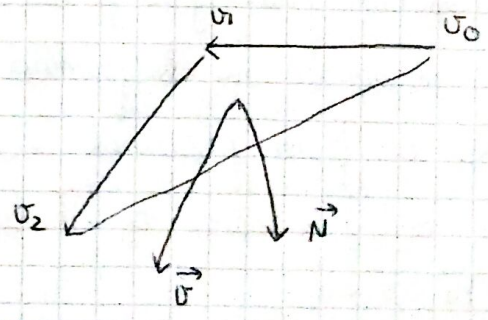
```
in VertexAtribute
{
    vec4 color;      ← trebuie primit de GS de la VS
    vec3 normal;
} vertex[]
```

ELIMINARE PĂRȚI NEVIZIBILE DINTR-UN OBIECT

Se folosește Back-face culling.

Dacă fața e orientată spre observator: $\measuredangle(\vec{N}, \vec{v}) \in [0, 90°]$

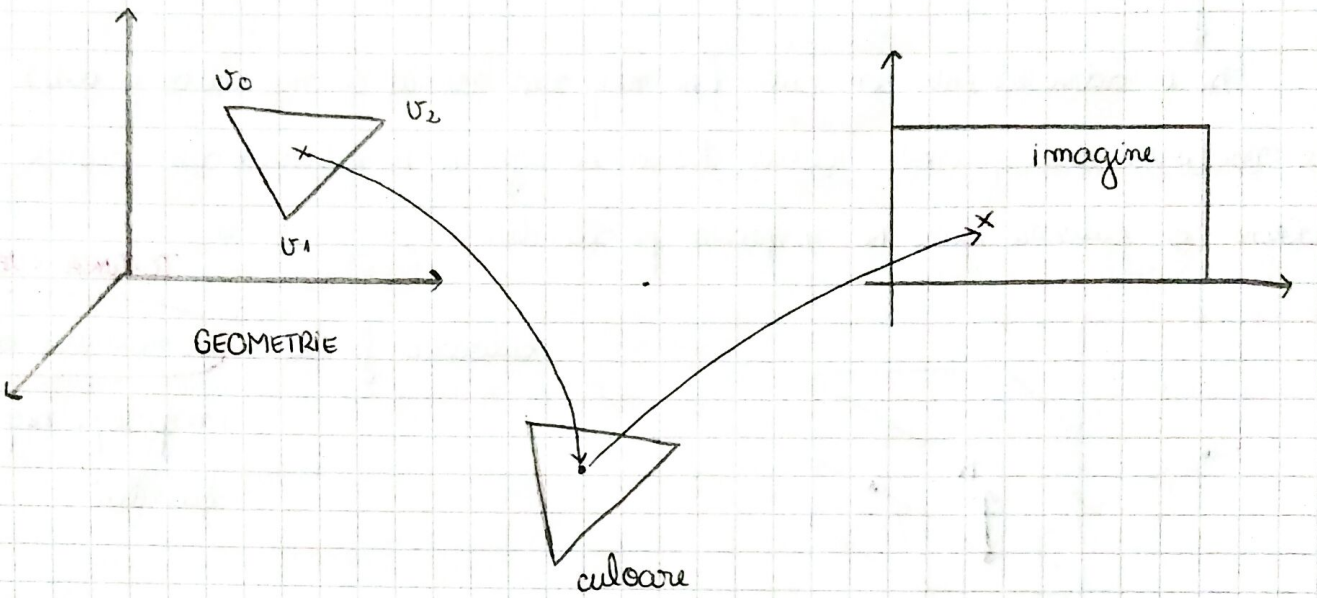
! Vezi curs 7 Back-face culling pentru mai multe detalii.

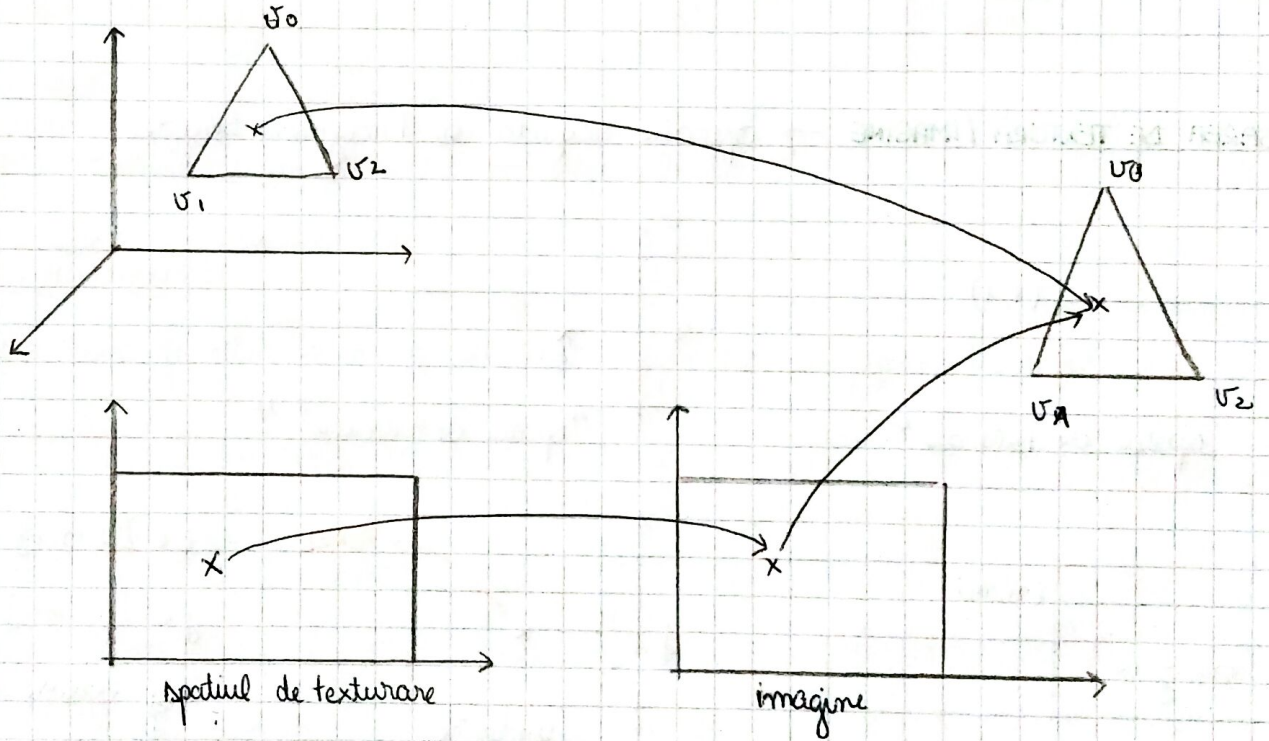


PENTRU EXAMEN

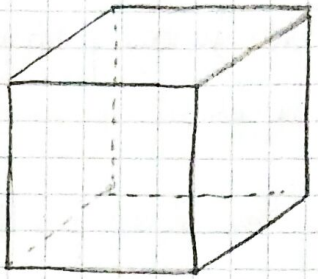
Geometry Shader: care sunt transformările (pseudocod), ce parametrii sunt in/out, ce reprezintă fiecare parametru, cum se face legătura cu faza de interpolare, rasterizare etc.

TEXTURI





ENVIRONMENT MAPPING



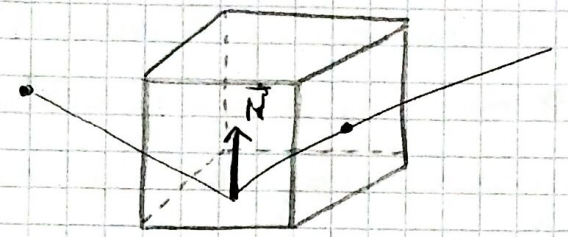
↓

Se încadrează într-un cub (se mapează mediul înconjurător pe cub)

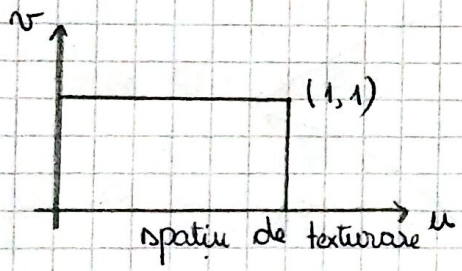
Observatorul care privește cubul. Raza se reflectă și intersectează cubul ⟹ Se obține o culoare care se mapează pe cilindru.

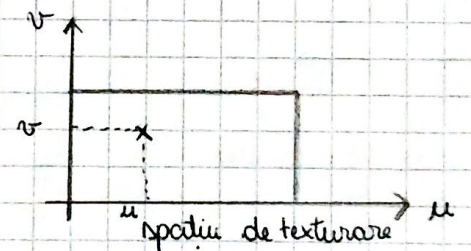
TEXTURĂ CUBICĂ

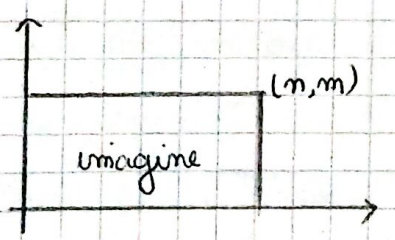
culoarea finală = combinație de 6 texturi mapate pe suprafața cubului

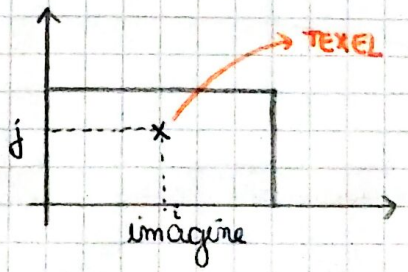


ACCESAREA DE TEXTURĂ / IMAGINE → operație rapidă în Fragment Shader









$i \in [0, n-1]$

$j \in [0, m-1]$

i, j valori reale

Facem o scalare pentru a aduce (u, v) în (i, j). Dacă se obțin valori reale:

(I) putem rotunji, dar nu se obțin rezultatele așteptate

(II) interpolare biliniară (mai costisitoare)

Cea mai folosită metodă: MAPARE INVERSĂ

(x, y, z) —M,V,P / împărțire perspectivă→ cub de latură 2 (x', y', z') —renunț la z'→ ecran $(x', y', \cancel{z'})$

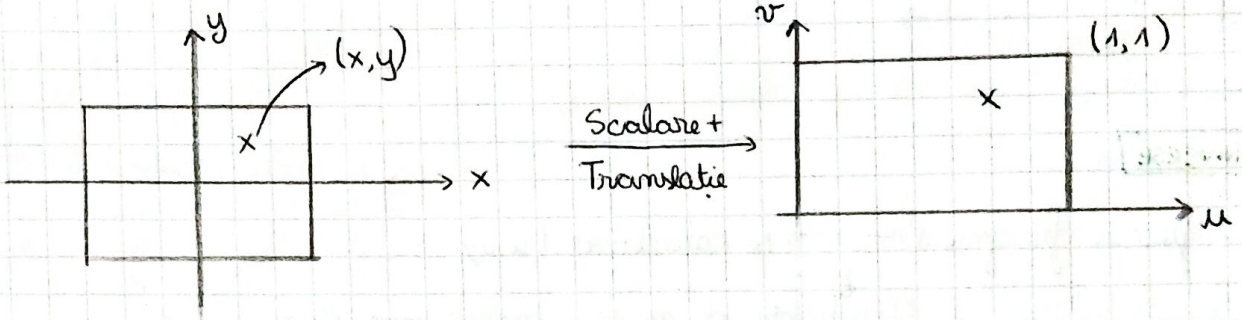
spațiu obiect

(x', y', z') —funcție de mapare inversă→ (u, v)

$\in$ spațiu real (cub de latură 2, centrat în origine)

mapare →
- plană
- cilindrică
- sferică
- cubică (maparea mediului înconjurător)

MAPARE PLANĂ



MAPARE CILINDRICĂ

u este deja în $[0,1]$

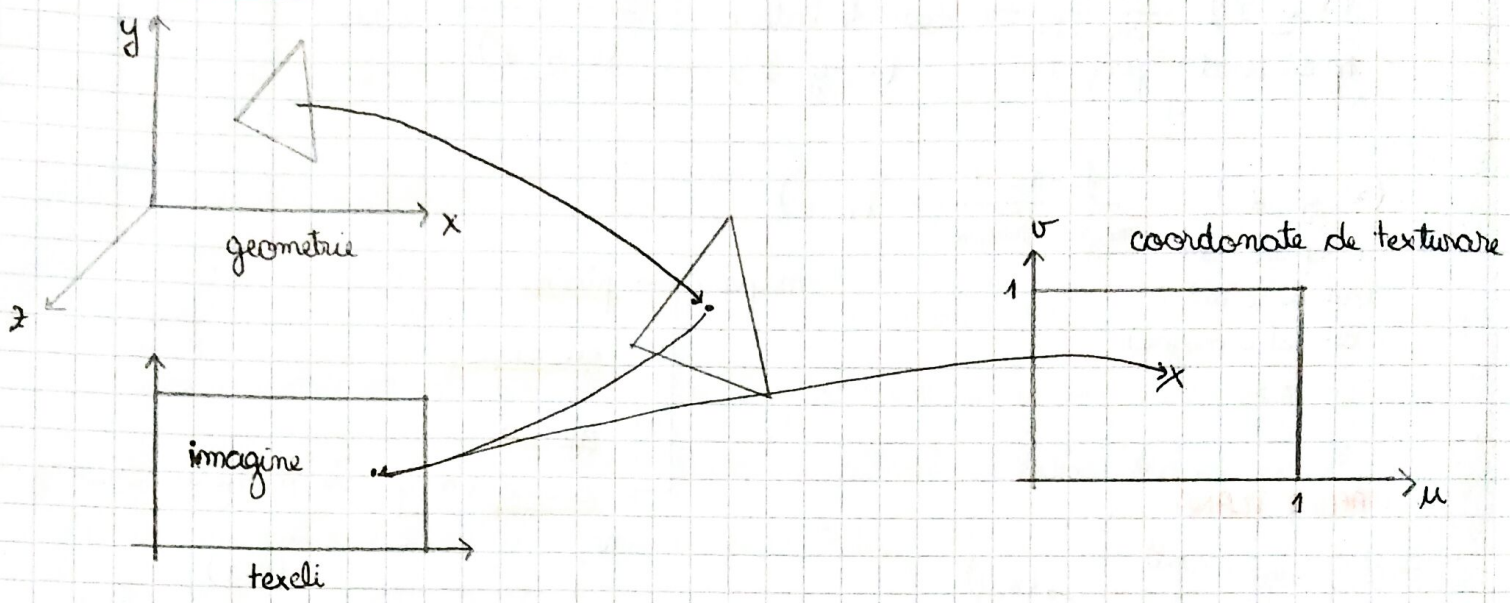
θ trebuie adus în $[0,1] \Rightarrow$ împărțire la 2π

PENTRU EXAMEN

Care sunt funcțiile de mapare de bază și ce fac ele? (fără formule)

CURS 13

MAPAREA TEXTURILOR



MAPARE INVERSĂ

pentru fiecare fragment ⟶ se calculează (u,v)

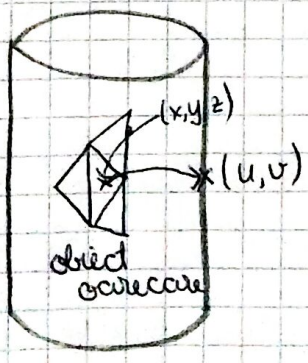
↓ funcție de mapare inversă →
- plană
- cilindrică
- sferică
- cubică (folosită pentru maparea mediului înconjurător)

MAPARE PLANĂ

Este echivalentă cu maparea unui plan de o imagine.

MAPARE CILINDRICĂ

Pentru fiecare punct de pe cilindru, se găsește punctul corespunzător mapat în $[0,1]$. (cilindrul încadrează obiectul).



Pentru fiecare fragment din cilindru/sferă/plan pot să obțin coordonatele de texturare (u,v)

MAPARE ÎN 2 PAȘI

① MAPARE O : mapare pe o suprafață cunoscută care înconjoară obiectul pe care vreau să aplic textura (mapare pe o față intermediară)

Pentru fiecare fragment (x_i, y_i, z_i) care aparține suprafeței intermediare, se găsește coordonata de texturare (u,v)

② MAPARE S : mapare pe suprafața obiectului de texturat (găsesc fragmentul (x,y,z) care aparține obiectului de texturat care se mapează în (x_i, y_i, z_i))

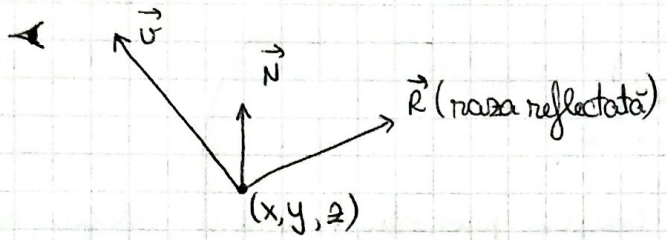
(x_i, y_i, z_i) = punctul de pe suprafața intermediară

$(x,y,z) \longrightarrow (x_i, y_i, z_i) \longrightarrow (u,v)$

(x,y,z) = spațiu obiect ②

(x_i, y_i, z_i) = suprafața intermediară ①

METODA 1 (rază reflectată)



Raza reflectată va intersecta suprafața intermediară. Punctul de intersecție este (x_i, y_i, z_i)

Această metodă este folosită la maparea mediului înconjurător

METODA 2

Am fragmentul (x,y,z) și mijlocul obiectului. Pentru a obține punctul de texturare, intersectăm vectorul cu suprafața intermediară.

FILTRAREA TEXTURII

Trebuie să ajungem de la coordonatele de texturare reale la coordonate întregi. Există 2 metode :

① Trunchiere la întreg / Aproximare cu cel mai apropiat întreg

② Interpolare bilaterală (în funcție de 2 vecini)

① = CLAMPING

(u', v') cu care accesez imaginea (după filtrare) : ne interesează doar valorile

între 0 și 1, restul se ignoră

② = WRAPPING

Se aplică aceeași textură și în afară (culorile sus și jos ar trebui să fie la fel pentru a nu se vedea diferența)

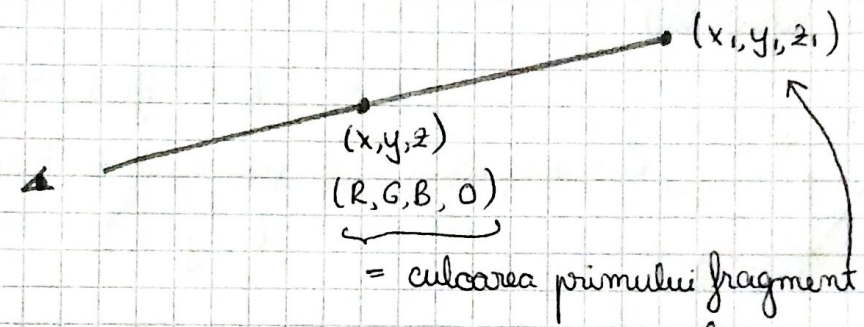
R, G, B A = transparență

→ = 1 ⇒ culoare opacă → afișat în culoarea fragmentului

→ = 0 ⇒ complet transparentă ⇒ culoare finală = ?

→ > 0 și < 1 ⇒ transparență parțială ⇒ culoare finală = ?

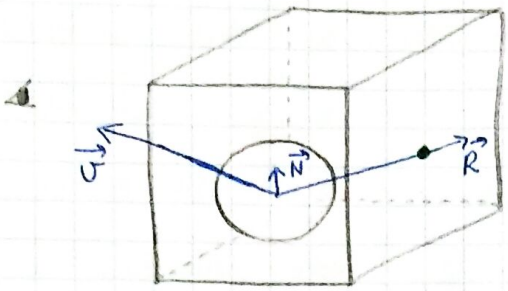
- Dacă fragmentul este transparent complet, atunci se va afișa fragmentul din spatele lui.



= culoarea primului fragment netransparent

- Dacă fragmentul este partial transparent, atunci se va face interpolare între următoarele fragmente:
  - dacă al doilea fragment este opac, se face interpolare între primul obiect și al doilea
  - dacă al doilea obiect are transparență, se face interpolare între primul fragment, al doilea, al treilea ...
    → până la primul fragment opac
    → până se ajunge la nivelul maxim setat de noi (maxim ... fragmente)

ENVIRONMENT MAPPING



Raza reflectată ($\vec{R}$) intersectează cubul într-un punct. → culoarea din mediu care va fi mapată pe punctul respectiv pe sferă

ETAPE :

PASUL 1 : Obiectul pe care se redă mediul înconjurător (O) + scena S (din jurul obiectului

PASUL 2 : Se încadrează O cu un cub

PASUL 3 : Se generează imaginea corespunzătoare scenei S pe fiecare față a cubului (proiecție plană pe fiecare <u>față de afișare</u>)

Fața cubului

Observatorul se află în centrul cubului

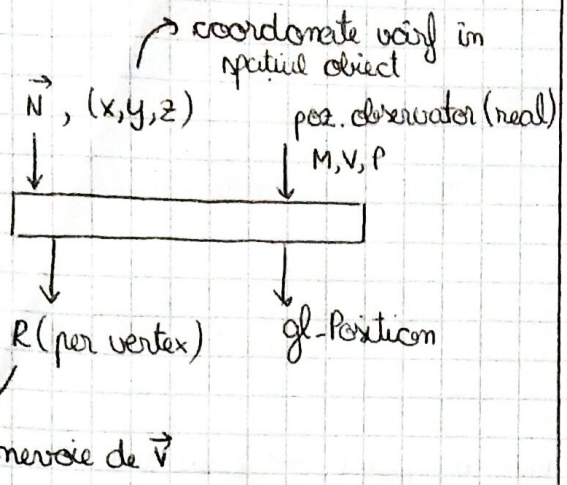
Se obțin 6 imagini care formează o <u>textură cubică</u>. Accesarea acestei texturi nu se mai face în 2D, ci trebuie să găsesc fața și după să determin texelul. ⇒ <u>coordonate de texturare 3D</u> (fiecare componentă ∈[0,1])

PASUL 4 : Se generează fiecare fragment corespunzător obiectului O (x, y, z)

PASUL 5 : Se calculează $\vec{V}, \vec{N}, \vec{R}$ în punctul (x, y, z) și R accesează textura cubică cu ajutorul căreia obțin culoarea care se mapează pe (x, y, z)

Știm : poziția observatorului, $\vec{N}$, ⇒ $\vec{R}$ se calculează în funcție de acestea

VERTEX SHADER | FRAGMENT SHADER



în R

$R = reflect(V, N)$

vrem să le aducem pe ambele în spațiul real

$\vec{V}(real) =$ poziție observator $-$ (x,y,z)

poziție observator: deja în spațiul obiect

(x,y,z): spațiu obiect ⇒ trebuie convertit în spațiul real

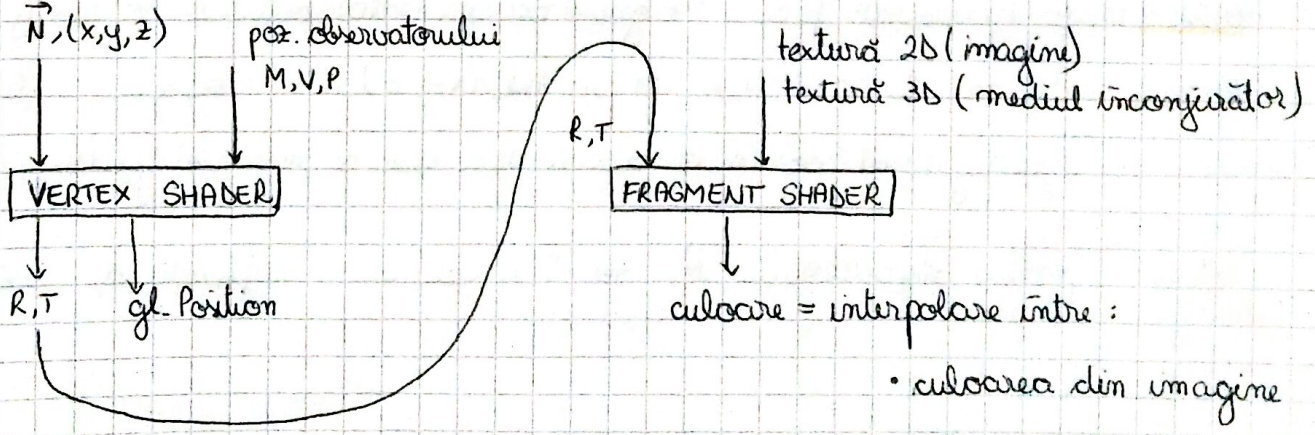
$\Downarrow$

$\vec{V}(real) =$ poziție observator $- M \cdot (x,y,z)$

$\vec{N}(real) = (M) \cdot N$

↳ ne interesează doar primele 3 linii

Pentru maparea mediului înconjurător se poate ține cont și de raza refractată.



PENTRU EXAMEN

- MAPARE INVERSĂ:
  - fără formule
  - care sunt pași pentru mapare
  - care sunt mapările
- MAPAREA MEDIULUI ÎNCONJURĂTOR: parametrii fragment shader, vertex shader, descriere, specificare parametrii, care sunt parametrii care trec prin interpolare

ILUMINARE

Modelele de iluminare se modifică dacă ținem cont de lumina reflectată/refractată.

ILUMINARE GLOBALĂ

- Zona de afișare ≡ planul față de la volumul de vizualizare
- Se pornește invers: se duc raze de la observator la obiecte și apoi le reflect pe direcția luminii.

Generez raze de la observator care se mapează în spațiul 3D ⇒ cel mai apropiat punct va fi afișat cu iluminare.

RAY CASTING

Pentru a calcula culoarea, avem nevoie de N (necesară pentru componenta difuză și cea speculară.

CONSTRUCȚIE RAZĂ (SLIDE 6 – Ray Tracing.pdf)

P_o = poziție observator

v – trebuie să baleeze pentru a determina fiecare punct

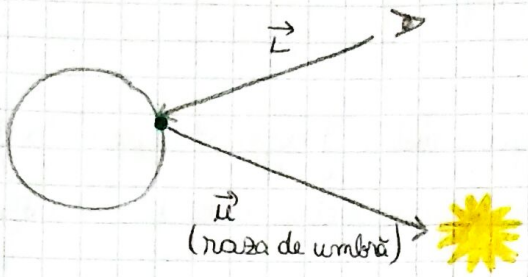
INTERSECȚIE (SLIDE 7 – Ray Tracing.pdf)

Se folosesc structuri de accelerare pentru a vedea dacă o rază intersectează sau dacă poate intersecta. Considerăm un obiect înconjurător. Dacă raza nu intersectează corpul înconjurător, atunci nu intersectează nici obiectul inițial

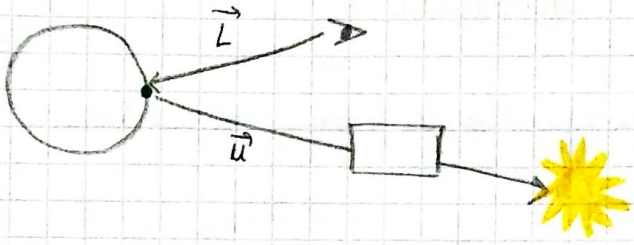
UMBRA (SLIDE 8 - RAY TRACING.pdf)

$\vec{L}$ = raza de lumină

$\vec{v}$ = raza reflectată



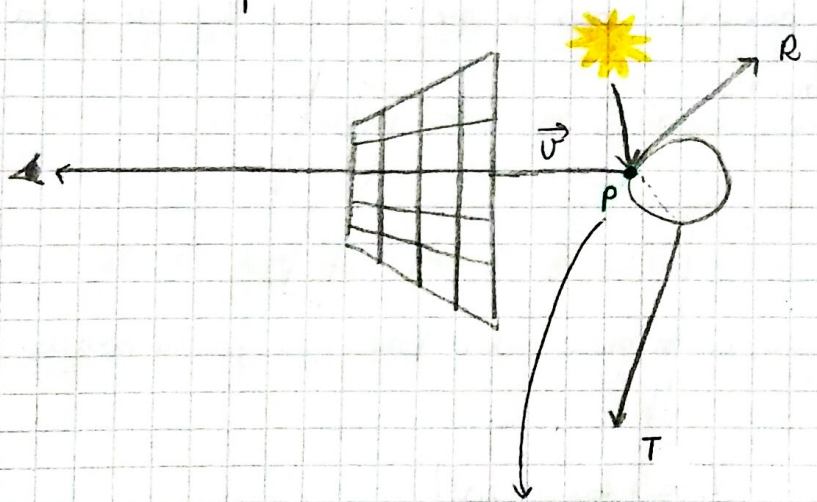
Dacă $\vec{u}$ nu intersectează nimic, atunci îl afișez cu lumină



Dacă $\vec{u}$ intersectează un obstacol, punctul trebuie să fie afișat în umbră.

RAY TRACING

- Razele de lumină pornesc de la sursa către observator



În acest punct se obțin R și T care se propagă în scenă și pot provoca la rândul lor iluminare asupra unor obiecte

Ca să calculez iluminarea în P, avem nevoie de:

- $\vec{V}$
- iluminarea locală { - componenta ambientală
  - componenta difuză
  - componenta speculară
- iluminare globală : R, T

  Aparent acestea nu influențează P, dar se pot propaga astfel încât să ajungă înapoi în punctul P.

Intensitatea razelor R și T scade la fiecare intersecție cu un obiect. Acestea se propagă până când nu mai e vizibil efectul sau putem impune un număr maxim de propagări

K_T = coeficient de transparență al razei refractate

SLIDE 19-21 (Ray Tracing.pdf)

Se obține primul punct de intersecție folosind $\vec{V}$. Asupra acestuia se aplică următoarele modificări:

- iluminare locală
- se folosește $\vec{U}$ pentru a verifica dacă este în umbră sau nu
- se aplică iluminare globală { - propagare rază R în scenă
  - propagare rază T în scenă

⇒ Se generează un arbore de raze (fiecare nod are 2 copii : R și T)

SLIDE 22

Se poate limita adâncimea arborelui de raze ⇒ nivel maxim la arbore ⇒
⇒ număr limitat de propagări

SLIDE 27 - VOLUME ÎNCADRATOARE

Folosim drept volum încadrator un paralelipiped. Intersectăm raza cu fețele paralelipipedului (mai rapid decât intersecția cu fiecare față a iepurelui) Dacă nu intersectează nicio față a paralelipipedului, atunci nu intersectează nici

ieșirele.

! Volumele încadratoare trebuie alese astfel încât să rămână cât mai puțin loc liber.

SLIDE 29 - IERARHII DE VOLUME ÎNCADRATOARE

A, B - grupate în volumul 2
D, E, F - grupate în volumul 3
C
} grupate în volumul 1

SLIDE 30 - INTERSECȚIA CU RAZA

Avem arborele pentru volumele încadratoare de pe slide.

Verificăm dacă $\vec{v}$ intersectează rădăcina:

$\vec{v} \cap 1 \neq \emptyset \Rightarrow$ Fac intersecții cu fiii rădăcinii:

$\vec{v} \cap 2 \neq \emptyset \Rightarrow$ se continuă intersecția

$\vec{v} \cap 3 = \emptyset \Rightarrow$ STOP

$\vec{v} \cap C \neq \emptyset \Rightarrow$ se continuă intersecția

$\vec{v} \cap C \neq \emptyset \Rightarrow$ Continui intersecția cu fiii $\Rightarrow \vec{v} \cap$ obiect C = punct intersecție $\Rightarrow$ STOP

$\vec{v} \cap 2 \neq 0 \Rightarrow$ Continui intersecția cu fiii

$\vec{v} \cap A = \emptyset \Rightarrow$ STOP

$\vec{v} \cap B \neq \emptyset \Rightarrow \vec{v} \cap$ obiect B = punct intersecție $\Rightarrow$ STOP